

Anhang: Befundkatalog Ist-Analyse JANS AI Hub (intern)

Raphael Jans Architekten ETH/SIA · Stand 19.09.2026 · Anhang zum Briefing «JANS AI Zentrum» ·
**Vertraulich. Weitergabe an den Ingenieur erst nach unterzeichneter
Geheimhaltungsvereinbarung.**

00 Lesehilfe

00.1 Herkunft. Zehn parallele, rein lesende Analyse-Läufe vom 19.09.2026 über den Git-Spiegel des Hub (Commit 379a4ba76) und die nicht versionierten NAS-Ordner. Jeder Befund trägt einen Beleg (Datei, Zeile oder Abschnitt) oder eine datierte Eigenmessung. Inhalte von Secret-Dateien wurden nicht gelesen; IP-Adressen, Kennungen und Personennamen Dritter sind bewusst nicht übernommen.

00.2 Aufbau. Je Subsystem: Ist-Zustand, Schwachstellen nach Schweregrad (kritisch, hoch, mittel, niedrig) mit Klasse und Beleg, Kennzahlen mit Quelle, offene Fragen. Die aus den Befunden abgeleiteten Leistungspakete stehen im Briefing, nicht hier.

00.3 Zählung. 120 Schwachstellen über zehn Subsysteme. Mehrere Befunde erscheinen aus verschiedenen Blickwinkeln mehrfach (SMB-Mount, M365-App-Rechte, Personendaten im Git, fehlende Tests); das ist Absicht und zeigt die Querschnittsthemen.

01 Infrastruktur und Netz

01.0 Ist-Zustand. Die Infrastruktur besteht aus drei Macs (Mac mini M2 Pro 32 GB als Always-On-Station, MacBook Pro M2 Pro 16 GB mobil, ein drittes MacBook Pro mit 32 GB), einer Synology DS918+ als Dateiserver und kanonischer Hub-Bibliothek, einer OPNsense-Firewall und Tailscale als einzigem Fernzugang. Einen dedizierten Server oder Linux-Host gibt es nicht: der Mac mini ist zugleich CAD-Arbeitsplatz (ArchiCAD, Cineware), Träger der Nachtschicht, Dispatch-Endpunkt und Tailscale-Subnet-Router. Rückgrat der gesamten Automatisierung ist ein SMB-Mount: Skills, Rules, Wissensbasen, Logbuch, Task-Queues und Heartbeat-Dateien liegen auf der Netzfregabe, 696 Pfadzeilen in 229 Dateien zeigen darauf. Weil der macOS-SMB-Client im Leerlauf abreisst (bis 346 Stalls an einem Tag), Git über SMB hängt, launchd-Jobs an TCC scheitern und 16 bis 32 GB RAM durch parallele Claude-Sessions und OneDrive erschöpft wurden, ist eine Schicht von rund 22 Wächter- und Heil-Scripts (4'077 Zeilen Bash) entstanden. Alle Dienste sind LaunchAgents im User-Kontext; mit aktivem FileVault startet nach einem Stromausfall nichts ohne physische Anmeldung. Das NAS-Volume ist zu 93 % belegt. Alarmiert wird über Apple Mail per osascript. Der Hub hat seine Schwächen selbst vorbildlich dokumentiert (Betriebs-Chronik, Hub-Audit 12.08.2026, Systemanalyse 11.09.2026 mit Zielbild «Git als einzige Quelle»), setzt die eigenen Massnahmen aber nur teilweise und mit Wochen Verzug um, weil Systemeingriffe bewusst beim Inhaber liegen und kein Konfigurationsmanagement existiert.

01.1 Schwachstellen

01.1.1 SMB-Mount ist das Rückgrat der gesamten Automatisierung (kritisch, SPOF / Architektur). Skills, Agenten, Rules, Wissensbasen, Logbuch, Task-Queues und Heartbeat-Dateien liegen auf einer SMB-Freigabe, auf die drei Stationen und Dutzende Nachtläufe zugreifen. Reisst der Mount ab, ist die Station für den ganzen Hub blind. Der macOS-SMB-Client reisst im Leerlauf regelmässig ab; Git über SMB hängt ununterbrechbar und wurde deshalb verboten und durch einen Eigenbau-Committer auf der Synology ersetzt. Der Hub hat dies selbst als Konstruktionsfehler diagnostiziert, der Umbau (V0+) ist seit 11.09.2026 nur teilweise umgesetzt. Beleg: docs/konzepte/260911-Systemanalyse-Hub-Architektur-NAS-Git/260911-Systemanalyse-Hub-Architektur-NAS-Git.md Abschnitte 00.1, 01.2 (696 Pfadzeilen in 229 Dateien), 02.1 (bis 275 Stalls/Tag, 5.5 h NAS-Ausfall 07.09.), 03.1 bis 03.4;

rules/betrieb-chronik.md Abschnitte 260831, 260831b, 260901, 260911b; heute gemessen: 346
STALL-Zeilen am 17.09.2026 in ~hub/.git/nas-keepalive.log

01.1.2 Always-On-Station übersteht keinen unbeaufsichtigten Neustart (kritisch, SPOF / Verfügbarkeit). Alle JANS-Dienste auf dem Mac mini sind LaunchAgents im User-Kontext, FileVault ist aktiv, Auto-Login nicht gesetzt. Nach Stromausfall, Kernel-Panic oder macOS-Update bleibt die Maschine am FileVault-Prompt stehen: kein Mount, keine Nachtschicht, kein Sync, kein Tailscale-Subnet-Router, kein Fernzugang aufs Büro. Eine USV ist nirgends dokumentiert. Die im Hub vorgesehene Gegenmassnahme «FileVault aus plus automatische Anmeldung» wäre ein Sicherheitsabbau auf einem Gerät mit Kundendaten und Tenant-Zertifikat und gehört fachlich beurteilt. Beleg: docs/konzepte/260812-Hub-Audit/260812-Hub-Audit.md Abschnitt 2.1 und Risiko R1; Systemanalyse 260911 Abschnitt 06.2; heute per ssh lesend gemessen auf dem Mac mini: «FileVault is On», 0 JANS-Einträge in /Library/LaunchDaemons, 14 LaunchAgents, autorestart 1; Repo-weite Suche nach USV/UPS ohne Treffer

01.1.3 Server und CAD-Arbeitsplatz sind dieselbe Maschine; kein dedizierter Host (hoch, Fehlende Trennung / Consumer-Hardware). Der Mac mini trägt Nachtschicht, Dispatch, Cockpit-Server und Subnet-Routing und ist gleichzeitig der interaktive Hauptarbeitsplatz mit ArchiCAD, Cineware und OneDrive. Arbeitsplatz-Last, Reboots und GUI-Dialoge treffen damit direkt die Serverrolle. Einen headless Linux-Host gibt es nicht, obwohl Claude Code Remote-SSH nur Linux-Hosts unterstützt und systemd, cgroups und unbeaufsichtigter Boot dort Standard wären. Beleg: docs/konzepte/260728-Speicher-Architektur/260728-Speicher-Architektur.md Abschnitt «Mac Mini (32 GB)» (OneDrive 33 GB, ArchiCAD rund 15 GB, 108 MB frei); Systemanalyse 260911 Nachtrag Zeile 5 (494 interaktive Sessions in 30 Tagen auf dem Mini); Hub-Audit Abschnitt 1.1 (fünf Reboots am 12.08.2026); heute gemessen: pmset auf dem Mini «sleep prevented by Cineware, Archicad, Claude»; CLAUDE.md Abschnitt Konzept

01.1.4 NAS DS918+ als Einzelpunkt nahe der Kapazitätsgrenze, Backup unverifiziert (hoch, SPOF / Datensicherheit). Ein einzelnes 4-Bay-NAS der Modellgeneration 2018 ist zugleich Bürodaten-Dateiserver, kanonisches Hub-Repo, Git-Committer und Träger aller Status- und Queue-Dateien. Das Volume ist seit mindestens fünf Wochen zu 93 % belegt. Am 07.09.2026 war das NAS 5.5 Stunden nicht erreichbar. Zum Backup existiert nur eine unbelegte Zeile; RAID-Level, Plattenalter, SMART-Überwachung, Offsite-Ziel und ein Restore-Test sind nirgends dokumentiert. Beleg: df heute: 6.0 TiB, 5.6 TiB belegt, 467 GiB frei, 93 %; docs/konzepte/260812-Hub-Audit/260812-Hub-Audit.md SPOF 5 (93 % am 12.08.); Systemanalyse 260911 Abschnitt 02.1 (07.09.) und 05 (SPOF NAS); docs/systemaudit-260610.md Zeile 52 («Hyper Backup/Snapshots vorhanden», ohne Beleg); Repo-Suche nach RAID/SHR/Btrfs ohne einschlägigen Treffer

01.1.5 Monitoring und Alarmierung sind Eigenbau und hängen an den überwachten Systemen (hoch, Eigenbau / Observability). Alarme gehen über Apple Mail per osascript mit einer Heuristik (Entwürfe vor und nach dem Senden zählen); fällt die Mail-App aus, ist der Monitor blind. Status- und Heartbeat-Dateien liegen auf dem NAS, der Stations-Watchdog gibt ohne Mount auf. Der Tailscale-Wächter wird vom NAS-Pfad gestartet: auf einer mobilen Station ausserhalb des Büros ist das NAS nur über Tailscale erreichbar, der Wächter kann den Tunnel also genau dann nicht heilen, wenn er unten ist. Es gibt keinen unabhängigen Out-of-Band-Kanal. Folge belegt: vier Tage Fernzugangs-Ausfall trotz vierfacher korrekter Erkennung. Beleg: scripts/stationen-watchdog.sh Zeilen 64-71 und 179-234; scripts/tailscale-waechter.sh Zeilen 7-26 und 92-143; templates/launchd/ch.jans.tailscale-waechter.plist Zeile 33 (Programmpfad auf /Volumes/daten); rules/betrieb-chronik.md Abschnitte 260824c, 260826 («Apple Mail osascript-Timeout, dritter Tag, Monitor blind»), 260914b; Rule auto-verbesserungen 260824

01.1.6 Kein Konfigurationsmanagement: Wochen zwischen «gebaut» und «installiert», Stationen driften (hoch, Betriebsaufwand / Governance). Wächter und Härtenungen werden geschrieben, aber nicht ausgerollt, weil Persistenz-Eingriffe bewusst beim Inhaber liegen und es keinen deklarativen Soll-Zustand gibt. Der Tailscale-Wächter lief 24 Tage nach dem Bau auf keiner Station. Auf dem Mac mini läuft heute weiterhin nur der einmalige Mount-Job ohne Wiederherstellung (Audit-Befund vom 12.08.), Auto-Mount, Keepalive und Stations-Watchdog fehlen; auf dem MacBook Pro ist der Stations-Watchdog ebenfalls nicht geladen. Jobs, SMB-Tuning und Plists unterscheiden sich je Station. Beleg: rules/betrieb-chronik.md Abschnitte 260917b («24 Tage zwischen gebaut und läuft»), 260826b, 260911b (Installation A1/A3/A4 vom Klassifikator geblockt), 260812f; Hub-Audit Abschnitt 1.2 (elf abgelöste Plists, exklusive Jobs je Station) und SPOF 3; heute gemessen: launchctl list auf dem Mini zeigt com.jans.nas-mount, aber weder nas-auto-mount noch nas-keepalive noch stationen-watchdog; auf dem MacBook Pro kein stationen-watchdog, ~/jans-watchdog leer seit 13.08.; nsmb.conf nur auf dem MacBook Pro (User-Ebene), auf dem Mini keine

01.1.7 Sicherheit: NAS-Freigabe als Code-Kanal, pauschale TCC-Ausnahme, geteilte Identitäten (hoch, Sicherheit / Datenschutz). Wer in die Sync-Task-Queue auf der SMB-Freigabe schreiben kann, führt alle 30 Minuten Shell-Code auf den Stationen aus; der Schutz ist ein Regex-Musterkatalog, der nachweislich bereits ein rm -rf durchliess. /bin/bash erhält als Pflichtschritt Festplattenvollzugriff, womit jedes Bash-Script die TCC-Grenzen umgeht. Ein einziges M365-App-Zertifikat mit tenantweiten Rechten wird per scp auf jede Station kopiert, auch auf mobile Geräte. SSH-Schlüssel liegen ohne Passphrase vor und sind voll vermascht. SMB-Signing ist clientseitig deaktiviert, was auch im LAN gilt. Mit Mitarbeitenden oder Mandanten ist dieses Vertrauensmodell nicht tragfähig (Schweizer DSG: Zugriffsbeschränkung, Nachvollziehbarkeit). Beleg: rules/betrieb-chronik.md Abschnitte 260730 und 260811; scripts/check-launchd-fda.sh Zeilen 78-90; scripts/neue-station.sh Zeilen 176-188; connectors/hub-setup-daten.json Abschnitte m365_connector und stationen.macbook-revendo; Hub-Audit Abschnitt 2.1 (SSH-Key ohne Passphrase); rules/betrieb-chronik.md 260824d (Vollvermaschung); /Users/raphaeljans/Library/Preferences/nsmb.conf (signing_required=no)

01.1.8 Wiederkehrende RAM-Notstände, Lastverteilung per Eigenbau auf Consumer-Hardware (hoch, Skalierung / Eigenbau). Zwei Speicher-Notstände mit erzwungenem Neustart (28.07. und 02.09.2026). Sechs unabhängige Feuermechanismen je Station wussten nichts voneinander; der Deckel (Lauf-Gate, Speicher-Wächter, Session-Wächter) ist Bash mit vm_stat-Parsing, Prozessmuster-Matching und SIGKILL. Ein 16-GB-Notebook trägt die Mehrheit der Scheduled Tasks und läuft als Always-On im Clamshell. Harte Ressourcenlimits pro Lauf (cgroups, Queue mit Concurrency) gibt es auf macOS nicht. Beleg: docs/konzepte/260728-Speicher-Architektur/260728-Speicher-Architektur.md Abschnitte Messergebnis und «keine gemeinsame Instanz»; rules/betrieb-chronik.md 260902; scripts/speicher-waechter.sh Zeilen 49-64 und 104-131; scripts/claude-session-waechter.sh Zeilen 114-124 und 134-155; Systemanalyse 260911 Abschnitt 01.2 (34 Tasks MacBook gegen 8 Mini); heute gemessen auf dem MacBook Pro: SleepDisabled 1, 16 GB

01.1.9 Mount-Logik mit bekannten Konstruktionsfehlern und hart kodierten Adressen (mittel, Eigenbau / Robustheit). Der Mountversuch ins Büro-LAN wird übersprungen, wenn ein Ping auf einen öffentlichen DNS-Server scheitert: im aktuellen Log 389 Skips, am 17.09.2026 von 04:40 bis 09:08 kein einziger Mountversuch. Das Muster umount -f plus Remount erzeugt die bekannte Schleife mit dem Mountpoint daten-1, die vier Stunden lang kein Wächter auflöste; die Härtenung ist seit 31.08. offen. IP-Adressen und Hostnamen stehen hart in den Scripts statt aus der kanonischen JSON-Quelle zu kommen. Die timeout-Eigenimplementierung ist dreifach kopiert, die Mail-Funktion ebenfalls. Beleg: scripts/nas-auto-mount.sh Zeilen 34-36, 70-78 und 84-87; ~/hub/.git/nas-auto-mount.log (389 Zeilen «Kein Internet», heute gezählt); rules/betrieb-chronik.md 260831 (SYN-64 offen); eigene Zählung: 63

Zeilen mit hart kodierten privaten oder Tailnet-Adressen in 31 Dateien unter scripts, connectors, templates, skills, commands, agents; _timeout() in 3 Scripts, mailen() in 3 Scripts

01.1.10 Logs flüchtig und verstreut, keine zentrale Observability (mittel, Observability). Sämtliche launchd-Ausgaben gehen nach /tmp und sind nach jedem Neustart weg, also genau dann, wenn sie den Absturz erklären müssten. Die Mount-Wächter loggen in das .git-Verzeichnis des SSD-Klons und rotieren auf 400 bis 500 Zeilen, was bei hoher Stall-Rate nur rund zwei Tage Historie ergibt. Speicher- und Gate-Logs liegen auf dem NAS und fallen bei Mount-Abriss aus. Es gibt weder Metriken noch Dashboards noch Aufbewahrungsregeln. Beleg: templates/launchd/ch.jans.tailscale-waechter.plist (StandardOutPath /tmp), scripts/com.jans.nas-auto-mount.plist, scripts/com.jans.git-auto-sync.plist; Hub-Audit Abschnitt 2.1 «Was der Reboot ausserdem zerstört»; scripts/nas-auto-mount.sh Zeilen 38 und 46-49; scripts/nas-keepalive.sh Zeilen 26-33; scripts/speicher-waechter.sh Zeilen 34-35; heute gemessen: nas-keepalive.log beginnt am 17.09.2026

01.1.11 Zeit-, Namens- und Registerdrift der Stationen (mittel, Skalierung / Governance). Eine Stationsuhr ging zehn Tage nach und verfälschte still eine Git-Auswertung. Das Stationsregister führt dieselbe Maschine doppelt. Drei Namensschemata (Stationsname, LocalHostName, Tailscale-Name) erzwingen Normalisierungs-Workarounds. Die dritte Station hat keine Status-Datei, ihr letzter Herzschlag ist zwei Tage alt, und der Watchdog prüft per Code nur zwei Stationen. Für ein Wachstum auf mehrere Mitarbeitende fehlt ein Inventar mit eindeutigen Identitäten. Beleg: rules/betrieb-chronik.md 260813e; docs/stationen.md Zeilen 9-10; scripts/stationen-watchdog.sh Zeilen 112-113 und 126-134; connectors/hub-setup-daten.json stationen.macbook-revendo (Feld besonderheit, ungeklärte Hostname-Messung); heute gemessen: /Volumes/daten/jans-ai-hub/station-status/ enthält nur mac-mini.md und macbook-pro.md, Stamps der dritten Station vom 17.09.2026

01.1.12 Netz- und Physik-Schicht undokumentiert, keine Tests und kein Patch-Management (mittel, Testlücke / Governance). Zu USV, VLAN oder Netzsegmentierung, RAID, Endpoint-Backup der Macs und OPNsense-Regelwerk gibt es im Repo keine Dokumentation; OPNsense wird genau einmal erwähnt. Das Büro hat ein Kabel-LAN und ein getrenntes WLAN-Subnetz mit dem Mac mini als einzigem Subnet-Router. Für 84 Shell-Scripts mit 12'420 Zeilen existieren zwei Selbsttest-Scripts, keine CI und kein Linter. Updates laufen ungesteuert: ein stilles Tailscale-Auto-Update kappte den Tunnel für vier Tage, macOS 27.0 lief unmittelbar nach Erscheinen auf den Produktivstationen und zog Nacharbeit nach sich. Beleg: Repo-Suche nach USV|UPS, VLAN|Netzsegment|Gastnetz, Time Machine: 0 Treffer; OPNsense nur in docs/konzepte/260712-Mac-Mini-Terminal-Verbindung.md Abschnitt 7; rules/betrieb-chronik.md 260824d (Netz-Topologie), 260916 (macOS-27-Update); scripts/tailscale-waechter.sh Zeilen 7-15; kein Verzeichnis .github/workflows; wc über scripts/*.sh: 84 Dateien, 12'420 Zeilen; sw_vers heute 27.0

01.2 Kennzahlen

01.2.1 Stationen und Kernhardware: 3 Macs (Mac mini M2 Pro 32 GB, MacBook Pro M2 Pro 16 GB, MacBook Pro 32 GB/12 Kerne), 1 NAS DS918+, 1 OPNsense, 1 Netzwerkdrucker. Quelle: sysctl lokal und per ssh lesend am 19.09.2026; connectors/hub-setup-daten.json

01.2.2 NAS-Volume Belegung: 6.0 TiB gesamt, 93 % belegt, 467 GiB frei (12.08.2026: 93 %, 477 GB frei). Quelle: df -h /Volumes/daten am 19.09.2026; Hub-Audit 260812 SPOF 5

01.2.3 Geladene launchd-Jobs mit JANS-Bezug: Mac mini 14 LaunchAgents und 0 LaunchDaemons; MacBook Pro 15 LaunchAgents und 1 LaunchDaemon; 36 verschiedene Labels im Repo erwähnt. Quelle: launchctl list lokal und per ssh am 19.09.2026; grep über scripts, templates, docs, rules/betrieb-chronik.md

01.2.4 SMB-Mount-Abrisse (MacBook Pro): 346 Keepalive-Stalls am 17.09.2026, 33 am 18.09.; historisch 275 am 07.09., 59 WARN-Zyklen am 31.08.. Quelle: ~hub/.git/nas-keepalive.log; Systemanalyse 260911 Abschnitt 02.1; Chronik 260831b

01.2.5 Längste belegte Ausfälle: Tailscale auf dem Mini 4 Tage (20. bis 24.08.2026); NAS 5.5 h (07.09.); Sync-Kette 41 h (11./12.08.) und 4 Tage Divergenz mit 488 Abgleichversuchen (29.08. bis 02.09.); App-Schicht beider Stationen rund 1.5 bis 2.5 Tage (11. bis 14.09.). Quelle: rules/betrieb-chronik.md 260824c, 260812, 260914b; Systemanalyse 260911 Abschnitt 02

01.2.6 Verzug zwischen Bau und Installation eines Wächters: 24 Tage (Tailscale-Wächter); Audit-Massnahmen A1/A4 vom 12.08. auf dem Mini am 19.09. weiterhin nicht geladen. Quelle: rules/betrieb-chronik.md 260917b; launchctl list per ssh am 19.09.2026

01.2.7 Speicher-Notstände: 2 (28.07.2026: Claude 20.24 GB bei 16 GB RAM, Mini OneDrive 33 GB und 108 MB frei; 02.09.2026: Claude 18.13 GB, 16 hängende Sessions). Quelle: docs/konzepte/260728-Speicher-Architektur/260728-Speicher-Architektur.md; rules/betrieb-chronik.md 260902

01.2.8 Wächter-, Guard- und Heil-Code: 22 Scripts mit 4'077 Zeilen Bash; gesamt 84 Shell-Scripts mit 12'420 Zeilen in scripts/. Quelle: wc -l über die benannten Scripts am 19.09.2026

01.2.9 Hart verdrahtete Abhängigkeiten: 696 Zeilen mit /Volumes/daten in 229 Dateien; 63 Zeilen mit hart kodierten privaten oder Tailnet-Adressen in 31 Dateien. Quelle: Systemanalyse 260911 Abschnitt 01.2; eigene grep-Zählung am 19.09.2026

01.2.10 Verteilung der Last: 34 Scheduled Tasks auf dem MacBook Pro gegen 8 auf dem Mac mini; interaktive Sessions in 30 Tagen 494 auf dem Mini gegen 142 auf dem MacBook. Quelle: Systemanalyse 260911 Abschnitt 01.2 und Nachtrag

01.2.11 Commit-Volumen der Sync-Kette: rund 4'080 Commits in 30 Tagen (136 je Tag), davon 2'700 vom NAS-Selfcommit; Repo-Pack 271 MB. Quelle: Systemanalyse 260911 Abschnitt 01.2

01.2.12 Tests und CI: 2 Selbsttest-Scripts, 0 CI-Workflows, kein Linter eingebunden. Quelle: find/ls im Worktree am 19.09.2026

01.3 Offene Fragen

01.3.1 Gibt es eine USV für NAS, Firewall und Mac mini, und fährt das NAS bei Stromausfall gesteuert herunter? Im Repo findet sich dazu nichts.

01.3.2 DS918+: RAID- oder SHR-Level, Anzahl und Alter der Platten, SMART-Zustand, RAM-Ausbau, DSM-Version und Supportende? Welcher Benutzer mit welchen Rechten dient dem SSH-Zugang der Stationen, und wie ist der DSM-Aufgabenplaner konfiguriert (laut Hub-Audit ohne sudo nicht lesbar)?

01.3.3 Backup: Welches Ziel hat Hyper Backup tatsächlich, liegt es ausser Haus, ist es verschlüsselt, und wann wurde zuletzt eine Wiederherstellung geübt? Werden die Macs selbst gesichert (Time Machine oder Ähnliches)? Im Repo steht nur die unbelegte Zeile in docs/systemaudit-260610.md Zeile 52.

01.3.4 OPNsense: Hardware, Firmware-Stand, Regelwerk, Konfigurations-Backup, IDS/IPS, VLAN-Fähigkeit der Switches? Wie entsteht das getrennte WLAN-Subnetz im Büro (eigener Router mit doppeltem NAT oder VLAN)?

01.3.5 Tailscale: Gibt es eine ACL-Policy oder gilt der Standard «alle dürfen alles»? Schlüsselablauf, MFA des Tailnet-Kontos, Freigabe der Subnet-Route, und welche Client-Variante läuft je Station (nas-auto-mount.sh spricht von der App-Store-Version, tailscale-waechter.sh von der Standalone-Version mit Sparkle)?

01.3.6 Was war die Ursache der 389 «Kein Internet»-Meldungen, insbesondere am 17.09.2026 von 04:40 bis 09:08: echter Internetausfall, Firewall-Eingriff oder nur blockiertes ICMP? War das NAS im LAN in dieser Zeit erreichbar?

01.3.7 Stand des Entscheids zur Neustart-Festigkeit des Mac mini (Hub-Audit B2, Systemanalyse E0): Bleibt FileVault an, und wird stattdessen ein dedizierter Host beschafft? Heute gemessen unverändert «FileVault is On» ohne LaunchDaemons.

01.3.8 Rolle und Zukunft der dritten Station: eigenes Gerät des Inhabers oder künftiges Mitarbeitenden-Gerät? Sie hat keine Status-Datei und seit 17.09.2026 keinen Herzschlag; ist sie ausser Betrieb?

01.3.9 Internet-Anschluss des Büros: Bandbreite, feste Adresse, Ausfallweg (z.B. Mobilfunk-Failover)? Relevant, weil das Zielbild GitHub und Cloud-Dienste in den kritischen Pfad rückt.

01.3.10 Physische Sicherheit und Standort der Hardware (Schrank, Zutritt, Klima) sowie Versicherungs- und Aufbewahrungsanforderungen für Projektdaten.

01.3.11 Wachstumsziel und Budgetrahmen: Wie viele Mitarbeitende, Mandanten und Hubs in welchem Zeitraum? Davon hängen Dimensionierung von Server, Storage, Lizenzen (MDM, Claude-Seats) und Betreuungsmodell ab.

01.3.12 Welche Personendaten Dritter liegen auf dem NAS und in den Cloud-Bibliotheken (Bewerbungen, Verträge, Kundendaten), und gibt es dafür Zugriffs- und Löschkonzepte? Die Rule auto-verbesserungen 260823 belegt, dass solche Bestände existieren.

02 Sync, Git und Datenhaltung

02.0 Ist-Zustand. Kanonische Quelle des Hub ist ein Git-Arbeitsbaum auf der Synology, den drei Macs und rund 40 Loops über eine SMB-Freigabe direkt beschreiben. Weil Git über SMB hängt, committet die Synology selbst: ein Eigenbau-Script fährt alle 15 Minuten (und auf Zuruf per SSH) `git add -A`, Commit und Push auf den einzigen Branch `main` eines privaten GitHub-Repos, das zugleich als einziges Off-Site-Backup gilt. Jede Station hält zusätzlich einen SSD-Klon, den ein `launchd`-Job abgleicht und der laut Messung ebenfalls beschrieben wird; Skills, Agenten und Commands hängen per Symlink am NAS-Mount. Das Repo (seit 23.03.2026, 11'865 Commits, 5'605 Dateien, rund 500 MB Arbeitsbaum) mischt Scripts, Rules, 22 Wissensbasen, Logbuch und Fristen-Register, Stations-Telemetrie, Renderings und Task-Ergebnisse. 94 % aller Commits stammen vom NAS-Bot, 77 % der Commits der letzten Woche betreffen nur Status- und Verbrauchsdateien. Belegt sind zwei Repo-Spaltungen (6 Stunden; 4 Tage mit 488 Fehlversuchen, Stufe P1), ein 41-Stunden-Stillstand und sieben Ausfalltage durch Mount-Abrisse. Der Inhaber hat die Bauweise am 11.09.2026 selbst als Konstruktionsfehler analysiert (Zielbild: GitHub als Origin, lokale Klone, Mac Mini als Zentrale, Migrationsplan E0 bis E5). Umgesetzt ist erst die Vorarbeit (Union-Merge, Takt 15 Min); der Variantenentscheid ist offen. Seit 02.09.2026 läuft der Abgleich ohne Fehlversuch.

02.1 Schwachstellen

02.1.1 Git-Arbeitsbaum auf SMB als kanonische Quelle, ganze Fähigkeit hängt an einem Mountpoint (kritisch, SPOF). Drei Stationen und die Loops editieren denselben Arbeitsbaum über

SMB. Fällt der Mount, fehlen Skills, Rules, Agenten und Logbuch auf der arbeitenden Station. Git ist über SMB verboten worden, statt die Bauweise zu ändern; acht Hilfsscripts existieren nur, um sie am Leben zu halten. Der Variantenentscheid (V2) ist seit 11.09.2026 offen, umgesetzt ist nur die Vorarbeit. Beleg: docs/konzepte/260911-Systemanalyse-Hub-Architektur-NAS-Git/260911-Systemanalyse-Hub-Architektur-NAS-Git.md Abschnitte 01.2, 02.1 (7 Ausfalltage, bis 275 Mount-Abrisse am Tag), 03.1, 03.2, 08 (Entscheide offen); rules/betrieb-chronik.md Abschnitt 260911b

02.1.2 Alles auf dem NAS-Pfad wird ungeprüft nach GitHub gepusht, inklusive Personendaten Dritter (hoch, Datenschutz). Der Committer fährt alle 15 Minuten `git add -A` und pusht. Es gibt keine aktiven Git-Hooks, keinen Secret- oder Personendaten-Scan und kein Grössen-Gate. Am 23.08.2026 gelangte so ein Inventar mit Bewerberdaten binnen 15 Minuten in das Repo; die Historie wurde bewusst nicht bereinigt. Versioniert sind zudem eine Kontaktliste, ein Korpus authentischer Mails (60 Dateien), Konversationsdestillate (56), Buchungsdaten (7), eine Kunden-KB und eine .eml-Datei. Löschung und Auslandsbekanntgabe nach DSGVO sind nicht geregelt. Beleg: scripts/nas-selfcommit.sh:81; /Volumes/daten/jans-ai-hub/rules/auto-verbesserungen.md Abschnitt 260823 (Commit 75d50502); eigene Messung 19.09.2026: .git/hooks ohne aktive Hooks, `git ls-files` wissen/twin/raw (60), logbuch/konversationen (56), logbuch/bexio-buchungen (7)

02.1.3 Code-Ausführung per Git-Push ohne Guard, schwach geschützte Schlüssel (hoch, Sicherheit). Der Stations-Sync ruft nach jedem Pull den Remote-Task-Runner auf, der jedes .sh unter remote-tasks/pending mit Benutzerrechten ausführt. Die Freigabe-Schwelle (sync-task-guard) liegt nicht vor diesem Kanal. Wer auf main pushen kann (GitHub-Konto, Deploy Key der Synology mit Schreibrecht, Cloud-Session), führt Code auf dem Mac Mini aus. Deploy Key und Stations-SSH-Key sind ohne Passphrase, der Erreichbarkeitstest läuft mit `StrictHostKeyChecking=no`. Branch-Schutz oder signierte Commits sind nirgends dokumentiert. Beleg: remote-tasks/runner.sh:19-20 und :75; scripts/git-auto-sync.sh:44 und :155-163; docs/plan-dsm-git.md:45-46; docs/konzepte/260812-Hub-Audit/260812-Hub-Audit.md:121, :234 und Abschnitt 2.3 Lücke 1

02.1.4 Git dient als Telemetrie- und Zustandsspeicher, die Historie ist Rauschen (hoch, Governance). Stations-Status und Kontingent-Verbrauch werden alle paar Minuten als Datei committet. In 7 Tagen berühren 603 von 784 Commits ausschliesslich diese Dateien; in 30 Tagen wurden station-status/mac-mini.md 3'042-mal und macbook-pro.md 2'622-mal committet. 4'065 Commits in 30 Tagen (135 je Tag), davon 2'734 mit generischer Message. Als Audit-Trail oder zur Fehlersuche ist die Historie damit kaum brauchbar, und das Repo wächst um rund 1 GB je Jahr. Beleg: Eigene Messung 19.09.2026 im Worktree (`git log --since=7.days/30.days --name-only`); docs/konzepte/260911-Vertiefung-Umbau-Verifikation-Routinen/260911-Vertiefung-Umbau-Verifikation-Routinen.md Abschnitt 01.5 R2 und R4

02.1.5 Commits sind weder atomar noch zuordenbar; Konfliktmarker werden nicht abgefangen (hoch, Governance). 94 % aller Commits (11'209 von 11'865) tragen den Bot als Autor. `git add -A` sammelt laufende Fremdarbeit unter der Message des zufällig fertigen Loops, `git log -- <datei>` taugt nicht als Datierungsbeleg. Am 23.08.2026 committete der Auto-Sync eine Datei mit unaufgelösten Konfliktmarkern, der NAS-Committer übernahm sie in die kanonische Quelle; die als offen vermerkte Schutzprüfung fehlt im Script weiterhin. Beleg: rules/betrieb-chronik.md:1716-1754 (260813d) und :1296-1339 (260823c, Härtelücke Z. 1336-1339); scripts/nas-selfcommit.sh (Stand 19.09.2026 ohne Marker-Prüfung)

02.1.6 Zwei-Schreiber-Modell auf main ist nur faktisch, nicht technisch aufgehoben (hoch, Betriebsaufwand). NAS-Committer und Stations-Auto-Sync pushen beide auf main. Rule und Praxis widersprechen sich: gemessen kamen 40 von 83 Dateien nur über den SSD-Klon nach draussen,

Dispatch-Läufe arbeiten systematisch im Klon. Folgen: Spaltung 6 Stunden (24.08.), Spaltung 4 Tage mit 488 Fehlversuchen (29.08. bis 02.09., Auflösung von Hand), 41 Stunden Stillstand (11./12.08.). Seit 01.09. stammen zwar alle Commits vom NAS, das Script committet und pusht aber unverändert und nutzt weiter --autostash, den Auslöser zweier Vorfälle. Beleg: scripts/git-auto-sync.sh:87, :125, :140; rules/betrieb-chronik.md:887-937 (260824f) und :939-978 (260824e); docs/konzepte/260830-Laufzeitschicht-Umbau/SPEC.md Abschnitt 2, Befund B6; /Volumes/daten/jans-ai-hub/sync-tasks/log/selfcommit-202608.log (588 Fehlversuche, 395 am 30.08.)

02.1.7 Binär- und Scratch-Daten im Git, kein LFS, Bereinigung seit 11.09. halb fertig (mittel, Skalierung). render-scratch (276 MB, 213 Dateien) und remote-tasks/results (73 MB) sind weiterhin getrackt, obwohl .gitignore sie seit 11.09.2026 nennt; der nötige git rm --cached wartet auf den Inhaber. Eine DXF mit 83.5 MB und eine Dateiliste mit 70.2 MB liegen nahe am GitHub-Limit von 100 MB je Datei; ein Überschreiten würde den Push und damit die ganze Kette blockieren. PNG 152 MB, DXF 110 MB, PDF 35 MB, OBJ/C4D/3dm 35 MB; LFS-Einträge: 0. Blobs bleiben in der Historie. Beleg: .gitignore:84-90 (Z. 88: wirksam erst nach git rm --cached); rules/betrieb-chronik.md:71-74; eigene Messung 19.09.2026 (git ls-files render-scratch = 213; git rev-list --objects --all, grösste Blobs)

02.1.8 Keine Trennung von Code, Konfiguration, Wissen, Zustand und Artefakten (mittel, Governance). Ein Repo, ein Branch, ein Berechtigungsraum für Scripts, Rules, 22 Wissensbasen (4'329 Dateien), Fristen-Register und Logbuch (je 1.1 MB Monolith, fristen.md 494 Commits), Telemetrie, Renderings und Task-Ergebnisse. Union-Merge liegt auch auf dem Fristen-Register, das Tabellenzeilen führt, obwohl der Kommentar Union für strukturierte Dateien ausschliesst. Die Live-Konfiguration der Scheduled Tasks liegt ausserhalb Git, der Spiegel ist nur Doku; belegter Drift-Vorfall. Die Task- und Freigabe-Queue ist gar nicht versioniert. Beleg: .gitattributes:5-6 und :10; scripts/nas-selfcommit.sh:143-155 (Tabellezeile ins Register); .gitignore:27; rules/betrieb-chronik.md:1442-1465 (260816); scripts/scheduled-tasks-mirror.sh:14-16

02.1.9 Eigenbau-Sync-Stack mit fünf Mechanismen statt Standardwerkzeugen (mittel, Eigenbau). nas-selfcommit, git-auto-sync, Sync-Task-Runner, remote-tasks und Cloud-Sync greifen ineinander; die Zahl möglicher gegenseitiger Blockaden wächst quadratisch (der 41-Stunden-Stillstand entstand genau so). Der Committer räumt bei jedem Lauf Synology-Indexordner aus .git, löscht index.lock nach einer Stunde selbst und nutzt mkdir-Locks. 730 Zeilen verdrahten den Pfad /Volumes/daten fest, dazu 158 Zeilen in 43 Task-Prompts. Beleg: docs/konzepte/260812-Hub-Audit/260812-Hub-Audit.md:128-143 und :433-439; scripts/nas-selfcommit.sh:34, :41, :44-47; eigene Messung 19.09.2026 (git grep -c /Volumes/daten in scripts, skills, agents, rules, commands, connectors, templates = 730 Zeilen); Vertiefung 260911 Abschnitt 00.1

02.1.10 Keine Tests, kein Lint, kein CI für die Scripts, an denen die Datenintegrität hängt (mittel, Testlücke). Im Repo liegt keine einzige Testdatei, kein CI-Verzeichnis, keine shellcheck-Konfiguration. Von 84 Scripts unter scripts/ setzen 14 set -e. Ein aus dem Schwesterscript kopierter Variablenname liess den Merge-Rückfall still scheitern und wurde nur durch Handtest gefunden; ein Testlauf schrieb in das echte Fristen-Register, weil der Pfad absolut verdrahtet ist. Beleg: Eigene Messung 19.09.2026 (git ls-files ohne Treffer für tests/, .bats, test_*.py; kein .github, kein .shellcheckrc; grep set -e in scripts/*.sh = 14 von 84); rules/betrieb-chronik.md:915-926

02.1.11 Backup und Wiederherstellung sind nicht nachgewiesen (mittel, SPOF). GitHub gilt als einziges Off-Site-Backup, deckt aber nur den versionierten Teil. Task-Queue, Committer-Logs, Lauf-Journal, schwere Referenzen (rund 490 MB) und der Secrets-Ordner liegen allein auf dem NAS, dessen Volume zu 93 % belegt ist. Das NAS-Backup ist nur als Inhaber-Aussage vom 10.06.2026 belegt; ein

Restore-Test, RPO/RTO-Ziele oder ein Notfall-Runbook finden sich nirgends. Beleg:
docs/konzepte/260812-Hub-Audit/260812-Hub-Audit.md:150; docs/systemaudit-260610.md:52;
Systemanalyse 260911 Abschnitt 01.2 (NAS 93 %, nicht versionierte Ordner); eigene Suche
19.09.2026 nach Restore-Test, Disaster Recovery, Notfallhandbuch in docs, rules, scripts: kein Treffer

02.1.12 Repo-Hygiene und fehlendes Mehrbenutzer-Modell (niedrig, Skalierung). 9 Worktrees und 52 lokale Branches (alle gemergt), zwei redundante Packs zu je 269 MB ohne gc, ein Garbage-Eintrag im Worktree-Ref. Hostnamen und LAN-Adressen stehen im Klartext in versionierten Scripts und Docs. Für Mitarbeitende gilt laut Entscheid vom 10.06.2026 voller Zugriff ohne Berechtigungsmodell; ein Branch- oder Review-Workflow existiert nicht, alles geht direkt auf main. Beleg: Eigene Messung 19.09.2026 (git worktree list, git branch, git count-objects -vH, ls .git/objects/pack); scripts/nas-commit-now.sh:23; scripts/git-divergenz.sh:17; docs/plan-dsm-git.md:42; docs/systemaudit-260610.md:57-58

02.2 Kennzahlen

02.2.1 Alter und Umfang des Repos: Erster Commit 23.03.2026; 11'865 Commits; 5'605 getrackte Dateien; rund 500 MB getrackter Arbeitsbaum. Quelle: git log, git ls-files im Worktree, 19.09.2026

02.2.2 Commit-Rate: 4'065 Commits in 30 Tagen (135 je Tag); 784 in 7 Tagen. Quelle: git log --since, 19.09.2026

02.2.3 Commit-Typen 30 Tage: 2'734 generische nas-selfcommit (67 %), 355 auto-sync, 955 mit sprechender Message, 3 Merges. Quelle: git log --since=30.days --format=%s, 19.09.2026

02.2.4 Bot-Anteil gesamt: 11'209 von 11'865 Commits (94 %) mit Autor NAS-Selfcommit; letzte 14 Tage 1'536 von 1'536. Quelle: git log --format=%an, 19.09.2026

02.2.5 Telemetrie-Anteil: 603 von 784 Commits (77 %) der letzten 7 Tage berühren nur station-status/ und logbuch/kontingent/. Quelle: git log --since=7.days --name-only, 19.09.2026

02.2.6 Höchster Datei-Churn 30 Tage: station-status/mac-mini.md 3'042, station-status/macbook-pro.md 2'622, verbrauch-Macmini.json 1'811, verbrauch-Macbookpro.json 1'259, logbuch/fristen.md 256. Quelle: git log --since=30.days --name-only, 19.09.2026

02.2.7 Getrackte Grösse nach Bereich: render-scratch 276 MB (213 Dateien), wissen 79 MB (4'329), remote-tasks 73 MB (119), skills 39 MB (361), docs 28 MB (54), logbuch 4.5 MB (163), scripts 0.7 MB (100). Quelle: git ls-files + stat, 19.09.2026

02.2.8 Grösste Dateien: DXF 83.5 MB, Dateiliste 70.2 MB (GitHub-Grenze 100 MB je Datei); Git LFS: 0 Einträge. Quelle: git rev-list --objects --all, .gitattributes, 19.09.2026

02.2.9 Binärdaten nach Typ: PNG 152 MB (74), DXF 110 MB (4), PDF 35 MB (50), OBJ 17 MB (75), C4D 14 MB (14), 3dm 3.7 MB (7). Quelle: git ls-files + stat, 19.09.2026

02.2.10 Pack-Grösse: Lokal 546 MiB in 4 Packs (zwei redundante zu je 269 MB, kein gc), effektiv rund 270 MB; 197'069 Objekte; Wachstum laut Vertiefung 84 MB je 30 Tage. Quelle: git count-objects -vH, 19.09.2026; Vertiefung 260911 Abschnitt 01.5 R2

02.2.11 Abgleich-Fehlversuche des NAS-Committers: August 588 (davon 395 am 30.08.), September 225 (nur 01./02.09.), seit 02.09. 0; Fehlzähler aktuell 0. Quelle: /Volumes/daten/jans-ai-hub/sync-tasks/log/selfcommit-202608.log und -202609.log, awk-Zählung 19.09.2026

02.2.12 Belegte Sync-Vorfälle: 41 h Stillstand (11./12.08.), 6 h Spaltung (24.08., 26/51 Commits), 4 Tage Spaltung P1 (29.08. bis 02.09., 488 Fehlversuche), 7 Ausfalltage Mount (bis 275 Abrisse am Tag). Quelle: Systemanalyse 260911 Abschnitte 02.1 und 02.2

02.2.13 Fest verdrahtete NAS-Pfade: 730 Zeilen in scripts, skills, agents, rules, commands, connectors, templates; Systemanalyse 11.09.: 696 Zeilen in 229 Dateien plus 158 Zeilen in 43 Task-Prompts und 6 launchd-Plists. Quelle: git grep -c, 19.09.2026; Vertiefung 260911 Abschnitt 00.1

02.2.14 Script-Qualität: 84 Scripts unter scripts/*.sh, davon 14 mit set -e, 67 mit set -u; 0 Testdateien, kein CI, keine aktiven Git-Hooks. Quelle: grep und git ls-files, 19.09.2026

02.2.15 Worktrees und Branches: 9 Worktrees, 52 lokale Branches (alle in main gemergt), 8 Remote-Branches, 1 Stash, 23 Merge-Commits gesamt. Quelle: git worktree list, git branch, 19.09.2026

02.2.16 Nicht versionierte NAS-Daten: sync-tasks/log 2.4 MB, dispatch 2.6 MB, output 1.3 MB, logbuch/laeufer 1.5 MB (53 Dateien), secrets 1 Eintrag; laut Systemanalyse zusätzlich Archimed 460 MB, Behörden-Spiegel 27 MB; NAS-Volume 93 % belegt. Quelle: du/ls auf /Volumes/daten/jans-ai-hub, 19.09.2026; Systemanalyse 260911 Abschnitt 01.2

02.2.17 Sync-Mechanismen und SMB-Hilfsscripts: 5 unabhängige Sync-Mechanismen; 8 Scripts existieren nur wegen der SMB-Bauweise. Quelle: Hub-Audit 260812 Abschnitt 1.5; Systemanalyse 260911 Abschnitt 01.2

02.3 Offene Fragen

02.3.1 Ist die NAS-Sicherung (Hyper Backup, Snapshots) tatsächlich aktiv, wohin sichert sie, mit welcher Aufbewahrung, und wurde je ein Restore getestet? Belegt ist nur die Inhaber-Aussage vom 10.06.2026.

02.3.2 GitHub-Seite nicht prüfbar (kein Netzwerkzugriff im Auftrag): Branch-Schutz, 2FA, Umfang der Deploy Keys, Collaborators, Secret Scanning, Standort und Vertragsgrundlage.

02.3.3 Warum stammen seit 01.09.2026 alle Commits vom NAS-Bot? Schreiben Dispatch- und Lern-Läufe nun auf den NAS-Pfad, oder bleibt Arbeit ungepusht in den SSD-Klonen liegen? Nicht gemessen.

02.3.4 Stand VO+: Der Schritt git rm --cached für render-scratch und die Entscheide 08.1 bis 08.6 der Systemanalyse sind laut Repo offen. Gilt die Empfehlung V2 für den Inhaber weiterhin, oder soll der Ingenieur die Variantenfrage neu aufrollen?

02.3.5 Konfiguration des DSM-Aufgabenplaners ist ohne sudo nicht einsehbar (Hub-Audit, Abschnitt «nicht geprüft»); zudem nennt docs/plan-dsm-git.md /volume1, das Script /volume2.

02.3.6 Ist der Commit mit den Bewerberdaten (23.08.2026) weiterhin in der GitHub-Historie? Laut Rule wurde die Historie bewusst nicht umgeschrieben; ob eine Bereinigung gewünscht ist, ist ein Entscheid des Inhabers.

02.3.7 Rechtliche Einordnung nach DSGVO für den versionierten Mail-Korpus, die Konversationsdestillate und die Kontaktliste (Rechtsgrundlage, Auslandsbekanntgabe, Aufbewahrung) ist juristisch zu klären, nicht technisch.

02.3.8 Dritte Station (Intel-MacBook): Inventur war am 11.09.2026 nicht möglich; Zustand von Klon, Auto-Sync und Schlüsseln dort unbekannt.

02.3.9 Erwartetes Datenwachstum durch die Archiv-Destillation (wissens-destillat) und durch weitere Hubs oder Mandanten ist nirgends beziffert.

02.3.10 Gilt der Entscheid vom 10.06.2026 «voller Zugriff für alle Mitarbeiter-Stationen, kein Berechtigungsmodell» angesichts der Ausbaubabsicht weiterhin?

02.3.11 Die lokale Pack-Grösse (546 MiB) weicht von der Messung der Systemanalyse (271 MB) ab; vermutlich zwei redundante Packs ohne gc, die Grösse des NAS-Repos selbst wurde auftragsgemäss nicht gemessen (kein git auf dem NAS).

03 Laufzeit und Orchestrierung

03.0 Ist-Zustand. Die Laufzeitschicht ist ein gewachsenes Geflecht aus Shell-Scripts ohne zentrales Job-System. Mindestens sieben Feuermechanismen laufen nebeneinander: die App-internen Claude-Scheduled-Tasks je Station (Spiegel: 34 MacBook Pro, 12 Mac Mini), launchd-Agenten (15 geladen allein auf dem MacBook Pro), der DSM-Aufgabenplaner der Synology, der Git-Hook-Runner für Remote-Tasks, die stündliche Nachtschicht des Mac Mini, Dispatch vom Handy und ad hoc per nohup gestartete Treiber. Dazu kommen fünf dateibasierte Queues (Sync-Tasks, Freigabe, Remote-Tasks, Widerruf, Commit-Anfragen) auf SMB-Share bzw. Git. Zustand liegt in JSONL-, Flag-, Stamp- und STOP-Dateien sowie in Markdown-Registern; Locks sind mkdir-Verzeichnisse in /tmp je Maschine. Gemeinsame Instanzen sind erst nach Vorfällen entstanden: lauf-gate.sh (Speicher, Prozesszahl, Wochenkontingent), clauderun.sh (Kostenjournal), arbeits-weiche.sh (Lastverteilung Mini/MacBook). Alle unbeaufsichtigten Läufe fahren über das Abo (OAuth-Token), bewusst nie über API; das Wochenlimit wird über einen aus Transcripts kalibrierten Proxy geschätzt. Das Lauf-Journal zeigt seit 29.07.2026 2'543 Läufe, davon 43 % fehlgeschlagen (Auth, Session-Limit). Der als richtig erkannte Umbau auf einen einzigen Einstieg (lauf.sh) ist gebaut, aber nicht umgeschaltet. Die operative Aufsicht (Tagesbriefing, Fristen-Radar) hängt an der von aussen nicht steuerbaren App-Registry der mobilen Station.

03.1 Schwachstellen

03.1.1 Kein zentrales Job-System: sieben Scheduler und fünf Queues nebeneinander (kritisch, Eigenbau). App-Scheduled-Tasks je Station, launchd, DSM-Aufgabenplaner, User-Crontab, Git-Hook-Runner, Dispatch und nohup-Treiber wissen nichts voneinander. Es gibt keinen Ort, der alle Jobs mit Zustand, Takt, letztem Ergebnis und nächstem Lauf kennt. Doppeltakte und Kollisionen wurden jeweils erst im Schadenfall entdeckt und mit einem weiteren Script geflickt. Beleg: scripts/lauf-gate.sh Z. 8-19 (Inventar der Feuermechanismen, «Keiner davon zählt»); docs/konzepte/260812-Hub-Audit/260812-Hub-Audit.md Abschnitt 2.4; rules/betrieb-chronik.md Abschnitt 260803 (Z. 2331, zwei Taktgeber für denselben Loop); Messung 19.09.2026: 15 geladene launchd-Jobs und 34 Task-Ordner allein auf dem MacBook Pro

03.1.2 App-Registry der Scheduled Tasks ist nicht steuerbar und trägt die gesamte Aufsicht auf der mobilen Station (kritisch, SPOF). Die Registry ist stationslokal, headless weder abfragbar noch umtaktbar; Frontmatter-Felder (enabled, cron_target, model) sind Doku ohne Wirkung. Nur 3 von 46 gespiegelten Tasks tragen überhaupt ein Takt-Feld. Tagesbriefing und Fristen-Radar liegen ausschliesslich auf dem MacBook Pro: vier Tage Ausfall blieben unbemerkt, zwei Warnmails lagen unversendet in den Entwürfen. Beleg: docs/konzepte/260812-Hub-Audit/260812-Hub-Audit.md Z. 419-430 (Lücke 4) und Z. 551-555 (R11); rules/betrieb-chronik.md Abschnitt 260812b (Z. 2034) und 260816 (Z. 1442, Station führte sechs Tage alten Prompt aus); templates/scheduled-tasks/README.md Abschnitt «Wichtig beim Lesen»

03.1.3 Fernausführung per Design bei maximalen Rechten der unbeaufsichtigten Läufe (kritisch, Sicherheit). Schreibzugriff auf den NAS-Share oder das Git-Repo bedeutet Codeausführung auf den

Stationen: Sync-Tasks und Widerrufs-Befehle laufen über eval aus Dateien, Remote-Tasks als bash aus pending/. Der Guard ist eine Regex-Denylist über den Task-Text; bei typ: prompt prüft er Prosa, nicht die später vom Modell ausgeführten Befehle. Gleichzeitig erlaubt die Projekt-Allowlist Bash(*) und Wildcard-MCP (Mail, M365, Computer-Steuerung) ohne einen einzigen deny-Eintrag, und headless-Läufe lesen Mails und Webinhalte (Prompt-Injection-Fläche). Beleg: scripts/sync-task-run.sh Z. 174 (eval); scripts/widerruf-queue.sh Z. 171 (eval); remote-tasks/README.md Z. 69-71 («wer pushen darf, darf ausführen»); .claude/settings.json (allow 28, deny 0, enthält Bash(*)); betrieb-chronik.md Abschnitte 260730 (Z. 2471, Task setzte SSH-Dauerzugang vor Freigabe) und 260811 (Z. 2140, Guard liess rm -rf durch)

03.1.4 43 % Fehlläufe; Anmeldung, Binary und Startweg sind nicht vereinheitlicht, der Umbau steht (hoch, Betriebsaufwand). Von 2'543 journalisierten Läufen endeten 1'091 mit Fehler, davon 594 mit Auth-/OAuth-Meldungen und 477 am Session-Limit (Eigenauswertung nach Stichwort). Ursache laut eigener Spec: zwei Anmeldeorte mit undokumentierter Vorrangordnung, drei claude-Binaries ohne Auflösungsregel, vererbte Session-Variablen bei nohup. Der Ersatz lauf.sh ist seit 03.09.2026 gebaut, hat aber keinen produktiven Aufrufer; V1 (Startweg-Matrix) und V3 sind nie gemessen, die B4-Ursache ist eine unverifizierte Hypothese. Beleg: /Volumes/daten/jans-ai-hub/logbuch/laeufer/*.jsonl (Auswertung 19.09.2026); docs/konzepte/260830-Laufzeitschicht-Umbau/SPEC.md Abschnitt 2 (B1-B4); docs/konzepte/260830-Laufzeitschicht-Umbau/VERIFIKATION.md Z. 23-31; grep nach lauf.sh in scripts/, skills/, templates/: einziger Treffer ist die Datei selbst

03.1.5 Leerlauf und Ad-hoc-Treiber ausserhalb des Repos verbrennen Kontingent (hoch, Kosten). 48 % aller journalisierten Läufe (1'231) stammen von Treibern mit Laufnamen mschub/vollschub, deren Scripts nicht im Repo liegen; eines lief verwaist aus /tmp mit hart kodierter Aufgabenrotation und erzeugte in einer Nacht 437 Fehlläufe, die über die kontoweite 5h-Sperre auch vier Loops der anderen Station blockierten. Die Schub-Lanes vom 29./30.08. fuhren 699 Läufe ohne Ertrag (rund 240 USD API-Äquivalent), weil der Treiber Läufe statt Ertrag zählte. Beleg: rules/betrieb-chronik.md Z. 1176-1200 (/tmp/vollschub-mini.sh), Z. 3211 ff. (Doppel-Dispatch, 437 Läufe rc=1, Sperre kontoweit); scripts/vollgas-schub.sh Z. 108-121 (Korrekturkommentar 30.08.); Lauf-Journal: schub-synopsis 351, schub-normen-pruefstand 194, schub-baurecht-thalwil 99, schub-grobkosten 55 Läufe

03.1.6 Kontingentsteuerung auf Schätzgrösse; operative und Lern-Läufe teilen einen Abo-Topf (hoch, Kosten). Die Wochenobergrenze (167 Mio «teure» Token) ist an einem einzigen Limit-Ereignis kalibriert und wird aus lokalen Transcripts hochgerechnet; die 5h-Sperre wird gar nicht prognostiziert. Das Gate erreicht nur launchd-Läufe, nicht die App-Tasks als grösste Verbraucher. Folge: 47 h leeres Kontingent inklusive operativer Briefings. Die Migration auf drei Töpfe (Max 20x plus zwei Team-Seats) ist seit 13.08.2026 nur Entscheidungsvorlage; API/Agent SDK ist für unbeaufsichtigte Läufe per Script ausgeschlossen. Beleg: scripts/kontingent-budget.sh Z. 24-43 und Z. 76; scripts/lauf-gate.sh Z. 189-193 («die halbe Miete»); docs/konzepte/260813-Kontingent-Takt/260813-Kontingent-Takt.md Z. 85-118 und Schlusssatz «Nichts ... wurde umgesetzt»; scripts/vollgas-runner.sh (unset ANTHROPIC_API_KEY, «NIE über API-Key fahren»)

03.1.7 Kein kanonischer Arbeitsbaum für automatische Läufe (hoch, Governance). Die Aufrufer arbeiten in drei verschiedenen Default-Arbeitsbäumen (SSD-Klon, NAS bevorzugt, NAS fest). Das verletzt die eigene Grundregel «nur auf dem NAS editieren» systematisch und führte zu einer Divergenz von 203 zu 115 Commits mit Beinah-Dubletten in einer Wissensbasis. vollgas-schub.sh setzt zudem git status über den SMB-Mount ab, gestützt auf eine im Script dokumentierte Fehlannahme. Beleg: docs/konzepte/260830-Laufzeitschicht-Umbau/BEFUND.md Abschnitte 1 und 2; SPEC.md

Abschnitt 2 Zeile B6; scripts/vollgas-schub.sh Z. 122-124 gegen Rule sync-kanonische-quelle.md (Absatz «lesende hängen ebenfalls», SYN-61)

03.1.8 Zustand, Locks und Nebenläufigkeit über Dateien auf SMB und /tmp (hoch, Eigenbau). Das Gate ist ein check-then-act über pgrep ohne Reservierung, Locks sind mkdir in /tmp und gelten nur je Maschine, das Journal wird von drei Stationen ohne Sperre an dieselbe JSONL-Datei auf SMB angehängt. Doppelläufe auf dieselben Wiki-Dateien sind mehrfach belegt. Lang laufende Scripts werden teils vom NAS-Pfad ausgeführt und sterben bei einem SMB-Stall kommentarlos (15 Tasks betroffen, Entscheid offen). Beleg: scripts/lauf-gate.sh Z. 60-62 und Z. 137-140; scripts/claude-run.sh Z. 68-71 und Z. 132; remote-tasks/runner.sh Z. 44 («wirkt nur pro Maschine»); betrieb-chronik.md Abschnitte 260824 Doppel-Dispatch (Z. 3211), 260813f (Z. 1649), 260807 (Z. 2294)

03.1.9 Queues ohne Retry-Grenze, Timeout, Idempotenz und Eskalation (mittel, Eigenbau). Ein fehlgeschlagener Sync-Task bleibt unbegrenzt in der Queue und wird alle 30 Minuten erneut ausgeführt; es gibt weder Versuchszähler noch Dead-Letter noch Idempotenz-Schlüssel. Remote-Tasks laufen ohne Timeout. Die Freigabe-Queue hielt eine korrekt erkannte Störung 41 Stunden zurück, weil niemand sie las. Beleg: scripts/sync-task-run.sh Z. 181-185; remote-tasks/runner.sh Z. 70-78 (bash "\$TASK" ohne Timeout); rules/betrieb-chronik.md Abschnitt 260812 («Wartezimmer ohne Arzt», 178 Skips, 44 uncommittete Dateien)

03.1.10 Betriebslogik als Prosa in Prompts und Kommentaren; keine Tests, keine CI, Job-Definitionen kaum versioniert (mittel, Testluecke). Die Zielauswahl der Nachtschicht ist ein einzelner Prompt-Absatz mit hart kodiertem Fortschrittsstand, Rücknahmen und Korrekturen, was der eigenen Rule 260830 widerspricht. Befristete Massnahmen stehen nach Ablauf weiter im Code. Von rund 30 referenzierten launchd-Labels liegen nur 7 plists im Repo; es gibt keine CI, kein shellcheck, keine automatisierten Tests der Laufzeit-Scripts. Beleg: scripts/nachtschicht-run.sh Z. 126; scripts/cron-training-mini.sh Z. 9-10 und Z. 17-20 (Befristung 10./11.08.2026 abgelaufen, Budget 25 USD steht noch); find nach *.plist: 5 unter templates/launchd/, 2 unter scripts/; kein Verzeichnis .github/workflows; Hub-Audit Abschnitt «Der Nachtschicht-Guard ist Prosa statt Code»

03.1.11 Stationsidentität uneinheitlich, Zwei-Stationen-Annahme hart verdrahtet (mittel, Skalierung). Vier verschiedene Verfahren bestimmen die Station (hostname -s, ComputerName, Hardware-Modell, ~/jans-station); das Journal führt vier Schreibweisen für drei Stationen, und zwei Stationsnamen kollidieren auf dem NAS nur in der Gross-/Kleinschreibung. Die Arbeits-Weiche kennt nur mini und macbook, das Dispatch-Ziel (Benutzer und Netzadresse) steht als Default in Script und README. Eine vierte Station oder ein Mitarbeitender passt in dieses Modell nicht. Beleg: scripts/claude-run.sh Z. 72; lauf-gate.sh Z. 36 und Z. 50-54; sync-task-run.sh Z. 42-57; arbeits-weiche.sh Z. 75-80; kontingent-budget.sh Z. 83-89; dispatch-run.sh Z. 129; Lauf-Journal Feld station: «Macmini», «mac-mini», «Macbookpro», Langname der dritten Station

03.1.12 Lauf- und Dispatch-Protokolle mit Inhaltsausschnitten ohne Schutz- und Löschkonzept (mittel, Datenschutz). Jede Journalzeile trägt 600 Zeichen Antworttext, jedes Dispatch-Protokoll den vollen Auftrag und die volle Antwort; das Repo hält selbst fest, dass darin Kundendaten und Beträge stehen können. Die Dateien liegen unverschlüsselt auf dem allgemeinen NAS-Share, von jeder Station les- und schreibbar. Eine Klassifizierung, Aufbewahrungsfrist oder Löschroutine ist im Subsystem nicht geregelt (DSG: Datenminimierung, Speicherbegrenzung). Beleg: .gitignore Kommentar zu logbuch/laeufer/; scripts/claude-run.sh Z. 122 (result_tail 600 Zeichen); scripts/dispatch-run.sh Z. 155-173 und Z. 228; /Volumes/daten/jans-ai-hub/dispatch/log/ mit 397 Protokollen (Zählung 19.09.2026)

03.2 Kennzahlen

03.2.1 Parallele Scheduler / Feuermechanismen: 7 (App-Tasks je Station, launchd, DSM-Aufgabenplaner, User-Crontab, Git-Hook-Runner, Dispatch, nohup-Treiber). Quelle: scripts/lauf-gate.sh Z. 8-19, remote-tasks/README.md, scripts/cron-training-mini.sh, Hub-Audit 260812 Abschnitt 1.5

03.2.2 Dateibasierte Queues: 5 (sync-tasks, freigabe, remote-tasks/pending, logbuch/widerruf, commit-*.task). Quelle: scripts/sync-task-run.sh, scripts/widerruf-queue.sh, remote-tasks/runner.sh

03.2.3 Gespiegelte Scheduled-Task-Definitionen: 46 (34 MacBook Pro, 12 Mac Mini); davon 3 mit dokumentiertem Takt-Feld. Quelle: templates/scheduled-tasks/ (Zählung 19.09.2026)

03.2.4 Geladene launchd-Jobs auf dem MacBook Pro: 15; im Repo versionierte plists gesamt: 7; referenzierte Labels im Repo: rund 30. Quelle: launchctl list (lesend, 19.09.2026), find *.plist, grep über scripts/ docs/ rules/

03.2.5 Laufzeit-Scripts im Untersuchungsumfang: 18 Scripts, 3'504 Zeilen bash; scripts/ gesamt 84 .sh-Dateien. Quelle: wc -l, ls scripts/

03.2.6 Journalisierte headless-Läufe 29.07.–19.09.2026: 2'543, davon 1'091 fehlgeschlagen (43 %): 594 Auth/OAuth, 477 Session-/Usage-Limit, 11 Wochenlimit, 5 Budgetdeckel. Quelle: /Volumes/daten/jans-ai-hub/logbuch/laeufe/*.jsonl, Eigenauswertung nach Stichwort im Feld result_tail

03.2.7 Kosten der journalisierten Läufe (API-Äquivalent, vom Abo gedeckt): 2'393 USD in rund 7.5 Wochen, 112 h Laufzeit; App-Scheduled-Tasks sind darin NICHT enthalten. Quelle: Lauf-Journal, Feld cost_usd

03.2.8 Läufe aus Treibern ausserhalb des Repos (mschub/vollschub): 1'231 (48 % aller journalisierten Läufe). Quelle: Lauf-Journal, Feld loop; betrieb-chronik.md Z. 1176

03.2.9 Ertraglose Schub-Läufe 29./30.08.2026: 699 Läufe, rund 240 USD API-Äquivalent. Quelle: Lauf-Journal (vier schub-Lanes); Rule auto-verbesserungen 260830

03.2.10 Gate-Abweisungen: Mac Mini 687 von 1'442 Einträgen (387 wegen Wochenkontingent); MacBook Pro 143 von 279. Quelle: /Volumes/daten/jans-ai-hub/logbuch/speicher/gate-*.log

03.2.11 Kalibrierung Wochenbudget: 167 Mio «teure» Token aus 1 Limit-Ereignis; Warnung 70 %, Drossel 85 %. Quelle: scripts/kontingent-budget.sh Z. 24-35, 76-78

03.2.12 Belegte Stillstände: 35 h Totalausfall 26./27.07.; 47 h leeres Wochenkontingent 01.–03.08.; 4 Tage ohne Briefing 09.–12.08.; 41 h blockierte Sync-Kette 11./12.08.. Quelle: scripts/vollgas-runner.sh Kopf; scripts/kontingent-budget.sh Kopf; betrieb-chronik.md 260812b und 260812

03.2.13 Umbau lauf.sh: gebaut 03.09.2026, 0 produktive Aufrufer, 1 von 4 Verifikationskriterien erfüllt. Quelle: docs/konzepte/260830-Laufzeitschicht-Umbau/VERIFIKATION.md Z. 30

03.2.14 Budgetdeckel je Lauf (uneinheitlich): Dispatch/Nachtschicht 5 USD, Schub 15 USD, cron-training 25 USD, claude-run Default 50 USD. Quelle: dispatch-run.sh Z. 42, vollgas-schub.sh, cron-training-mini.sh Z. 20, claude-run.sh Z. 39

03.2.15 Queue-Durchsatz und Latenz: 184 erledigte Sync-Tasks, 397 Dispatch-Protokolle; Sync-Task-Poll 30 Min, Remote-Task-Latenz 5–10 Min je Richtung. Quelle: /Volumes/daten/jans-ai-hub/sync-tasks/done, dispatch/log; remote-tasks/README.md Z. 21

03.2.16 Markdown-Register als Betriebsdatenbank: RADAR.md 952 kB, FRUEHWARNUNG.md 341 kB (versioniert, von Aufsichtsläufen gelesen). Quelle: logbuch/vollgas/ (wc -c)

03.3 Offene Fragen

03.3.1 Live-Zustand der Scheduled-Task-Registries: wie viele Tasks sind je Station tatsächlich aktiv und mit welchem Takt? Nur in der App abfragbar; der Repo-Spiegel ist ausdrücklich Doku. Gilt auch für die dritte Station.

03.3.2 Fünf Tasks sind auf beiden Stationen gespiegelt (vollgas-chef-radar, vollgas-fruehwarnung, heartbeat-daily, bauleitung-training, baurecht-buch-training): welche Instanz ist jeweils aktiv?

03.3.3 Ist die User-Crontab auf dem Mac Mini (cron-training-mini.sh, befristet bis 10./11.08.2026, Budget 25 USD) noch eingetragen? Auf dem MacBook Pro ist die Crontab leer; der Mini wurde nicht gemessen.

03.3.4 Welche launchd-Jobs sind auf Mac Mini und dritter Station geladen, und wo liegen deren plists (nur 7 im Repo)?

03.3.5 Ist die Team-Abo-Migration (zwei Seats) vollzogen, und welches Konto läuft auf welcher Station? Stand im Repo: Entscheidungsvorlage vom 13.08.2026, nichts umgesetzt.

03.3.6 Ist unbeaufsichtigter Dauerbetrieb über Max-/Team-Abo mit OAuth-Token von den Anthropic-Nutzungsbedingungen gedeckt, und was würde derselbe Betrieb über API/Agent SDK kosten? Das Journal weist rund 2'393 USD API-Äquivalent in 7.5 Wochen aus, ohne die App-Scheduled-Tasks.

03.3.7 Welchen Anteil am Wochenkontingent haben die App-Scheduled-Tasks, die nicht im Lauf-Journal erscheinen? Anhaltspunkt laufende Woche: MacBook Pro 42.8 Mio gegen Mac Mini 16.8 Mio «teure» Token.

03.3.8 Offene Inhaber-Entscheide aus der Spec: F1 (NAS als Git-Remote über SSH) und F2 (sollen Scheduled Tasks bewusst ein unabhängiger Auth-Kanal bleiben?) sowie die Freigabe für Phase 3 des Umbaus.

03.3.9 Die Ursache von B4 («OAuth session expired» bei abgelösten Läufen) ist eine unverifizierte Hypothese; ein realer Ablösetest mit und ohne unset fehlt.

03.3.10 Woher stammen die Treiber mit den Laufnamen mschub und rev (667 bzw. 115 Läufe)? Im Repo nicht auffindbar.

03.3.11 Gibt es ein Aufbewahrungs- und Löschkonzept für /Volumes/daten/jans-ai-hub/dispatch/log und logbuch/laeufe, und wer hat auf dem NAS-Share Lese- und Schreibrechte auf die Queue-Ordner (sync-tasks, widerruf)?

03.3.12 Wie wird der DSM-Aufgabenplaner (nas-selfcommit, 15 Min, root) überwacht und gesichert? Laut Hub-Audit ohne sudo nicht prüfbar.

04 Sicherheit, Identität und Datenschutz

04.0 Ist-Zustand. Repo-Root aller Belege: ~hub/.claude/worktrees/jans-ai-hub-architecture-4e38fe (Git-Spiegel); NAS-Pfade sind als /Volumes/daten/jans-ai-hub/... ausgewiesen. Die Sicherheit des Hub

beruht auf vier selbstgebauten Schichten: (1) einer Aktions-Whitelist v2 mit Autonomieklassen A1 bis A9 und Widerrufsfenster (30/60 Min, Veto per Mailantwort), (2) einer Regex-Freigabe-Schwelle für die Sync-Task-Queue, (3) Verhaltensregeln für interaktive Eingriffe und (4) dem Auto-Mode-Klassifikator von Claude Code als faktisch einziger technischer Schranke. Darunter liegt eine Allowlist mit Bash(*), Write(*), Edit(*) und sämtlichen MCP-Servern bei leerer Deny-Liste. Secrets liegen als Klartextdateien (chmod 600) im Home jeder Station, werden per scp zwischen den Stationen kopiert und sind sauber aus Git ausgeschlossen; nur bexio nutzt neu den Schlüsselbund. Die M365-App arbeitet app-only mit tenantweiten Rechten (SharePoint FullControl, Graph ReadWrite auf Mail/Files/Group/User, Mail.Send). Die drei Macs sind per SSH passwortlos voll vermascht, das mobile MacBook läuft bewusst ohne Sleep, ohne MDM, mit abgeschalteter Firewall. Der Agent liest täglich Mails samt Anhängen und darf an bekannte Empfänger versenden sowie seit 17.09.2026 ohne Betragsgrenze buchen. Prompt-Injection, Schweizer DSG, Auftragsbearbeitung und Datenklassifizierung kommen im ganzen Repo nicht vor, während Mailkorpora, Kontaktregister und Fristenregister versioniert und alle 15 Minuten nach GitHub gepusht werden. Die Vorfallskultur (Chronik mit Ursachenanalyse, Klassifikator wird nie umgangen) ist bemerkenswert reif, die Technik darunter ist es nicht.

04.1 Schwachstellen

04.1.1 M365-App mit tenantweiten Vollrechten, Private Key unverschlüsselt auf drei Stationen

(kritisch, Least Privilege / Secrets). Die App-Registrierung arbeitet app-only mit SharePoint Sites.FullControl.All, Graph ReadWrite auf Files/Group/Mail/Sites/User und Mail.Send (Versand als jeder Benutzer). App-only-Zertifikate unterliegen weder MFA noch benutzerbezogenem Conditional Access. Der Private Key wird mit -nodes unverschlüsselt erzeugt, liegt als PEM im Home und wird per scp zwischen Stationen kopiert: drei Stationen teilen sich zwei Zertifikate, ein einzelnes Gerät lässt sich nicht gezielt sperren. Auf dem Mac Mini liegt zudem noch die .env einer alten App mit Client-Secret. Wer eine PEM-Datei besitzt, hat Vollzugriff auf alle Postfächer und alle SharePoint-Sites des Tenants. Beleg: connectors/hub-setup-daten.json Schlüssel m365_connector/berechtigungen; .env.example:9-11; rules/betrieb-chronik.md:97-110 (scp-Kopie, zwei Zertifikate, altes AZURE_CLIENT_SECRET-Schema); scripts/zugangsdatei-sync.sh:23

04.1.2 Agent hat faktisch uneingeschränkte Rechte, einzige technische Schranke ist ein

Hersteller-Klassifikator (kritisch, Berechtigung / Sandbox). Die versionierte Allowlist gibt Bash(*), Read(*), Edit(*), Write(*), WebFetch(*), alle MCP-Server inklusive computer-use und Control_your_Mac frei, die Deny-Liste ist leer, dazu Leserecht auf den ganzen NAS-Share und Schreibrecht auf alle geteilten OneDrive-Bibliotheken. Dieselbe Liste gilt für rund 40 unbeaufsichtigte Läufe (acceptEdits). Es gibt keine Rollenprofile je Task, keinen eigenen OS-Benutzer, keine Sandbox, keine Egress-Kontrolle. Die Whitelist A1 bis A9 ist reine Prompt-Policy. Der Auto-Mode-Klassifikator ist eine Blackbox des Anbieters ohne Zusicherung und kann sich mit jedem Update ändern. Beleg: .claude/settings.json:4-33 (allow/deny); scripts/dispatch-run.sh:36-39; scripts/claude-run.sh:74; logbuch/AKTIONS-WHITELIST.md:86-91

04.1.3 Prompt-Injection ist nirgends adressiert, obwohl der Agent Mails liest und handeln darf

(kritisch, Sicherheit / KI-spezifisch). Tägliche Loops sichten den Maileingang, Anhänge sind laut Regel Pflichtlektüre, dazu Web- und Portalinhalte. Derselbe Kontext hält Bash(*), M365-Vollzugriff und Versandvollmacht (A7/A8 laufen nach 30 bzw. 60 Minuten automatisch). Eine präparierte Mail oder ein PDF kann damit Datenabfluss, Versand im Namen des Büros oder Codeausführung auslösen. Im gesamten Repo (Rules, Skills, Agents, Scripts, Konzepte) gibt es keinen einzigen Treffer zu Prompt-Injection; es fehlt die Trennung zwischen lesendem und handelndem Agenten, jede Inhaltsquarantäne und jeder Red-Team-Test. Beleg: grep -ri 'prompt.injection' über *.md/*.sh/*.mjs: 0 Treffer;

logbuch/AKTIONS-WHITELIST.md:33-35 und 49-56; skills/hub-chef/SKILL.md:75; rules/auto-verbesserungen.md Eintrag 260709

04.1.4 Drei Ausführungskanäle mit eval ohne Authentisierung des Inhalts (hoch, Eigenbau / Remote Code Execution). (a) remote-tasks/runner.sh führt jede nach GitHub gepushte .sh-Datei mit Benutzerrechten aus, ohne Guard; das Sicherheitsmodell lautet wörtlich, wer pushen darf, darf ausführen. (b) Die Sync-Task-Queue liegt auf einem SMB-Share und wird alle 30 Minuten per eval geleert; der Schutz ist eine Regex-Blacklist mit 15 Mustern, die bereits ein `rm -rf` auf Git-Internas durchliess und durch Umschreiben (`python -c`, Variablen, Base64 in Datei) trivial zu umgehen ist. (c) Die Widerrufs-Queue evaluiert das Feld `befehl=` aus einer Task-Datei auf dem NAS. Keine Signaturen, keine Befehls-Allowlist, keine Isolation. Beleg: remote-tasks/runner.sh:73-75; remote-tasks/README.md:69-71; scripts/sync-task-run.sh:174; scripts/sync-task-guard.sh:43-64; rules/betrieb-chronik.md Abschnitt 260811 (ab Zeile 2140); scripts/widerruf-queue.sh:165-171

04.1.5 Geräteverlust: mobiles Always-On-MacBook trägt alle Schlüssel, kein MDM, kein Notfall-Runbook (hoch, Geräteverwaltung / SPOF). Am MacBook Pro gemessen (19.09.2026): keine MDM-Registrierung, Firewall aus, SSH (22), Bildschirmfreigabe (5900) und SMB (445) lauschen auf allen Interfaces, sshd ohne eigene Härtung (Passwort-Login nicht abgeschaltet), SSH-Private-Key ohne Passphrase. Im Home liegen mindestens acht Klartext-Secrets (M365-Zertifikatsschlüssel, bexio-Token, Claude-Token, Portalzugänge). Von dort geht es passwortlos auf Mac Mini, dritte Station und NAS. Das Gerät ist bewusst ohne Sleep konfiguriert, FileVault schützt ein laufendes Gerät nicht. Ein Verlust-, Sperr- oder Rotationsablauf ist nirgends dokumentiert. Beleg: Messung MacBook Pro: profiles status (MDM No), socketfilterfw (disabled), netstat LISTEN 22/445/5900, ssh-keygen-Probe; rules/betrieb-chronik.md:1033-1035 (sechs Richtungen passwortlos); grep nach Diebstahl/revoke/Rotation in docs/referenz und rules: 0 Treffer

04.1.6 Autonome Buchhaltung ohne Betragsgrenze, Governance-Widerspruch und nur sprachliche Zahlungssperre (hoch, Governance / Finanzkontrolle). Seit 17.09.2026 bucht der Agent Bankbewegungen selbständig, ohne Betragsgrenze, monatlich unbeaufsichtigt auf dem Mac Mini. Die Spec hält fest, dass es keinen Storno-Code gibt, während die weiterhin gültige Whitelist v2 für A9 Stornierbarkeit und ein 60-Minuten-Widerrufsfenster verlangt; die Whitelist wurde nicht nachgeführt. Geplant ist zusätzlich der Scope `bank_payment_edit`, der Zahlungsaufträge technisch ermöglichen würde. Die Zahlungssperre besteht aus einer Regel und einem Stichwort-grep auf den Beschreibungstext. Haftung, Vier-Augen-Prinzip und Abstimmung mit dem Treuhänder sind nicht geregelt. Beleg: rules/auto-verbesserungen.md Eintrag 260702 (Neufassung 260917); wissen/spec/outputs/2026-09-17_bexio-selbstaendig-buchen_spec.md:7 und :11; logbuch/AKTIONS-WHITELIST.md:35 und 101-104; scripts/widerruf-queue.sh:103

04.1.7 Personendaten Dritter in Git, auf GitHub und in jedem LLM-Kontext; kein DSGVO-Rahmen (hoch, Datenschutz (DSG)). Versioniert und alle 15 Minuten per `git add -A` nach GitHub gepusht werden unter anderem: ein Kontaktregister mit Namen, Arbeitgebern und Rollen (per `@-Import` in jeder Session im Modellkontext), 60 Rohdateien mit authentischen Mailkorpora, eine Kunden-KB, das Fristenregister (1.1 MB) und das Logbuch. Am 23.08.2026 gelangte der Name einer bewerbenden Person in einen Push, die Historie wurde bewusst nicht bereinigt. Es gibt kein Bearbeitungsverzeichnis, keine dokumentierten Auftragsbearbeitungsverträge (Anthropic, Microsoft, GitHub, bexio, Tailscale, Hoster), keine Regelung zur Auslandsbekanntgabe, keine Datenklassifizierung, kein Löschkonzept und kein Secret- oder PII-Scanning vor dem Commit. Beleg: `git ls-files`: rules/anrede-kontakte.md, wissen/twin/raw (60 Dateien), wissen/kunde-<name> (18), logbuch/fristen.md; scripts/nas-selfcommit.sh:81; rules/auto-verbesserungen.md Eintrag 260823; grep nach DSGVO/Auftragsbearbeitung/Bearbeitungsverzeichnis in rules, skills, agents, docs: 0 Treffer

04.1.8 Freigabe-Rückkanal authentisiert nur über die Absenderadresse (mittel, Authentisierung). freigabe-inbox.sh durchsucht alle Konten und alle Postfächer von Apple Mail (also auch Junk und das private iCloud-Konto) und akzeptiert jede Mail, deren From-Adresse dem Inhaber entspricht. Die im Kommentar behauptete SPF-/DKIM-Prüfung findet im Script nicht statt. Ein JA <id> löst die sofortige Ausführung einer angemeldeten Aussenwirkung aus; die Kennung ist aus Zeitstempel und PID vorhersagbar und steht in der Meldemail. Ein gefälschter Absender genügt, um ein Widerrufsfenster zu überspringen. Beleg: scripts/freigabe-inbox.sh:19-21 (Behauptung), :82-90 (alle Accounts/Mailboxen, reiner Adressvergleich), :124-127 und :146-156; scripts/widerruf-queue.sh:109

04.1.9 Konten- und Schlüsselhygiene: Admin-Konto im Alltag, Standardkonto des Refurbishers, Dauer-Schlüssel (mittel, Identität / Governance). OneDrive synchronisiert auf den Arbeitsstationen unter dem Tenant-Administratorkonto statt unter dem Arbeitskonto. Die dritte Station läuft unter dem vorgefundenen Benutzerkonto des Refurbishers. Die Synology hält einen passphrasenlosen SSH-Schlüssel als GitHub-Deploy-Key mit Schreibrecht und hat SSH aktiv. Auf dem Mac Mini ist der Tailscale-Schlüsselablauf deaktiviert, und er ist Subnet-Router für das gesamte Büro-LAN. Keine Trennung zwischen privat und geschäftlich (private iCloud- und Outlook-Konten im selben Mail-Client, den der Agent steuert). Beleg: connectors/WEGE.md:1249; rules/betrieb-chronik.md:1026-1027 und :1062-1067; docs/plan-dsm-git.md:45-46; connectors/hub-setup-daten.json Schlüssel stationen/macbook-revendo/benutzer und mail_konten

04.1.10 Infrastruktur-Intern im GitHub-Backup (mittel, Sicherheit / Informationsabfluss). Netzadressen, Tenant- und App-Kennungen, Zertifikats-Fingerprints, Netztopologie und Störungsdetails stehen in versionierten Dateien; 42 Dateien enthalten Tailnet-Adressen, ein Script trägt eine Stationsadresse fest verdrahtet. Ein Leck des privaten Repos oder des GitHub-Kontos liefert einem Angreifer die vollständige Karte samt Angriffszielen. Der 2FA- und Zugriffsstand des GitHub-Kontos ist im Hub nicht dokumentiert. Beleg: grep nach Tailnet-Adressmuster: 42 versionierte Dateien; connectors/hub-setup-daten.json (lan_ip, tailscale_ip, app_id, tenant_id); rules/betrieb-chronik.md:98-100 und :1049-1050; scripts/trust-check.sh:37

04.1.11 Schutzmechanismen ohne automatisierte Tests, Hooks fail-open, Injection in Shell-Scripts (mittel, Testlücke / Eigenbau). Der tägliche Selbsttest prüft nur Lauf-Gate und Speicher-Wächter; für Sync-Task-Guard, Zahlungssperre, Freigabe-Rückkanal und Mail-Guard gibt es keine Regressionstests, kein CI und kein Review durch eine zweite Instanz. Alle Hooks enden mit || true und fallen bei Fehlern still aus. versand-mail-senden.sh und widerruf-queue.sh setzen Betreff und Adresse ungeschützt in AppleScript ein (ein Anführungszeichen bricht aus). 6 Scripts nutzen eval, 21 rufen ssh auf. Beleg: scripts/schutzmechanik-selbsttest.sh:55-169 (Prüfumfang); .claude/settings.json:50, :62, :74, :85 (|| true); scripts/versand-mail-senden.sh:43-49; scripts/widerruf-queue.sh:63-72; kein tests/, kein .github/, kein pre-commit im Repo

04.1.12 Audit-Trail nicht manipulationssicher und verstreut; FileVault-Abbau als Empfehlung im Raum (mittel, Audit / Governance). Logbuch, versand.log, Dispatch-Logs und Runner-Logs sind normale Dateien auf dem SMB-Share, schreibbar durch denselben Agenten, den sie protokollieren; das Lauf-Journal liegt nur lokal je Station. Es gibt keine zentrale, unveränderliche Protokollierung und keine Korrelation mit M365-Audit-Logs. Zwei Konzeptdokumente empfehlen, auf dem Mac Mini FileVault abzuschalten und Auto-Login zu setzen, um Neustartfestigkeit zu erreichen: genau die Station, die künftig Aufsicht, Buchungslauf und alle Secrets trägt, verlöre den Schutz ruhender Daten. Beleg: scripts/widerruf-queue.sh:78-81; scripts/versand-mail-senden.sh:21 und :59; .gitignore:30-35; docs/konzepte/260812-Hub-Audit/260812-Hub-Audit.md:672-684; docs/konzepte/260911-Systemanalyse-Hub-Architektur-NAS-Git/260911-Systemanalyse-Hub-Architektur-NAS-Git.md:190

04.2 Kennzahlen

04.2.1 Allowlist-Einträge allow / deny: 28 / 0 (darunter Bash(*), Write(*), Edit(*), WebFetch(*), 14 MCP-Wildcards). Quelle: .claude/settings.json:3-33

04.2.2 Autonomieklassen: A1 bis A9; Widerrufsfenster 30 Min (Mail) und 60 Min (Vertragswirkung, Buchungen); Queue-Takt 5 Min. Quelle: logbuch/AKTIONS-WHITELIST.md:25-35 und :76-79

04.2.3 Muster der Freigabe-Schwelle: 15 Regex-Muster (Blacklist); 1 belegtes Falsch-Negativ (rm -rf relativ, 11.08.2026). Quelle: scripts/sync-task-guard.sh:43-64; rules/betrieb-chronik.md Abschnitt 260811

04.2.4 Sync-Tasks erledigt / in Freigabe wartend: 184 / 1 (wartet seit 18.09.2026). Quelle: /Volumes/daten/jans-ai-hub/sync-tasks/done und /freigabe/macbook-pro (Dateiliste)

04.2.5 Widerrufs-Queue Vorgänge: 3 erledigt, 1 gestoppt, 0 offen. Quelle: /Volumes/daten/jans-ai-hub/logbuch/widerruf/ (Dateiliste)

04.2.6 Dispatch-Audit-Logs: 397 Dateien, 2.6 MB. Quelle: /Volumes/daten/jans-ai-hub/dispatch/log (Dateiliste)

04.2.7 Klartext-Secret-Dateien je Station (referenziert): mindestens 8 verschiedene (M365-Zertifikat kombiniert und Key, Hub-.env, bexio, dispatch, versand, zefix, truninger, ebaugesuche); dazu 1 Altdatei eines abgelösten Tokens. Quelle: grep über scripts/, connectors/, skills/; ls -la im Home des MacBook Pro (nur Namen, Rechte 600)

04.2.8 M365-Zertifikate / Stationen: 2 Zertifikate auf 3 Stationen, Laufzeit 2 Jahre (bis 03/2028), Private Key unverschlüsselt. Quelle: rules/betrieb-chronik.md:97-103; .env.example:9; CLAUDE.md Abschnitt M365 Connector

04.2.9 M365-App-Rechte (Art): Application-Permissions: Graph Files/Group/Mail/Sites/User Read bzw. ReadWrite, Mail.Send, SharePoint Sites.FullControl.All (tenantweit). Quelle: connectors/hub-setup-daten.json Schlüssel m365_connector/berechtigungen

04.2.10 SSH-Vermaschung: 6 von 6 Richtungen passwortlos zwischen 3 Macs, zusätzlich Stationen zu NAS; Private Key MacBook Pro ohne Passphrase. Quelle: rules/betrieb-chronik.md:1033-1035; Messung 19.09.2026

04.2.11 Gerätezustand MacBook Pro (gemessen 19.09.2026): FileVault an, Firewall aus, MDM nein, Ports 22/445/5900 offen auf allen Interfaces. Quelle: fdesetup status, socketfilterfw --getglobalstate, profiles status, netstat

04.2.12 Versionierte Dateien / davon mit Personenbezug (Auswahl): 5605 gesamt; 60 Mailkorpus-Rohdateien, 18 Kunden-KB-Dateien, Kontaktregister, Fristenregister 1.1 MB; Auto-Push alle 15 Min. Quelle: git ls-files; scripts/nas-selfcommit.sh:81

04.2.13 Versionierte Dateien mit Tailnet-Adressen: 42. Quelle: grep über *.sh, *.md, *.json, *.mjs

04.2.14 Treffer Prompt-Injection / DSG im Hub: 0 / 0. Quelle: grep über rules, skills, agents, scripts, docs, services

04.2.15 Erster autonomer Buchungslauf: 88 Buchungen, CHF 1'637.91, ohne Betragsgrenze; erster unbeaufsichtigter Monatslauf geplant 02.10.2026. Quelle: wissen/spec/outputs/2026-09-17_bexio-selbstaendig-buchen_spec.md:11 und Plan-Abschnitt

04.2.16 Scripts mit eval / mit ssh: 6 / 21 von 84 in scripts/. Quelle: grep über scripts/*.sh

04.3 Offene Fragen

04.3.1 GitHub-Konto des Hub-Repos: 2FA-Stand, weitere Collaborators, Deploy Keys, Branch Protection, welche Tokens existieren? Im Hub nicht dokumentiert, aber wegen remote-tasks/runner.sh sicherheitskritisch.

04.3.2 Tailscale: Gibt es ACLs, Tags und Device Approval, oder darf jedes Tailnet-Gerät alles erreichen (inklusive Subnet-Route ins Büro-LAN)? Schlüsselablauf auf den übrigen Geräten?

04.3.3 Synology DS918: Ist das für SSH und SMB genutzte Konto ein DSM-Administrator? 2FA auf DSM, Volume-Verschlüsselung, unveränderliche Snapshots, Offsite-Backup, SMB-Rechte je Benutzer?

04.3.4 OPNsense: Regelwerk, Portweiterleitungen, VLAN-Trennung (Büro, Gäste, IPTV, Drucker), Firmware-Stand? Nicht Teil des Repos.

04.3.5 Sind Find My und Activation Lock auf den drei Macs aktiv, und wie kurz ist die Bildschirmsperre auf dem Always-On-MacBook?

04.3.6 Sind die SSH-Private-Keys auf Mac Mini und dritter Station ebenfalls ohne Passphrase? Gemessen wurde nur das MacBook Pro.

04.3.7 Hat das vorgefundene Benutzerkonto der Refurbished-Station Administratorrechte, und wurde das Gerät vor Inbetriebnahme frisch installiert?

04.3.8 Ist die alte M365-App mit Client-Secret (Schema AZURE_CLIENT_SECRET in der .env des Mac Mini, rules/betrieb-chronik.md:105-110) in Entra noch aktiv und berechtigt?

04.3.9 M365: Security Defaults oder Conditional Access aktiv? Ist der Telefon-Rückfall für MFA noch hinterlegt? Wird das Admin-Konto lizenziert im Alltag verwendet? (Stand nur aus Sitzungsnotizen, nicht aus dem Repo belegt.)

04.3.10 Anthropic: Unter welchen Vertragsbedingungen läuft das Max-Abo (Consumer- oder Commercial-Terms), ist das Training mit eigenen Daten ausgeschlossen, gibt es einen Auftragsbearbeitungsvertrag? Entscheidend für die DSGVO-Beurteilung.

04.3.11 Bestehen Geheimhaltungsvereinbarungen mit Bauherrschaften (insbesondere im Spitalprojekt), die eine Bearbeitung in Cloud-KI oder die Ablage auf GitHub einschränken?

04.3.12 Wer trägt die Verantwortung für autonome Buchungen gegenüber Treuhänder und Steuerbehörde, und deckt die Berufs- oder Cyberversicherung KI-automatisierte Handlungen ab?

04.3.13 Inhalt von .claude/settings.local.json (nicht versioniert, nicht geprüft): laut Rule 260917b stehen dort weitere Freigaben, darunter hyperspezifische ssh-Einträge.

04.3.14 Wie werden Secrets nach Geräteverlust oder Defekt wiederhergestellt (Backup der Zertifikate und Tokens), und wer ausser dem Inhaber hat im Notfall Zugriff (Stellvertretung, Nachfolge)?

05 Connectoren und Integrationen

05.0 Ist-Zustand. Gemessen am 19.09.2026 im Git-Spiegel (WT = ~hub/.claude/worktrees/jans-ai-hub-architecture-4e38fe, Commit 379a4ba76); alle Belegpfade unten sind relativ zu WT, NAS-Pfade sind ausgeschrieben. Das Subsystem besteht aus 19 selbst gebauten Netz-Connectoren (rund 6'900 Zeilen, fast durchwegs Node-ESM ohne Fremdbhängigkeiten, ein Python-Script), sechs Shell-Brücken

zu CAD und Druck, einem lokalen MCP-Server (CLI for Microsoft 365, Version 0.1.17, über Bash-Wrapper) und einem Remote-MCP (Figma). Dazu kommen pro Station im Browser eingerichtete claude.ai-Cloud-Connectoren. Jeder Connector ist eine eigene CLI mit eigenem Flag-Vokabular, eigenem env-Parser und eigener Fehlerbehandlung. Offizielle APIs überwiegen (Graph, bexio, Zefix, Geoshop, geo.admin, OEREB, GWR, Post); gescrapt wird bei Truninger DS3 (HTML plus Cookie) und eBaugesucheZH (Playwright-Login mit Mobile ID, Session rund zehn Tage). Zugangsdaten liegen an fünf Orten (Keychain, ~/.*.env, Projekt-.env, NAS-Ordner secrets/, PEM und CLI-Cache im Home) und werden von Hand zwischen den Stationen kopiert. Das Wissen über Wege und Sackgassen steckt in einem 94-kB-Prosa-Register (connectors/WEGE.md, 1'319 Zeilen, 21 Nachträge). Es gibt keine Tests, keine CI, kein Lockfile im Git, kein Retry und kein synthetisches Monitoring. Mehrere Integrationen waren wochenlang unbemerkt tot. Schreibende Aktionen (buchen, mahnen, Mail senden, Berechtigungen ändern) sind nur durch Code-Konvention (--ja) und den Auto-Mode-Klassifikator von Claude Code geschützt, bei Bash(*) und leerer Deny-Liste.

05.1 Schwachstellen

05.1.1 Eine überprivilegierte M365-App mit einem kopierten Schlüssel für alle Stationen

(kritisch, Sicherheit). Sämtliche M365-Zugriffe (CLI, MCP, eigener Graph-Connector, PnP) laufen app-only über eine einzige App-Registrierung mit mandantenweitem SharePoint-Vollzugriff, Mail.Send und Lese-/Schreibrechten auf Files, Group, Mail, Sites, User. Das selbstsignierte Zertifikat (RSA 2048, gültig bis 23.03.2028) liegt samt privatem Schlüssel als PEM im Home jeder Station, wird per Script kopiert und laut Erstellbefehl ohne Passphrase erzeugt. Es gibt keine Identität je Station oder Person, keinen gezielten Widerruf, kein Least Privilege. Der CLI-Verbindungs-cache enthält laut Doctor Schlüssel und Tokens im Klartext und lag bis 09.08.2026 mit 644. m365-graph.mjs gibt auf Wunsch ein Access-Token (auch Admin-Scope) auf stdout aus; das MCP-Tool run_command führt beliebige CLI-Befehle mit diesen Rechten aus. Beleg: connectors/WEGE.md:172-177; scripts/m365-mcp-server.sh:56 (openssl ... -nodes); connectors/m365-graph.mjs:249-260; scripts/zugangsdatei-sync.sh:22-30; scripts/wege-doctor.sh:226-236; /Volumes/daten/jans-ai-hub/logbuch/wege/doctor-Macmini.log (DEFEKT 644)

05.1.2 Schreibende Aktionen nur durch Konvention geschützt (kritisch, Governance). Die Projekt-Permissions erlauben Bash(*), WebFetch(*) und alle MCP-Server per Wildcard, die Deny-Liste ist leer. Ob ein Agent bucht, mahnt, Mails sendet, kostenpflichtige Post-Etiketten erzeugt oder SharePoint-Berechtigungen ändert, hängt am Flag --ja im Connector-Code, an Textregeln und am Auto-Mode-Klassifikator von Claude Code. Letzterer ist ein fremdes, nicht steuerbares und nicht spezifiziertes Produktverhalten. Seit 17.09.2026 bucht Claude Banktransaktionen ohne Betragsgrenze selbständig. Eine technische Freigabeschicht ausserhalb des Sprachmodells und ein unveränderliches Audit-Log fehlen. Beleg: .claude/settings.json:4 (Bash(*)) und :33 (deny leer); connectors/bexio.mjs:589-667; connectors/WEGE.md:500-512; Rule auto-verbesserungen 260702 Neufassung 260917

05.1.3 Kein Monitoring je Connector, Ausfälle bleiben wochenlang unbemerkt (hoch, Betriebsaufwand). Der Heartbeat prüft nur m365 status (lokaler Anmeldecache, kein echter API-Aufruf). Der Wege-Doctor prüft bei bexio, Truninger, Zefix, Versand und eBaugesuche lediglich, ob die env-Datei existiert, und läuft nicht geplant (je Station sechs Läufe, Mac Mini zuletzt 12.08.2026, MacBook 11.09.2026). Folgen: bexio vom 12.08. bis 17.09.2026 tot mit falscher Eigendiagnose; eBaugesuche seit Ende Juli mit HTTP 401; der M365-MCP-Server meldete vom 13.07. bis 25.07.2026 bei jedem Werkzeug einen Paketfehler, die Loops wichen still auf Apple Mail aus. Beleg: rules/betrieb-chronik.md:34-44; connectors/WEGE.md:548-560; scripts/m365-mcp-server.sh:27-38; scripts/heartbeat.sh:86-108; scripts/wege-doctor.sh:252-262; /Volumes/daten/jans-ai-hub/logbuch/wege/doctor-*.log

05.1.4 Fragile Eigenbau-Logins gegen Portale ohne Maschinenzugang (hoch, Eigenbau).

eBaugesucheZH hängt an einem Playwright-Durchlauf durch SAML-Login und Mobile ID; die Session lebt rund zehn Tage und braucht danach das Handy des Inhabers. Das Portal stellt auf AGOV um, der gebaute Weg trägt danach nicht mehr; die Session-Datei wurde zuletzt am 02.07.2026 geschrieben, der Connector war also nur Tage produktiv. Truninger DS3 wird per Form-Login und generischem Regex-HTML-Parsing gelesen; eine Suche über den ganzen Projektraum lief nach 70 Minuten noch. Jede Layout- oder Login-Änderung der Gegenseite bricht den Weg ohne Vorwarnung. Beleg: connectors/ebaugesuche-zh.mjs:10-24; connectors/WEGE.md:548-560; ~/ebaugesuche-zh.session.json (mtime 02.07.2026, nur Metadaten gelesen); connectors/truninger-ds3.mjs:29-32; connectors/README.md:210-211

05.1.5 Keine Tests, keine CI, keine reproduzierbaren Builds (hoch, Testlücke). Im Repo existiert keine einzige Testdatei, kein .github-Verzeichnis, kein Lint oder Typecheck. package-lock.json ist ausdrücklich gitignoriert, die vier Abhängigkeiten tragen ^-Bereiche, claude-usage lädt zur Laufzeit npx -y ccusage@latest. Mehrere Kopfkommentare halten fest, dass Endpunkte oder Feldnamen erst beim ersten echten Lauf zu bestätigen sind. Eine per Rule bekannte Fehlerklasse (UTC-Datum via toISOString) steckt weiterhin im bexio-Connector und fließt in die Verzugsberechnung und den Protokoll-Dateinamen ein, genau im Nachtfenster der automatischen Läufe. Beleg: .gitignore:20-22; package.json; connectors/claude-usage.mjs:213; connectors/versand/post-label.mjs:7-9; connectors/bexio.mjs:358, 387, 666; Rule dateinamen-konvention Abschnitt toISOString

05.1.6 Kein Retry, kein Backoff, vielfach kein Timeout (hoch, Skalierung). In keinem der 19 Connectoren findet sich Retry-, Backoff- oder 429-Behandlung. Elf Connectoren mit fetch-Aufrufen haben keinerlei Timeout-Bezug. Belegt ist ein Hänger von über vier Minuten bei m365-graph.mjs, dessen Ursache bis heute ungeklärt ist; als Abhilfe dient ein perl-alarm-Wrapper. bexio lädt Rechnungen mit limit=2000 ohne Paginierung, bei wachsendem Bestand wird still abgeschnitten. Für unbeaufsichtigte Nachtläufe ist das nicht tragfähig. Beleg: connectors/m365-graph.mjs:120, 146, 168; connectors/WEGE.md:1135-1150; connectors/bexio.mjs:161, 287, 388; grep-Messung über connectors/ und skills/*/connectors/

05.1.7 Secrets an fünf Orten, Verteilung und Rotation von Hand (hoch, Sicherheit). Zugangsdaten liegen im macOS-Schlüsselbund, in fünf ~/.env-Dateien mit Klartext-Passwörtern (darunter das Behördenportal-Passwort), in der Projekt-.env, im NAS-Ordner secrets/ auf dem SMB-Share (für jede gemountete Station und künftige Mitarbeitende lesbar; der Google-Key wird als Kommandozeilenargument übergeben) sowie als PEM und CLI-Cache im Home. Rotation und Ablauf überwacht nur das M365-Zertifikat. Der bexio-PAT verfällt alle 60 Tage und bleibt für den Bank-Endpunkt zwingend, weil OIDC dort 403 liefert; die selbständige Monatsbuchung hängt damit an einem manuellen Re-Login. In ssh- und launchd-Kontexten ist der Schlüsselbund gesperrt (rc=36). Zefix-Zugangsdaten fehlen seit dem Bau des Connectors. Beleg: connectors/README.md:54-60, 114-120, 154-159; skills/unternehmerfindung/connectors/set-google-key.sh:15, 38-41; connectors/bexio.mjs:255-262; connectors/bexio-auth.mjs:55-60; connectors/WEGE.md:1251-1263; docs/konzepte/260812-Hub-Audit/260812-Hub-Audit.md:111-123, 337-342

05.1.8 CLI-Wildwuchs statt einheitlicher Schnittstelle, MCP nur als Durchreicher (mittel, Eigenbau). 19 CLIs mit je eigenem Flag-Vokabular (deutsch und englisch gemischt), eigenem env-Parser pro Datei, uneinheitlichen Exit-Codes und nur teilweiser JSON-Ausgabe. Die hauseigene Konvention --hilfe erfüllen 7 von 19 Connectoren. Der einzige lokale MCP-Server ist ein generischer Befehls-Durchreicher in Version 0.1.17, der nur dank Symlink-Trick auf einen benutzereigenen npm-Prefix startet und an .env und node_modules im Projektpfad hängt: in einem Git-Worktree fehlen beide, in dieser Analyse-Session meldete der Server CONNECTION_CLOSED. Typisierte, fachliche MCP-

Tools (etwa oereb_auszug, bexio_offene_posten) gibt es nicht; das Modell muss Shell-Aufrufe aus Prosa zusammensetzen. Beleg: connectors/README.md:13-14 gegen grep-Messung; scripts/m365-mcp-server.sh:15-22, 27-44; .mcp.json; Worktree ohne .env und node_modules (gemessen 19.09.2026)

05.1.9 Identifikatoren, Topologie und Privates im Geschäfts-Repo (mittel, Datenschutz). Die M365-App-ID steht in 43, die Tenant-ID in 9 getrackten Dateien, dazu Site- und Drive-IDs, Collection-IDs eines Kundenprojektraums und die Netzwerktopologie in hub-setup-daten.json; alles wird automatisch nach GitHub gepusht. Das Wege-Register und scripts/ enthalten private Inhalte (IPTV-Bedienung samt Kontobezeichner, privates iCloud-Mailkonto als Default). Für den Ausbau mit Mitarbeitenden oder Mandanten fehlen Datenklassifikation je Connector, ein Bearbeitungsverzeichnis nach DSGVO und die Trennung privat und geschäftlich; der app-only Zugriff reicht technisch auf alle Postfächer des Tenants. Beleg: git grep (Zählung 19.09.2026); connectors/WEGE.md:63, 172-173, 379-388; scripts/iptv-*.py; connectors/icloud-mail.py:37; connectors/README.md:174-178

05.1.10 Prosa-Register statt maschinenlesbarem Katalog, Drift belegt (mittel, Betriebsaufwand). WEGE.md ist zugleich Runbook, Incident-Log, API-Dokumentation und Wissensspeicher. Es ist nicht prüfbar und driftet: die Fähigkeits-Matrix nennt connectors/geo-zh.mjs, die Datei liegt aber unter skills/planungsgrundlagen/connectors/; der Index erklärt sich zum einen Ort für jeden Connector, führt die fünf CAD-Brücken aber nicht; CLAUDE.md spricht von 16 Connectoren bei 19 Indexzeilen. Das Register vermerkt selbst, dass ein eingetragener Befehl zwei Läufe lang nicht gefunden wurde. Status (produktiv, tot, parkiert), Owner, Auth-Art und Testbefehl sind nirgends strukturiert erfasst. Beleg: connectors/WEGE.md:53, 1284-1287; connectors/README.md:10-11, 21-41; CLAUDE.md:207

05.1.11 Werkzeuge sind an Station, Benutzer und GUI gebunden (mittel, Skalierung). m365 liegt nur unter node_modules/.bin des SSD-Klons, node fehlt in nicht-interaktiven ssh-Shells im PATH, LibreDWG und das pdf2dwg-venv gibt es nur auf dem Mac Mini, die C4D-Lizenz ebenso, Rhino-Importe brauchen beim Erstkontakt einen Dialogklick an einer besetzten Station. Kalender und Kontakte laufen über osascript, CNContactStore und CGEvent-Klicks, weil der App die Graph-Berechtigung fehlt (403). Zwei Stationen teilen sich denselben Hostnamen im Doctor-Log. Default-Absender, Liefer-Mail und Home-Pfade sind hartcodiert. Das skaliert weder auf Mitarbeitende noch auf Mandanten. Beleg: connectors/WEGE.md:45-46, 96-110, 1151-1153; connectors/cad/README.md; /Volumes/daten/jans-ai-hub/logbuch/wege/doctor-Macbookpro.log (Eintrag 11.09.2026 mit fremdem Home-Pfad); skills/planungsgrundlagen/connectors/geoshop-zh.mjs:54

05.1.12 Usage-Connector greift in die Anmeldung des Kernwerkzeugs ein (mittel, SPOF). claude-usage.mjs liest das OAuth-Token der Claude-CLI aus dem Schlüsselbund, erneuert es selbst über einen nicht dokumentierten Endpunkt mit der Client-ID der CLI und schreibt das rotierte Refresh-Token in den Keychain-Eintrag der CLI zurück; das Token wird dabei als Prozessargument an security übergeben. Der Kommentar nennt zwei Ausfälle (12.07. und 19.07.2026) wegen der Rotation. Ein Fehler hier trifft den Login, an dem alle rund 40 Scheduled Tasks hängen. Die Schnittstelle kann vom Hersteller jederzeit geändert werden. Beleg: connectors/claude-usage.mjs:50-66, 73-80, 141-144

05.2 Kennzahlen

05.2.1 Eigenbau-Connectoren (Netz): 19 (18 Node-ESM, 1 Python), dazu 6 Shell-Brücken CAD/Druck und 1 Remote-Render-Script. Quelle: find/wc über connectors/ und skills/*/connectors/, 19.09.2026

05.2.2 Codeumfang Connectoren: rund 6'900 Zeilen; grösste Dateien bexio.mjs 845, geo-zh.mjs 807, ebaugesuche-zh.mjs 529. Quelle: wc -l im Worktree

05.2.3 Automatisierte Tests / CI-Pipelines: 0 / 0. Quelle: find nach *.test.*, *.spec.*, test_*.py; kein .github-Verzeichnis

05.2.4 Lockfile im Git: nein, package-lock.json ist gitignoriert; 4 Abhängigkeiten mit ^-Bereich, 115 Pakete in node_modules. Quelle: .gitignore:22, package.json, ~/Developer/jans-ai-hub/node_modules

05.2.5 Connectoren mit Retry/Backoff/429-Behandlung: 0 von 19. Quelle: grep -ciE 'retry|429|Retry-After|backoff'

05.2.6 Connectoren mit fetch() ohne jeden Timeout-Bezug: 11 von 17 mit fetch-Aufrufen. Quelle: grep fetch(gegen AbortSignal|AbortController|timeout

05.2.7 Erfüllung der eigenen --hilfe-Konvention: 7 von 19 (icloud-mail.py zusätzlich über argparse). Quelle: grep --hilfe|--help|willHilfe; Konvention in connectors/README.md:13-14

05.2.8 MCP-Server im Projekt: 2 (1 lokal stdio, M365-CLI-Durchreicher v0.1.17 mit 3 generischen Tools; 1 remote HTTP Figma); fachlich typisierte MCP-Tools: 0. Quelle: .mcp.json; node_modules/@pnp/cli-microsoft365-mcp-server/package.json und dist/index.js

05.2.9 Auth-Muster / Secret-Ablageorte: mindestens 7 Muster (Zertifikat app-only, OIDC-Refresh, PAT, HTTP Basic, Form-Login mit Cookie, Playwright mit Mobile ID, App-Passwort IMAP, OAuth Client Credentials, API-Key) an 5 Ablageorten. Quelle: Kopfkommentare der Connectoren, scripts/wege-doctor.sh:226-262, set-google-key.sh

05.2.10 Längster belegter stiller Ausfall: bexio 36 Tage (12.08. bis 17.09.2026); eBaugesucheZH HTTP 401 seit Ende Juli 2026 (über 7 Wochen, Session-Datei zuletzt 02.07.2026); M365-MCP 12 Tage (13.07. bis 25.07.2026). Quelle: rules/betrieb-chronik.md:34-44; connectors/WEGE.md:548-560; scripts/m365-mcp-server.sh:27-38

05.2.11 Manuelle Re-Login-Zyklen: eBaugesuche rund 10 Tage (Handy nötig); bexio-PAT 60 Tage; bexio-OIDC-Offline-Session 1 Jahr; M365-Zertifikat 2 Jahre (bis 23.03.2028). Quelle: connectors/README.md:109-112; connectors/bexio-auth.mjs:24-26; connectors/WEGE.md Matrixzeile Buchhaltung; openssl x509 -enddate auf öffentliches Zertifikat

05.2.12 Läufe des Wege-Doctors: 6 je Station seit 09.08.2026; letzter Lauf Mac Mini 12.08.2026, MacBook 11.09.2026 (nicht geplant). Quelle: /Volumes/daten/jans-ai-hub/logbuch/wege/doctor-*.log

05.2.13 Getrackte Dateien mit M365-App-ID / Tenant-ID: 43 / 9. Quelle: git grep -l im Worktree (Werte selbst nicht übernommen)

05.2.14 Umfang Wege-Register: 94 kB, 1'319 Zeilen, 21 datierte Nachträge in sechs Wochen. Quelle: wc, grep '^## Nachtrag' auf connectors/WEGE.md

05.2.15 Projekt-Permissions: Bash(*), WebFetch(*), 14 MCP-Wildcards erlaubt; deny-Liste leer. Quelle: .claude/settings.json:4, :33

05.2.16 Nie produktiv gewordene Connectoren: Zefix (Zugangsdaten fehlen seit Bau, auch heute auf dieser Station), Versand-Trio (Skill parkiert, Feldnamen unvalidiert). Quelle: Hub-Audit 260812 Z. 337-342; ls ~/zefix.env; connectors/versand/post-label.mjs:7-9

05.3 Offene Fragen

05.3.1 Welche Application-Permissions trägt die M365-App-Registrierung in Entra tatsächlich (hier nur aus connectors/WEGE.md:176-177 abgeleitet, nicht live geprüft), und werden Sign-in-Logs der App ausgewertet bzw. gibt es Conditional Access für Workload-Identitäten?

05.3.2 Ist der private Schlüssel in `~/cli-m365-cert-combined.pem` wirklich unverschlüsselt? Abgeleitet aus dem Erstellbefehl mit `-nodes` und der Dateigrösse; der Inhalt wurde bewusst nicht gelesen.

05.3.3 Ist die AGOV-Migration des eBaugesuche-Kontos vollzogen, und bietet der Kanton Zürich bzw. AGOV einen Maschinenzugang oder eine API für Gesuchsteller?

05.3.4 Gibt es beim Betreiber des DS3-Projektraums eine offizielle API oder WebDAV-Schnittstelle, und erlauben dessen Nutzungsbedingungen den automatisierten Abruf?

05.3.5 Welche claude.ai-Cloud-Connectoren sind auf welcher Station aktiv? In dieser Session sichtbar: Gmail, Google Drive, Google Calendar, Microsoft 365, Figma (doppelt: `.mcp.json` und Cloud), Microsoft Learn. Gmail ist laut CLAUDE.md nicht in Gebrauch, aber per Wildcard inklusive Senden erlaubt: gewollt?

05.3.6 Die Ursache des `CONNECTION_CLOSED` des M365-MCP-Servers in dieser Session ist plausibel (Worktree ohne `.env` und `node_modules`, Wrapper bricht dann ab), aber nicht durch ein Server-Log belegt.

05.3.7 Die Ursache des über vierminütigen Hängers von `m365-graph.mjs` auf der dritten Station ist laut Register ungeklärt (fehlender Timeout ist nur Kandidat).

05.3.8 Welche der rund 40 Scheduled Tasks hängen an welchem Connector? Eine Abhängigkeitskarte existiert nicht und wurde hier nicht erhoben; sie bestimmt die Reihenfolge eines Umbaus.

05.3.9 Wurde der Zefix-Zugang je beantragt, und sollen die Versand-Connectoren (Skill seit 11.07.2026 parkiert) erhalten, archiviert oder gelöscht werden?

05.3.10 Sollen Integrationen künftig zentral laufen (Mac Mini, Container auf der Synology DS918, Linux-Host, Schweizer Cloud) oder weiterhin je Station? Das entscheidet über Secret-Store, Netzwerkzugang via Tailscale und Lizenzbindung der CAD-Worker.

05.3.11 Wer hat Zugriff auf das private GitHub-Repo (2FA, Deploy-Keys, SSH-Key ohne Passphrase laut Hub-Audit), und ist die bestehende Git-History mit Identifikatoren und Kundenprojekt-Kennungen für den Inhaber ein Thema?

05.3.12 Gibt es vertragliche Grundlagen (Auftragsbearbeitung, Datenstandort) für die Bearbeitung von Bauherren- und Drittdaten über Anthropic, Google und die übrigen angebundenen Dienste? Im Repo nicht gefunden.

06 Wissens-Layer

06.0 Ist-Zustand. Pfade relativ zum Hub-Stamm (gelesen im Worktree `~hub/.claude/worktrees/jans-ai-hub-architecture-4e38fe`, kanonisch `/Volumes/daten/jans-ai-hub`). Der Wissens-Layer ist ein reiner Markdown-Dateibaum unter `wissen/` mit 19 Wissensbasen nach einheitlichem Bauplan (`raw`, `wiki`, `outputs`, teils `destillate` und `training`). Er umfasst 4'329 versionierte Dateien (77 % aller Repo-Dateien, 90 MB), darunter 1'656 kompilierte Artikel mit Frontmatter, Quellenfeld und Status (718 established, 520 emerging, 307 speculative). Befüllt wird er fast ausschliesslich durch unbeaufsichtigte Claude-

Läufe (11 Lernprogramme, vierstufige Destillat-Pipeline, Wissens-Chef), die direkt in den kanonischen NAS-Bestand schreiben; ein nativer Committer pusht alle 15 Minuten nach GitHub. Retrieval geschieht über Markdown-Indizes, Read und grep. Einzig die KB architekten-synopsis besitzt einen Vektorindex (853 Vektoren, lokales Modell intfloat/multilingual-e5-base, numpy-Datei, nur auf dem Mac Mini lauffähig). Die Qualitätssicherung besteht aus einem monatlichen LLM-Health-Check und sieben deterministischen Prüfskripts, die nicht in den Regelbetrieb eingebunden sind. Die Fachdisziplin ist bemerkenswert hoch (Quellenpflicht, Refuter- und Abdeckungsrunden, dokumentierte Fehlerkultur). Es fehlen die Eigenschaften einer Plattform: Datenklassifikation und Zugriffsrechte, maschinell erzwungene Sperrlisten, stabile Provenienz, Schema-Validierung, Transaktionsschutz beim Schreiben, unabhängige Evaluations-Sets und eine Suche, die mit dem Bestand mitwächst. Index- und Journaldateien haben Grössen bis über 1 MB erreicht, die ein Sprachmodell nicht mehr ganz lesen kann.

06.1 Schwachstellen

06.1.1 Personendaten und Mandatsinhalte im versionierten, extern gepushten Repo, faktisch nicht löschar (kritisch, Datenschutz). Der Wissens-Layer enthält Mail-Korpora, eine kundenbezogene KB und Kontaktangaben. Alles wird innert 15 Minuten versioniert und nach GitHub gepusht. Ein Vorfall mit Bewerberdaten ist dokumentiert; die Git-Historie wurde bewusst nicht bereinigt. Das Meta-Schema kennt weder Datenklassifikation noch Aufbewahrungs-, Auskunfts- oder Löscharprozess. Für das Schweizer DSG (Verhältnismässigkeit, Löscharkeit, Auftragsbearbeitung im Ausland) ist das die grösste Lücke des Subsystems. Beleg:

wissen/architektur-fachwissen/CLAUDE.md:35-36 (Repo automatisch gepusht, binnen einer Viertelstunde ausserhalb der Station); rules/auto-verbesserungen.md Eintrag 260823 (Inventardatei mit Bewerbername committet und gepusht, Historie nicht umgeschrieben); Messung: 303 eindeutige E-Mail-Adressen in wissen/, 60 Mail-Batch-Dateien (900 kB) in wissen/twin/raw/, Kunden-KB wissen/kunde-<name>/ im selben Repo; wissen/WISSEN-CLAUDE.md enthält keinen Abschnitt zu Datenschutz, Klassifikation oder Zugriff

06.1.2 Sperrliste für heikle Ordner existiert nur als Prosa, nicht in der Pipeline-Konfiguration (hoch, Datenschutz). Die KB erklärt drei Sektionen (Verträge, Bewerbungsmappen, Adressen) als dauerhaft ausgeschlossen. Das maschinell ausgewertete Ausschlussfeld der beiden betroffenen Korpora enthält diese Ordner jedoch nicht; das Inventar-Script filtert ausschliesslich nach dieser Datei. Der Schutz hängt damit davon ab, dass jeder LLM-Lauf die Prosa-Regel liest und befolgt. Beleg: wissen/architektur-fachwissen/CLAUDE.md:24-27 (dauerhaft ausgeschlossen) gegenüber skills/wissens-destillat/tools/korpus.conf:17-18 (Ausschlussfeld nur @eaDir, thumb, .DS_Store, ~\$, #recycle); skills/wissens-destillat/tools/inventar.sh:35,94,137 (Filter nur aus korpus.conf)

06.1.3 Destillate urheberrechtlich geschützter Normen und eines Fachbuchs liegen im GitHub-gespiegelten Repo (hoch, Governance). Die KB-Regel sagt, Destillate bleiben intern auf dem NAS. Tatsächlich sind 464 Normen-Destillate (bis 98 kB je Norm, inklusive aufgenommener Piktogramm- und Tabelleninhalte) sowie Destillate aus 780 Screenshot-Doppelseiten eines Standardwerks Teil des automatisch gepushten Repos und werden über einen Cloud-LLM-Dienst verarbeitet. Ob Lizenz und Eigengebrauchsschranke das tragen, ist nirgends abgeklärt. Beleg: wissen/normen/CLAUDE.md:28-30 (Destillate bleiben intern, NAS); wissen/baurecht/buecher/CLAUDE.md:15-18 (interne Nutzung, keine Weiterverbreitung) und seiten-inventar.md Frontmatter (total_screenshots: 780); Messung wissen/normen/destillate: 464 Dateien, 7.8 MB, grösste 98 kB; Git-Log 19.09.2026 (33 Anhang-Piktogramme aufgenommen)

06.1.4 Status established misst Richtigkeit von Stichproben, nicht Vollständigkeit; kein unabhängiges Evaluations-Set (hoch, Testlücke). Die Qualitätssicherung ist selbstreferenziell: ein

LLM prüft LLM-Destillate. Die KB hat selbst gemessen, dass fast kein established-Destillat einen dokumentierten Prüfstand trägt und dass beide nachträglichen Abdeckungsläufe erhebliche Lücken fanden. Falsche Werte sind trotz zweifacher Verifikation in established-Artikeln belegt und wurden in Fach-Skills weitergereicht. Es gibt keine Gold-Fragen, keine Regressionstests und keine menschliche Fachstichprobe. Beleg: wissen/normen/training/n62-3-pruefstand-kandidaten.md Abschnitt 1 (310 established, davon 2 mit Prüfstand; SIA 118 nach vier Wochen established: 84 Kern- und 82 Nebenlücken); wissen/tools/README.md:24-28 (Grenzwert 25 % zu streng in zweimal verifiziertem established-Destillat, Autarkie 53 % statt 23 %); rules/normen-referenz.md Ziff. 1b (Bedeutungsumkehr wurde in einen Fach-Skill weitergereicht)

06.1.5 Retrieval über Markdown-Index und grep skaliert nicht mehr; Einstiegsdateien übersteigen das Lesbare (hoch, Skalierung). Der vorgeschriebene Einstieg (Register zuerst lesen) führt in Dateien von mehreren hundert Kilobyte. Journale und Fragenlisten sind monolithische Append-only-Dateien bis über 1 MB. Rund 11 % des Layers sind Buchhaltungsdateien. Der Health-Check räumt ein, den Widerspruchs-Audit aus Budgetgründen nicht mehr vollständig fahren zu können. Es gibt weder Volltextindex noch Ranking, Metadatenfilter oder Chunking. Beleg: Messung: wissen/normen/wiki/REGISTER.md 234 kB, wissen/energie/destillate/INDEX.md 312 kB, wissen/energie/wiki/BAUHERREN-FAQ.md 897 kB, wissen/normen/CHANGELOG.md 1.08 MB, wissen/energie/wiki/QUESTIONS.md 797 kB; Summe INDEX/REGISTER/QUESTIONS/CHANGELOG 9.75 MB von 90 MB; wissen/normen/outputs/2026-09-01_health-check.md Abschnitt Methode (Audit A sprengt das Budget, CHANGELOG 12'927 Zeilen nur in Stichproben gelesen); wissen/normen/CLAUDE.md:13-15 (immer zuerst Register)

06.1.6 Vektorsuche ist ein Einzelstück mit sehr grober Granularität (hoch, Eigenbau). Nur 1 von 19 KBs hat semantische Suche. Der Index hält einen einzigen Vektor je Architekt, gebildet aus den ersten 4'000 Zeichen, für eine Bibliothek von 1.9 TB. Kein Chunking, keine Hybrid-Suche, keine Metadatenfilter, kein Reranking, Brute-Force über eine numpy-Datei, vollständiger Neuaufbau je Lauf, lauffähig nur in einem venv auf dem Mac Mini. Die 1'656 Fachartikel der übrigen KBs sind semantisch nicht durchsuchbar. Beleg: wissen/architekten-synopsis/tools/synopsis_embed.py:27-28 (Modell, MAX_CHARS 4000) und :56-63 (Vollaufbau, savez); wissen/architekten-synopsis/CLAUDE.md:3-4 und :74 (nur Mac Mini); Messung: documents.jsonl 853 Zeilen, vectors.npz 2.4 MB

06.1.7 Unbeaufsichtigte Läufe schreiben ohne Transaktionsschutz in den kanonischen Bestand; Datenverluste sind belegt (hoch, Governance). Alle Loops schreiben direkt in die einzige Wahrheit, teils parallel in dieselben Journaldateien. Schutz besteht nur aus Prosa-Regeln, die nach Vorfällen ergänzt wurden. Es gibt keinen Staging-Bereich, kein Diff-Gate (Löschquote, Statuswechsel), keine Schema-Validierung beim Schreiben. Verifizierter Bestand wurde bereits durch Stubs überschrieben und nur zufällig wiederentdeckt; Aufsichtsläufe reparieren regelmässig Syntaxfehler früherer Läufe. Beleg: skills/wissenscheck/SKILL.md:61-65 (477 Zeilen verifizierter Bestand gelöscht, dreieinhalb Stunden unbemerkt, zufällig gefunden) und :72-82 (erfundene Titel, Phantom-Artikel); rules/auto-verbesserungen.md Eintrag 260811 (246 Zeilen eines Append-only-Journals umgeschrieben); wissen/baurecht/CHANGELOG.md Kopfeintrag 2026-09-18 (Frontmatter-Syntax aus Vorlauf repariert)

06.1.8 Provenienz ist ein stationslokaler Dateipfad ohne Dokument-ID, Hash oder Quellversion (mittel, Governance). Destillate verweisen auf die Quelle über den absoluten OneDrive-Pfad im Benutzerverzeichnis einer Station. Wird die Quelle ersetzt, verschoben oder von einer anderen Person geöffnet, bricht der Bezug still. Kein Artikel trägt einen Inhalts-Hash oder eine SharePoint-Objekt-ID; eine Änderungserkennung an den Quellen ist damit unmöglich. Beleg: wissen/normen/destillate/sia-416-1-2007.md Frontmatter quelle: (absoluter /Users/-Pfad in die

OneDrive-Bibliothek); Messung: 0 Dateien mit Hash-/Checksum-Feld, 80 Dateien mit /Users/-Pfad, 707 Dateien mit /Volumes/daten-Pfad

06.1.9 Keine Zugriffstrennung zwischen Bürowissen, Kunden-KB und persönlichem Mail-Korpus (mittel, Sicherheit). Alle KBs liegen in einem Repo und einem NAS-Share. Wer den Hub klonet oder mountet, liest alles, einschliesslich Kundenprofil und Stil-Korpus aus Geschäftsmails. Für Mitarbeitende, externe Partner oder Mandanten gibt es nur alles oder nichts. Das Meta-Schema kennt keine Vertraulichkeitsstufe je KB oder Artikel. Beleg: Verzeichnisbaum wissen/ (kunde-<name>, twin, koordination neben Fach-KBs im selben Git); wissen/WISSEN-CLAUDE.md:94-104 (Neuanlage und Sync ohne Rechtekonzept)

06.1.10 Prüfwerkzeuge sind nicht in den Regelbetrieb eingebunden, ungetestet und an feste Pfade gebunden (mittel, Testlücke). Der monatliche Health-Check ruft keines der sieben Werkzeuge auf und misst Veraltung mit einem Mass, das ein eigenes Werkzeug als ungeeignet belegt hat. Zwei Werkzeuge messen fest die NAS-Kopie statt der Arbeitskopie. Es gibt keine Tests, kein CI, keine gepinnten Abhängigkeiten; 12 von 17 Scripts im Layer tragen harte NAS-Pfade. Beleg: wissen/tools/README.md:264-271 (wissenscheck ruft keines der Werkzeuge auf) und :60-73 (HUB fest auf NAS, Fehlmessung belegt); wissen/tools/wiki-konsistenz.sh:18, wissen/tools/link-frischecheck.sh:41; kein Verzeichnis .github/workflows, keine requirements.txt oder pyproject.toml im Repo; wissen/architekten-synopsis/tools/synopsis_embed.py:23 (harter NAS-Pfad)

06.1.11 Build-Artefakte im Git und unsicheres Laden des Index (mittel, Sicherheit). Der binäre Vektorindex, die Embedding-Eingabedatei und eine kompilierte Python-Datei sind versioniert, obwohl die Ignore-Regel sie ausschliessen will. Der Index wird mit aktivierter Pickle-Deserialisierung aus einem geteilten Netzlaufwerk geladen; eine manipulierte Datei führt beim Suchen Code aus. Beleg: wissen/architekten-synopsis/tools/synopsis_query.py:124 (np.load mit allow_pickle=True); git ls-files: catalog/vectors.npz (2.4 MB), catalog/documents.jsonl (2.6 MB), tools/_pycache_/synopsis_embed.cpython-312.pyc trotz .gitignore:75-76

06.1.12 Schema-Drift und veraltete Einstiegsdokumentation (niedrig, Governance). Das Meta-Schema definiert drei Statuswerte, im Bestand existieren mindestens 15 Schreibweisen (superseded, destilliert, verified, Freitext, gequotete Varianten); ein Vokabular-Entscheid ist seit Wochen offen. Das README des Layers nennt 3 KBs bei 19 vorhandenen. Eine KB führt ein 84 kB grosses Schema-Dokument, in dem Schema und Daten vermischt sind. Ohne maschinenlesbares Schema lässt sich kein verlässlicher Metadatenfilter bauen. Beleg: wissen/WISSEN-CLAUDE.md:47 (emerging, established, speculative) gegenüber Messung (superseded 31, destilliert 17, weitere Freitextwerte); wissen/normen/outputs/2026-09-01_health-check.md Grundzahlen (Vokabular-Entscheid seit Run 52 offen); wissen/README.md:18-24; wissen/twin/CLAUDE.md 83'876 Byte

06.2 Kennzahlen

06.2.1 Wissensbasen (Verzeichnisse unter wissen/, ohne tools): 19 (Aufgabenstellung nennt 22). Quelle: ls wissen/ im Worktree, 19.09.2026

06.2.2 Versionierte Dateien im Wissens-Layer: 4'329 von 5'605 Repo-Dateien (77 %); davon 3'318 Markdown, 842 JSON. Quelle: git ls-files wissen

06.2.3 Grösse Wissens-Layer: 90 MB im Arbeitsbaum; ca. 57 MiB gepackt in der Historie bei 546 MiB Gesamt-Pack. Quelle: du -sk wissen; git count-objects -vH; git cat-file --batch-check nach Top-Level-Ordner

06.2.4 Kompilierte Artikel (wiki + destillate + buecher): 1'656; ohne Frontmatter 1, ohne Quellenfeld 72, ohne Status 52. Quelle: Eigene Auszählung der ersten 30 Zeilen je Datei

06.2.5 Statusverteilung: established 718, emerging 520, speculative 307, superseded 31, übrige nicht-kanonische Werte ca. 28. Quelle: Eigene Auszählung status: im Frontmatter

06.2.6 Normen-Destillate: 464 Dateien, 7.8 MB, Mittel 17 kB, Maximum 98 kB. Quelle: ls -la wissen/normen/destillate

06.2.7 established-Normendestillate mit dokumentiertem Prüfstand: 2 von 310 (27.08.2026); 35 mit Prüfstand-Vermerk, davon 8 mit vollständiger Abdeckungsrunde (01.09.2026). Quelle: wissen/normen/training/n62-3-pruefstand-kandidaten.md; wissen/normen/outputs/2026-09-01_health-check.md

06.2.8 Lücken beim ersten Abdeckungslauf eines established-Destillats: 84 Kern- und 82 Nebenlücken bei null falschen Werten (SIA 118). Quelle: wissen/normen/training/n62-3-pruefstand-kandidaten.md, Anlass

06.2.9 Vektorindex: 853 Vektoren (1 je Architekt, max. 4'000 Zeichen), Modell intfloat/multilingual-e5-base, vectors.npz 2.4 MB; Quelle 853 Ordner, ca. 390'000 Dateien, 1.9 TB. Quelle: wissen/architekten-synobsis/tools/synobsis_embed.py; catalog/documents.jsonl (wc -l); wissen/architekten-synobsis/CLAUDE.md:3-4

06.2.10 KBs mit semantischer Suche: 1 von 19. Quelle: grep nach Embedding-/Vektor-Code im Repo

06.2.11 Journal- und Indexdateien (INDEX, REGISTER, QUESTIONS, CHANGELOG): 9.75 MB gesamt; grösste wissen/normen/CHANGELOG.md 1.08 MB (12'927 Zeilen). Quelle: find + wc -c; Health-Check normen 01.09.2026

06.2.12 Erzeugte Reports in outputs/: 1'254, davon 109 Health-Check-Reports. Quelle: find wissen -path '*/outputs/*'

06.2.13 Lernprogramme und Laufzähler: 11 PROGRAMM.md; energie Run 208, normen Run 88, Wissens-Chef Lauf 62. Quelle: find wissen -name PROGRAMM.md; Kopfeinträge der CHANGELOG.md

06.2.14 Commit-Last durch den Layer: 2'485 von 11'865 Commits berühren wissen/; normen/CHANGELOG.md allein 352 Commits. Quelle: git rev-list --count HEAD -- <pfad>

06.2.15 Eindeutige E-Mail-Adressen im Layer: 303 (davon 103 in twin, kunde-<name>, koordination). Quelle: grep -o E-Mail-Muster, nur gezählt, nicht ausgegeben

06.2.16 Prüfwerkzeuge, Tests, CI: 7 Werkzeuge, 0 Tests, 0 CI-Workflows, 0 gepinnte Abhängigkeiten; 12 von 17 Scripts mit hartem NAS-Pfad. Quelle: wissen/tools/README.md; find nach test_*, .github/workflows, requirements/pyproject; grep /Volumes/daten

06.2.17 Belegter Datenverlust durch unbeaufsichtigten Lauf: 477 Zeilen verifizierter Bestand, 3.5 h unbemerkt; zusätzlich 246 Journalzeilen umgeschrieben. Quelle: skills/wissenscheck/SKILL.md:61-65; rules/auto-verbesserungen.md 260811

06.3 Offene Fragen

06.3.1 Die Aufgabenstellung nennt 22 Wissensbasen, gemessen wurden 19 Verzeichnisse unter wissen/. Zählen skills/* /wissensbasis, docs/baurecht oder references/ dazu?

06.3.2 Welche Lizenzbedingungen gelten für die bezogenen Normen (SIA, VKF, DIN, VSS, Lignum) und das E-Book? Erlauben sie maschinelle Destillate, Speicherung bei einem Drittanbieter und Verarbeitung über einen Cloud-LLM-Dienst? Im Repo nicht belegt.

06.3.3 Datenstandort und Vertragslage für das private GitHub-Repo sowie Datenverwendung unter dem Claude-Max-Abo (Auftragsbearbeitung, Bekanntgabe ins Ausland) wurden nicht geprüft.

06.3.4 Sind die 60 Mail-Batch-Dateien in wissen/twin/raw/ anonymisiert? Die Schreibregel verlangt Anonymisierung nur für das Wiki; der Inhalt von raw/ wurde bewusst nicht gelesen.

06.3.5 Soll die Git-Historie wegen des dokumentierten Bewerberdaten-Commits bereinigt werden (Entscheid des Inhabers ausstehend, technisch heikel bei drei klonenden Stationen)?

06.3.6 Kosten und Tokenverbrauch je Lern-Loop und KB sind im Subsystem nicht messbar; das Lauf-Journal ist bewusst nicht versioniert.

06.3.7 Die tatsächliche Retrieval-Qualität (Trefferquote, Antwortzeit) wurde nie gemessen; es gibt keine Nutzungsstatistik, welche KBs in der Projektarbeit wirklich gezogen werden.

06.3.8 Ist der semantische Index auf dem Mac Mini produktiv in Gebrauch? Letzter Commit von vectors.npz am 01.09.2026; die KB gilt als saturiert.

06.3.9 Gibt es ein vom Git unabhängiges Backup- und Restore-Verfahren für den Wissens-Layer und die Quellkorpora (NAS-Snapshots, externe Sicherung)? Im Subsystem nicht belegt.

06.3.10 Zielbild Ausbau: wie viele Mitarbeitende, welche Rollen, sollen sie schreibend am Wiki arbeiten oder nur abfragen? Bestimmt Rechte- und Plattformdesign.

06.3.11 Der Vokabular-Entscheid für Statuswerte (superseded, destilliert usw.) ist laut Health-Check seit Run 52 offen; wer entscheidet ihn?

07 Harness: Skills, Agenten, Rules

07.0 Ist-Zustand. Der Harness umfasst 51 Skills, 43 Agenten, 13 Commands, 29 Rule-Dateien (24 davon per absolutem @-Import vom NAS in jede Session geladen), 46 gespiegelte Task-Prompts (256 kB) sowie rund 13'400 Zeilen Code in scripts/ und nochmals 13'400 in skills/. Alles liegt als Markdown und Shell auf dem NAS, wird dort direkt editiert und ist ohne Commit sofort auf allen Stationen wirksam. Git sichert nur nachträglich: 4'065 Commits in 30 Tagen, zwei Drittel davon automatische Sammel-Commits ohne Aussage, 0 Tags, keine CI. Das System verändert seinen eigenen Prompt: ein UserPromptSubmit-Hook, der DNA-Compiler des Twin und mehrere Aufsichts-Loops schreiben in importierte Rules. Der Grundkontext ist trotz zweier Diät-Runden von 85'901 B (03.08.2026) auf 151'483 B (19.09.2026, rund 37'900 Tokens) gewachsen, bei einer eigenen Schwelle von 100'000 B. Durchgesetzt wird fast alles per Prompt; technisch greifen fünf Hooks, die Berechtigungen stehen auf Bash(*), Write(*), Edit(*) ohne deny-Liste. Regressionstests oder Evals für Skills, Prompts und Rules fehlen. Einzige Eval ist der tägliche Twin-Fidelity-Review (Scores 23 bis 60 von 100, laut eigenem Vermerk zwischen Läufen nicht vergleichbar). Erhaltenswertes geistiges Eigentum sind die Fach-Skills mit Contract und Prozessketten, der Twin-Harness, die Wissensbasen (823 Wiki-Artikel) und die Mess- und Belegkultur. Betriebs-Klebstoff sind Sync-, Wächter- und Treiber-Scripts, die handgepflegten Kataloge und das als Fliesstext mit eingebetteter Vorfallchronik gewachsene Regelwerk. Messbasis: Git-Spiegel Stand 19.09.2026 10:00, Belegpfade relativ zur Repo-Wurzel /Volumes/daten/jans-ai-hub.

07.1 Schwachstellen

07.1.1 Edit direkt in Produktion, kein Release-Prozess (kritisch, Governance). Skills, Rules, Agenten und CLAUDE.md werden auf dem NAS editiert und wirken ohne Commit sofort auf allen Stationen und in allen unbeaufsichtigten Loops. Es gibt weder dev/staging noch Review, Tags oder definierten Rollback. Git ist nachgelagertes Backup; die Historie ist als Änderungsnachweis kaum lesbar, weil die Mehrheit der Commits automatische Sammel-Commits sind. Worktrees existieren (60 Branches), dürfen laut Rule aber gerade nicht für geteilte Inhalte verwendet werden. Beleg: rules/README.md:16-17 («sofort auf allen Stationen wirksam», «Kein Git-Commit/Push noetig»); rules/sync-kanonische-quelle.md:7; git log 30 Tage: 4'065 Commits, davon 2'734 «nas-selfcommit: N Datei(en)»; git tag: 0; kein .github/, keine pre-commit- oder Lint-Konfiguration

07.1.2 Keine Regressionstests oder Evals für Skills, Prompts und Rules (kritisch, Testluecke). Eine Änderung an einer Rule oder SKILL.md kann das Verhalten von 51 Skills und rund 40 Loops verschieben, ohne dass etwas es bemerkt. Es existiert kein Golden-Set, kein Trigger-Test (welche Phrase löst welchen Skill aus), kein Format-Test gegen die Goldstandards. Der einzige Selbsttest prüft zwei Betriebs-Scripts. Der Twin-Fidelity-Review ist die einzige Eval, wechselt aber Methode und Vorlagen zwischen Läufen und ist damit keine Regressionsmessung. Beleg: git ls-files: einzige Testdatei scripts/schutzmechanik-selbsttest.sh (prüft lauf-gate und speicher-waechter); package.json ohne scripts; scripts/contract-check.sh prüft nur Zeichenketten per grep; wissen/twin/outputs/2026-09-16_fidelity.md («Nicht vergleichbar mit den 43 vom 15.09.»); Scores 10.09. bis 19.09.: 26, 38, 43, 58, 52, 56, 60

07.1.3 Berechtigungsmodell ohne Least Privilege (kritisch, Sicherheit). Die versionierte Projekt-Konfiguration erlaubt pauschal Shell, Schreiben, Editieren und Webzugriff sowie ganze MCP-Server (M365, Mail, Computer-Use, Mac-Steuerung) und führt keine deny-Liste. Der Schutz ruht auf Prompt-Regeln, dem Auto-Mode-Klassifikator des Herstellers und einem Guard für genau einen Kanal. In Kombination mit unbeaufsichtigten Loops und selbstverändertem Prompt ist das die grösste strukturelle Sicherheitslücke des Harness. Es gibt keine getrennten Profile für interaktiv, unbeaufsichtigt und Cloud. Beleg: .claude/settings.json: permissions.allow enthält Bash(*), Read(*), Edit(*), Write(*), WebFetch(*), mcp_Control_your_Mac_*, mcp_computer-use_*; permissions.deny: 0 Einträge; docs/konzepte/260812-Hub-Audit/260812-Hub-Audit.md Abschnitt 2.3 Lücke 1 (Guard schützt nur Sync-Tasks)

07.1.4 Selbstmodifikation des Prompts ohne Review-Gate (hoch, Governance). Ein Hook weist Claude bei Stichwörtern wie «immer», «nie», «korrektur» an, Rule-Einträge zu schreiben oder neue Rules samt @-Import anzulegen. Zusätzlich setzen Aufsichts-Loops (Wissens-Chef, Synergie-Lauf, Fidelity-Review, Radar) Vermerke in importierte Rules. Weil sie den Wortlaut nicht ändern dürfen, schichten sie Warnabsätze übereinander; die Rule enthält dann Regel, Gegenbeleg und Gegen-Gegenbeleg nebeneinander, mit «gilt bis zum Entscheid» als Dauerzustand. Das Modell muss in jeder Session widersprüchliche Schichten selbst auflösen. Beleg: scripts/verbesserung-capture.sh:17-33; rules/anrede-kontakte.md: 11 Warnvermerke, 8 Stellen «bis zum Entscheid» bzw. «Entscheid ausstehend», Vermerke aus mindestens 14 verschiedenen automatischen Läufen in importierten Rules (grep nach Wissens-Chef, Synergie-Lauf, Fidelity-Review, Vollgas-Radar, Buch-Run); 31 Commits auf importierte Rules in 30 Tagen

07.1.5 Grundkontext wächst ungebremsst, die Messung warnt nur (hoch, Kosten). Zwei Diät-Runden haben den Grundkontext auf 85.9 kB gesenkt; sechs Wochen später liegt er bei 151.5 kB und damit 51 % über der eigenen Schwelle. Haupttreiber sind genau die selbstgeschriebenen Dateien. Jede der rund 40 Tasks und jeder Subagent lädt diese Last. Das Instrument existiert, hat aber keine Durchsetzung; auch der monatliche Doctor-Lauf ist überfällig. Beleg: Eigene Messung nach Methode

scripts/grundkontext-messung.sh: 151'483 B bei GK_MAX=100000 (Zeile 30); rules/auto-verbesserungen.md 43'669 B bei AV_MAX=22000 (Zeile 31), Verlauf aus Git: 13'303 B (20.07.), 21'551 B (13.08.), 43'669 B (19.09.); rules/anrede-kontakte.md 1'511 B auf 18'970 B; logbuch/doctor-letzterlauf.txt: 2026-07-30 bei Schwelle 35 Tage; docs/konzepte/260803-Grundkontext-Diaet-Runde-2 (Ergebnis 85'901 B)

07.1.6 Personendaten Dritter im Grundkontext und im Git-Backup (hoch, Datenschutz). Ein Kontaktregister mit Namen, Arbeitgebern, Rollen und Beziehungsnotizen wird als importierte Rule in jede Session und jeden Subagenten geladen und damit laufend an den Modellanbieter übermittelt sowie alle 15 Minuten nach GitHub gesichert. Im Repo liegen zudem Rohauszüge authentischer Mails, Gesprächsdestillate und eine kundenbezogene KB. Ein belegter Vorfall zeigt, dass Bewerberdaten gepusht und die Historie bewusst nicht bereinigt wurde. Datenminimierung, Zweckbindung und Lösbarkeit nach DSGVO sind so nicht nachweisbar. Beleg: CLAUDE.md @-Import rules/anrede-kontakte.md (18'970 B); git ls-files wissen/twin/raw: 60 Dateien, davon 56 Mail-Batches; logbuch/konversationen/ und wissen/kunde-* versioniert; rules/auto-verbesserungen.md Eintrag 260823 (Inventardatei mit Bewerbernamen gepusht, Historie nicht umgeschrieben)

07.1.7 Leitplanken hängen an einem SMB-Mount (hoch, SPOF). Alle 24 importierten Rules und die Symlinks auf Skills, Agenten und Commands zeigen mit absoluten Pfaden auf das NAS. Bricht der Mount ab, arbeitet eine Session ohne Stil-, Quellen- und Sicherheitsregeln weiter; der SessionStart-Hook gibt nur eine Warnzeile aus. Mount-Abrisse sind gemessen häufig. Beleg: rules/README.md:57-58 («Bei Offline-Arbeit fallen die Rules weg»); CLAUDE.md: 24 Zeilen @/Volumes/daten/...; docs/konzepte/260911-Systemanalyse-Hub-Architektur-NAS-Git Abschnitt 00.2 (bis 275 Mount-Abrisse an einem Tag, sieben Ausfalltage); .claude/settings.json SessionStart-Hook (nur echo)

07.1.8 Durchsetzung fast ausschliesslich per Prompt (hoch, Governance). Pflichten wie Korrektur vor jeder Ausgabe, Twin-Gate, Identifikatoren verifizieren, BKP- und Normen-Nachschlag oder Wissens-Rücklauf sind reine Textanweisungen. Technisch greifen fünf Hooks; vier davon schlucken jeden Fehler und laufen aus dem SSD-Klon, nicht aus der kanonischen Quelle. Das eigene Konzeptpapier stellt fest, dass es keinen harten Korrektur-Gate gibt. Auch Konfigurationsfelder (model, enabled) sind nachweislich wirkungslos. Beleg: .claude/settings.json: 5 Hooks, 4 davon mit «2>/dev/null || true»; docs/konzepte/260803-Grundkontext-Diaet-Runde-2:67-73; rules/modellwahl-routine.md Ziffer 4

07.1.9 Drift zwischen Contract, Registern, Katalogen und Bestand (mittel, Governance). Pflichtschemata werden nicht validiert und laufen auseinander: ein Viertel der Skills erfüllt den Contract nicht, Agenten fehlen im Rollen-Register vollständig, der Katalog in CLAUDE.md ist Handarbeit. Agenten haben mehrheitlich keine Werkzeug-Einschränkung. Die Prüfung ist grep-basiert und läuft nur auf Zuruf. Beleg: scripts/contract-check.sh gegen Git-Spiegel: 12 von 51 Skills unvollständig (4 ohne Block, 8 mit fehlenden Feldern), rc=1; logbuch/rollen/rollen-map.tsv: 0 Zeilen vom Typ agent trotz rules/rollen-taxonomie.md:13 («gilt für Skills, Agenten und Loops»), Skill energie ohne Zeile; agents/*.md: 14 von 43 mit tools, 4 mit model; CLAUDE.md-Tabellen: Skill twin und 10 Agenten fehlen

07.1.10 Regelwerk und Steuerlogik als Fliesstext mit eingebetteter Chronik (mittel, Eigenbau). Regeln, Belege, Korrekturen und Rücknahmen stehen vermischt in Prosa. Daraus entstehen Widersprüche zwischen Dokumenten und nicht testbare Steuerlogik in Prompts. Eine 260 kB grosse Chronik gilt als Pflichtlektüre vor jeder Infrastrukturarbeit, was kein Kontextfenster sinnvoll trägt. Beleg: scripts/nachtschicht-run.sh:126 (eine Zeile mit 5'746 Zeichen, am 12.08. laut Hub-Audit R14 rund 4'000); rules/README.md:36 (modellwahl «Haiku für Routine-Läufe, spart 6 %») gegen

rules/modellwahl-routine.md Ziffer 4 (Feld wirkt nicht); scripts/verbesserung-capture.sh:31 verweist auf CLAUDE.md im SSD-Klon gegen rules/sync-kanonische-quelle.md:7; rules/betrieb-chronik.md 259'913 B mit 95 Abschnitten; rules/auto-verbesserungen.md Abschnitt «Betrieb» (Regelsatz in nicht importierter Chronik)

07.1.11 Auf eine Person und eine Maschine-Topologie fest verdrahtet (hoch, Skalierung). Pfade, Benutzername, Absender, Stationsnamen und Freigaben sind im Harness hart codiert; ein zweiter Mitarbeitender, ein Mandant oder eine andere Laufzeit ist nicht vorgesehen, obwohl das Geschäftsmodell ein Backend für Dritte nennt. Alle Entscheide laufen über den Inhaber, der Rückstau ist sichtbar und wird von den Loops schneller erzeugt als abgebaut. Beleg: grep: /Volumes/daten in 58 Skill-, 25 Agenten-, 11 Rule-, 7 Command- und 67 Script-Dateien (Systemanalyse 260911 Abschnitt 01.2: 696 Zeilen in 229 Dateien); /Users/raphaeljans in 31 Dateien; docs/konzepte/260715-Geschäftsmodell-KI-Architekturbuero Abschnitt GM4; logbuch/fristen.md: 11 Zeilen mit Status «Entscheid Raphael», wissen/*/wiki/QUESTIONS.md total 3.0 MB; docs/konzepte/260729-Rollen-Taxonomie Abschnitt 7 (sechs offene Entscheide seit 29.07.)

07.1.12 Meta-Schicht bindet mehr Aufwand als die Facharbeit (mittel, Betriebsaufwand). Die grösste Rolle im Bestand ist Betrieb und Aufsicht. Läufe und Kosten entfallen überwiegend auf Lern- und Aufsichts-Loops, ein Teil davon belegt ertraglos. Fast die Hälfte der Skills wurde im Messfenster nie aufgerufen. Das System baut Wächter für Wächter, statt Standardmittel zu verwenden. Beleg: logbuch/rollen/260911-rollen-bilanz.md Abschnitt 1 und 2: 2'439 Läufe in 30 Tagen, 1'074 Fehlstarts, Kosten USD 2'095 (Selbstauskunft SDK), maintainer 35 von 110 Bausteinen; logbuch/rollen/260911-nutzungs-radar.md: 21 von 49 Skills nie aufgerufen; rules/auto-verbesserungen.md Eintrag 260830 (über 600 ertraglose Läufe in einer Nacht)

07.2 Kennzahlen

07.2.1 Skills (Ordner mit SKILL.md): 51 (49 aktiv plus 2 parkiert), 554 kB SKILL.md-Text. Quelle: ls skills/*/SKILL.md, wc -c; logbuch/rollen/260911-nutzungs-radar.md

07.2.2 Agenten: 43 (plus README), davon 14 mit tools-Einschränkung, 4 mit model-Angabe. Quelle: grep Frontmatter agents/*.md

07.2.3 Rules: 29 Dateien, 523 kB; 24 per @-Import in jeder Session. Quelle: wc -c rules/*.md; grep '^@' CLAUDE.md

07.2.4 Grundkontext je Session: 151'483 B, rund 37'900 Tokens (Schwelle 100'000 B); 03.08.2026: 85'901 B; 19.07.2026: rund 163 kB. Quelle: Eigene Messung nach scripts/grundkontext-messung.sh am Git-Spiegel 19.09.2026; docs/konzepte/260803-..., 260719-...

07.2.5 Grösste importierte Rules: auto-verbesserungen.md 43'669 B (Schwelle 22'000; 20.07.: 13'303 B), anrede-kontakte.md 18'970 B (20.07.: 1'511 B), interaktive-eingriffe.md 8'216 B. Quelle: wc -c; git cat-file -s je Stichtag

07.2.6 Contract-Erfüllung Skills: 39 von 51 vollständig, 4 ohne Block, 8 unvollständig. Quelle: HUB=<Git-Spiegel> bash scripts/contract-check.sh, rc=1

07.2.7 Rollen-Register: 110 Bausteine (35 maintainer, 27 prototyper, 22 builder, 17 sweeper, 9 grower); 0 Agenten erfasst. Quelle: logbuch/rollen/rollen-map.tsv; logbuch/rollen/260911-rollen-bilanz.md

07.2.8 Skill-Nutzung: 28 von 49 aufgerufen, 21 nie (12 erwartet, 9 zu klären). Quelle: logbuch/rollen/260911-nutzungs-radar.md

07.2.9 Automatische Läufe 30 Tage: 2'439 Läufe, 1'365 geliefert (95.8 h), 1'074 Fehlstarts, USD 2'095 laut SDK-Selbstauskunft. Quelle: logbuch/rollen/260911-rollen-bilanz.md Abschnitt 1

07.2.10 Commits 30 Tage: 4'065, davon 2'734 nas-selfcommit und 376 sync; 116 auf rules/, 31 auf importierte Rules, 32 auf SKILL.md, 5 auf CLAUDE.md; 0 Tags; 60 Branches. Quelle: git log --since='30 days ago'

07.2.11 Tests und CI: 1 Selbsttest-Script (2 Schutzmechaniken), 0 Skill-/Prompt-Evals, keine CI, kein Lint, kein pre-commit. Quelle: git ls-files, ls -a Repo-Wurzel

07.2.12 Hooks: 5 (2 SessionStart, 1 UserPromptSubmit, 1 PreToolUse, 1 Stop); 4 mit Fehlerunterdrückung. Quelle: .claude/settings.json

07.2.13 Berechtigungen: 28 allow-Einträge inkl. Bash(*), Write(*), Edit(*), WebFetch(*); 0 deny. Quelle: .claude/settings.json

07.2.14 Twin-Fidelity (einzige Eval): 105 Reports; Gesamtscore 09.09. bis 19.09.2026 zwischen 25 und 60 von 100, schwächste Facetten Haltung und Beziehungsregister je 45. Quelle: wissen/twin/outputs/*_fidelity.md

07.2.15 Wissensbestand: 20 KB-Ordner, 823 Wiki-Artikel (241 established, 193 speculative), QUESTIONS.md total 3.0 MB. Quelle: find wissen/*/wiki; grep status

07.2.16 Code-Umfang: scripts/ 13'438 Zeilen (84 Shell-Scripts, 100 Dateien), skills/ 13'373 Zeilen in 59 Code-Dateien, connectors/ 3'804 Zeilen. Quelle: git ls-files | xargs wc -l

07.2.17 Task-Prompts: 46 im Spiegel, 255'622 B; längste Steuerzeile im Code 5'746 Zeichen. Quelle: templates/scheduled-tasks/; scripts/nachtschicht-run.sh:126

07.2.18 Fest verdrahtete Pfade: /Volumes/daten in 229 Dateien (696 Zeilen); /Users/raphaeljans in 31 Dateien. Quelle: docs/konzepte/260911-Systemanalyse Abschnitt 01.2; eigener grep

07.3 Offene Fragen

07.3.1 Gemessen wurde der Git-Spiegel (HEAD vom 19.09.2026 10:00). Ob der Live-Stand auf dem NAS davon abweicht und ob der heartbeat den Schwellenriss des Grundkontexts aktuell meldet, wurde nicht geprüft.

07.3.2 Inhalt von .claude/settings.local.json wurde bewusst nicht gelesen; zusätzliche allow- oder deny-Regeln je Station sind daher unbekannt.

07.3.3 Schutzwirkung des Auto-Mode-Klassifikators von Claude Code ist von aussen nicht messbar; unklar ist, in welchen Laufarten (Loops, Dispatch, Cloud) er überhaupt aktiv ist.

07.3.4 Existieren Auftragsbearbeitungsverträge, ein Bearbeitungsverzeichnis und eine geprüfte Einstellung zur Datennutzung bei Modell-, Git- und Cloud-Anbietern? Im Repo nicht belegt.

07.3.5 Zugriffsschutz des GitHub-Repos (Branch-Protection, Zwei-Faktor, Deploy-Keys, weitere Berechtigte) ist aus dem Repo nicht prüfbar.

07.3.6 Die Kostenangabe USD 2'095 in 30 Tagen ist eine Selbstauskunft des SDK innerhalb von Abonnementen; der reale Geldabfluss (Max-, Team-Abo, Extra-Usage) ist offen.

07.3.7 Gibt es je Fach-Skill eine fachliche Abnahme durch den Inhaber (Freigabestatus, Version)? Ein solches Feld ist weder im Contract noch im Rollen-Register vorhanden.

07.3.8 Welche Laufzeit ist Zielplattform: bleibt Claude Code die einzige Engine, oder soll das Kern-IP portabel (Agent SDK, API, andere Modelle) werden? Das bestimmt Paketierung und Eval-Architektur.

07.3.9 Der Entscheid zwischen den Umbauvarianten V2 (Git als einzige Quelle) und V0+ aus der Systemanalyse 260911 ist offen und bestimmt, worauf ein Release-Prozess aufsetzt.

07.3.10 Wie viele der 193 als speculative markierten Wiki-Artikel werden in Fach-Skills faktisch zitiert? Eine Durchsetzung des Statusfelds ist nicht belegt.

07.3.11 Ob die Meta-Learning-Schleife (masterclass/harness-review.md), auf die der Skill-Contract verweist, tatsächlich läuft: laut Nutzungs-Radar wurde der Skill nie geladen.

07.3.12 Anzahl und Rollen künftiger Mitarbeitender sowie Art der Leistungen an Dritte (Geschäftsmodell GM4) sind nicht spezifiziert; davon hängt der Umfang der Mandantenfähigkeit ab.

08 Geschäftsmodell und Produkte

08.0 Ist-Zustand. Das Geschäftsmodell ist konzeptionell klar gefasst. Ein Konzeptpapier vom 15.07.2026 definiert sechs Bausteine (GM1 produktisierte Studien, GM2 typologischer KI-Entwurfsprozess, GM3 Wettbewerbs-Fabrik, GM4 White-Label-Backend für Partnerbüros, GM5 Wissens-Kompoundierung, GM6 interaktive Deliverables). Der Service-Katalog führt sechs Services (S1 bis S6) mit Startbefehl, Pipeline, Kunden-Input und Durchlaufzeit, aber ohne Preise. Technisch existieren die Produkte als Skill-Pipelines für genau einen Operator: alles läuft über Abo, Identität und Zugangsdaten des Inhabers. Kundenseitig gibt es zwei Artefakte: das Machbarkeits-Studio (selbsttragendes HTML, per Script auf ein US-CDN publiziert, Schutz allein durch Zufallslink, ein Studio live seit 27.06.2026) und ein Bewertungs-Webtool (eine HTML-Datei, Marktdaten hart codiert, keine Lead-Erfassung). Das Cockpit ist ein internes Dashboard ohne Anmeldung. Marketing und Telesales sind Textgeneratoren ohne CRM, Lead-Datenbank oder Erfolgsmessung. Der Synergie-Orchestrator läuft täglich (39 Läufe, 92 Befunde), die geschäftsrelevanten Befunde (Demo-Studio, B2B-Angebot, Stammdaten) stehen seit 15.07.2026 auf «offen». Die AG-Gründung ist in Arbeit und schafft einen zweiten Rechtsträger (Einzelfirma für Consulting, AG für Ausführung und GU-Mandate). Das Systemaudit vom 10.06.2026 hält für Mitarbeiter-Stationen fest: voller Zugriff für alle, kein Berechtigungsmodell, zentrales Claude-Konto. Für Mandamentrennung, Kundenportal, Abrechnung, SLA und Datenschutz nach DSG existiert kein Konzept.

08.1 Schwachstellen

08.1.1 Kein Identitäts-, Rollen- und Mandantenmodell (kritisch, Skalierung). Das System kennt genau einen Benutzer. Für Mitarbeiter-Stationen ist ausdrücklich voller Zugriff ohne Berechtigungsmodell und ein zentrales Claude-Konto entschieden. Neue Stationen erhalten das tenant-weite M365-App-Zertifikat und den bexio-Zugang als Dateikopie. Mitarbeitende, Partnerbüros und Bauherren sind damit nicht abbildbar, Handlungen nicht personenbezogen nachvollziehbar. Beleg: docs/systemaudit-260610.md Z. 57-61; rules/betrieb-chronik.md Abschnitt «260911 Station-03» (Zertifikat per scp, .env und bexio-Zugang kopiert); scripts/neue-station.sh Z. 183-188

08.1.2 Kein Datenschutz-Konzept nach DSG bei gleichzeitig breiter Personendatenhaltung im Git-Repo (kritisch, Datenschutz). Es gibt kein Bearbeitungsverzeichnis, keine Übersicht der Auftragsbearbeiter (KI-Anbieter, GitHub, CDN, Microsoft), keine Datenklassifikation und keine Löschfristen. Gleichzeitig liegen Mail-Korpora, eine Kunden-KB, Anbieter-Stammdaten, das Logbuch und die AG-Spec samt Bankverbindung im Repo, das automatisch auf GitHub gepusht wird. Beleg: Volltextsuche Datenschutz/DSG/Auftragsbearbeit in docs/, rules/, services/, skills/hub-chef: nur Fachinhalte und macOS-Hinweise (z.B. scripts/install-synctask-runner.sh Z. 84); git ls-files: 56 Dateien wissen/twin/raw/*mailbatch*, 18 Dateien wissen/kunde-*, 45 Dateien

skills/ausschreibung/anbieter/, 163 Dateien logbuch/; wissen/spec/outputs/2026-06-21_ag-gruendung_spec.md CHANGELOG-Einträge 27./28.08.2026

08.1.3 Kunden-Studios auf US-CDN, Schutz nur durch Zufallslink (hoch, Datenschutz). Studios mit Parzelle, Landwert, Marge und Marktannahmen des Bauherrn werden ohne Authentisierung publiziert. Der Slug trägt 4 Byte Zufall, mit --name ist ein fester, erratbarer Slug möglich. Es gibt weder Ablaufdatum noch Widerruf noch Zugriffsprotokoll, einzig ein noindex-Header. Hosting-Standort und Vertragsgrundlage sind nicht geregelt. Beleg: scripts/deploy-studio.sh Z. 3-16, Z. 35-42, Z. 46

08.1.4 GM4 White-Label für Partnerbüros ohne Mandantentrennung (hoch, Governance). Das Konzept sieht vor, dass Fremdaufträge die eigene Kennwerte-DB weiter füttern. Technisch würden Daten Dritter im selben Repo, denselben KBs und demselben Logbuch landen. Vertragsgrundlage, Anonymisierung und Trennung fehlen, das B2B-Angebot ist seit dem Erstlauf nicht ausformuliert. Beleg: docs/konzepte/260715-Geschäftsmodell-KI-Architekturbuero/...md Z. 64-72; services/KATALOG.md Z. 46; wissen/koordination/SYNERGIE-REGISTER.md SYN-11 Status «offen» seit 15.07.2026

08.1.5 Zwei Rechtsträger, aber ein einziger Hub-Kontext (hoch, Governance). Die AG soll unabhängig von der Einzelfirma Ausführungs- und GU-Mandate tragen, bexio führt sie als eigenen Mandanten. Der Hub kennt jedoch nur einen Absender, eine Stimme, ein Logbuch, ein Fristen-Register und einen Katalog. Die Haftungstrennung, die das Ziel der Gründung ist, wird auf Systemebene nicht nachvollzogen. Beleg: wissen/spec/outputs/2026-06-21_ag-gruendung_spec.md Abschnitte «Ziel» und «Schlüsselentscheide»; wissen/spec/outputs/2026-07-02_buchhaltungssystem_spec.md Z. 17, 43, 71; rules/jans-absenderadresse.md (eine Firma fest verdrahtet)

08.1.6 Service-Produktion hängt an Abo-Kontingent, kein SLA möglich (hoch, SPOF). Kommerzielle Deliverables und rund 40 Lern-Loops teilen sich ein Abo mit Wochenlimit. Geschäftsentwicklung wurde nachweislich auf den Limit-Reset verschoben. Es gibt keine Auftragsqueue, keine Kapazitätsplanung, keine Kosten je Auftrag und mit dem Mac Mini einen einzigen Render- und Lizenzknoten für die Präsentationsqualität. Beleg: docs/konzepte/260715-...md Z. 52 und Z. 118; docs/konzepte/260813-Kontingent-Takt/260813-Kontingent-Takt.md Abschnitt «Ausgangslage» und 2.1 (nichts umgestellt); skills/machbarkeit-studio/SKILL.md Z. 172-178

08.1.7 Rechenkern doppelt geführt, keine automatisierten Tests, kein CI (hoch, Testluecke). Der Python-Rechenkern wird in der JS-Engine des Studios von Hand gespiegelt. Korrektheit und Deckungsgleichheit wurden einmalig manuell im Nacht-Build geprüft. Repo-weit existieren weder Testverzeichnisse noch eine CI-Pipeline. Bauherren treffen auf diesen Zahlen Entscheide in Millionenhöhe. Im Bewertungstool sind die Marktkennwerte hart codiert, 5 von 10 Gemeinden tragen den Stand 12/2023. Beleg: skills/machbarkeit-studio/SKILL.md Z. 117-119; skills/machbarkeit-studio/tools/build_studio.py Z. 8; skills/machbarkeit-studio/SPEC.md K1/K2; find nach tests/, *.test.*, test_*.py ohne Treffer; kein .github/; webtools/bewertungstool/index.html Z. 226-237

08.1.8 Studio-Deploy stationsgebunden und nicht reproduzierbar, Doku widerspricht Code (hoch, SPOF). Das Deploy-Verzeichnis mit allen bisherigen Studios liegt nur lokal auf einer Station, ausserhalb von Git und Backup. Weil jeweils der ganze Ordner deployt wird, würde ein Deploy von einer anderen Station bestehende Kunden-Links entfernen. wrangler wird ungepinnt als @latest nachgeladen. Der Katalog nennt Hostpoint als Hoster, das Script deployt auf Cloudflare Pages. Beleg: scripts/deploy-studio.sh Z. 12-15, Z. 25, Z. 50; services/KATALOG.md Z. 16; Dateiliste ~/jans-studio-pages (3 Dateien, Stand 27.06.2026)

08.1.9 Kein Preis-, Offert- und Abrechnungsmodell für die Services (mittel, Governance). Der Katalog enthält keine Preise, die Standard-Offerte je Service ist seit 15.07.2026 als nächster Schritt notiert, der Honorar-Skill kennt den Katalog nicht. Für den wiederkehrenden Posten «gehostete Kunden-Studios» (GM6) fehlt jede Abrechnungsmechanik und Nachkalkulation. Beleg: services/KATALOG.md Z. 6; docs/konzepte/260715-...md Z. 33-34 und Z. 88-90; grep «Katalog/Fixpreis/S1» in skills/honorarberechnung-sia102/SKILL.md ohne fachlichen Treffer

08.1.10 Akquise ohne Datenbasis und ohne Erfolgsmessung (mittel, Eigenbau). Marketing und Telesales erzeugen Texte, es gibt keine Lead- oder CRM-Struktur und keine Kennzahlen. Telesales ist im Prozessgraph eine Insel. Das Bewertungstool erfasst keine Leads, das Demo-Studio für die Website ist offen. Kontaktdaten werden dreifach gepflegt. Beleg: skills/telesales/SKILL.md Z. 14-15 (Vorgelagert/Nachgelagert leer); webtools/bewertungstool/README.md Z. 46; SYNERGIE-REGISTER.md SYN-09 und SYN-05 Status «offen»

08.1.11 Aufsicht blickt nach innen, Geschäftsergebnis wird nicht gemessen (mittel, Betriebsaufwand). Von 92 Registerbefunden betreffen 63 interne Ketten-Lücken, nur 6 Services oder Entwurfsprozesse. Trotz 39 täglicher Läufe stehen die drei geschäftsrelevanten Befunde seit 66 Tagen offen. Kennzahlen zu Anfragen, Offerten, Durchlaufzeit je Service oder Marge existieren nicht. Das System optimiert vorwiegend sich selbst. Beleg: wissen/koordination/SYNERGIE-REGISTER.md Typ- und Statusspalte (Auszahlung 19.09.2026); wissen/koordination/outputs/ (39 synergie-lauf-Dateien); keine Treffer für Umsatz/Auftragseingang in docs/, services/, skills/hub-chef/

08.1.12 Cockpit und Register für Mehrbenutzerbetrieb ungeeignet (mittel, Sicherheit). Der statische Cockpit-Server bindet auf alle Interfaces, spricht HTTP ohne Anmeldung und zeigt das gesamte Fristen-Register mit Geld-, Kunden- und Behördenpunkten jedem Gerät im LAN und Tailnet. Als Datenbank dient eine einzelne Markdown-Datei von rund 1.1 MB, Zeilen werden über einen Text-Hash adressiert. Mit Mitarbeitenden braucht es Anmeldung, Projektfilter und strukturierte Datenhaltung. Beleg: webtools/cockpit/serve-cockpit.mjs Z. 34; webtools/cockpit/README.md Z. 41 und Z. 107-118; Ausbaustufen Z. 139-148

08.2 Kennzahlen

08.2.1 Katalog-Services: 6 (S1 bis S6), davon 0 mit hinterlegtem Preis. Quelle: services/KATALOG.md

08.2.2 Geschäftsmodell-Bausteine: 6 (GM1 bis GM6), Dokumentstand 15.07.2026, seither nicht nachgeführt. Quelle: docs/konzepte/260715-Geschäftsmodell-KI-Architekturbuero/...md, git log

08.2.3 Skills / Agenten / Wissensbasen im Worktree: 51 / 44 / 20. Quelle: ls skills/, agents/*.md, wissen/ am 19.09.2026

08.2.4 Synergie-Läufe und Registerbefunde: 39 Läufe, 92 Befunde; Typ 1: 63, Typ 2: 8, Typ 3: 12, Typ 4: 5, Typ 5: 1, Typ 6: 3. Quelle: wissen/koordination/outputs/, SYNERGIE-REGISTER.md

08.2.5 Offene Geschäftsmodell-Befunde: SYN-05, SYN-09, SYN-11 offen seit 15.07.2026 (66 Tage). Quelle: SYNERGIE-REGISTER.md Statusspalte

08.2.6 Grösse Synergie-Register: 398 kB, 2'042 Zeilen in einer Markdown-Datei. Quelle: wc

08.2.7 Live gehostete Studios: 1 Datei (5.6 MB), deployt 27.06.2026. Quelle: Dateiliste ~/jans-studio-pages (Inhalt nicht gelesen)

08.2.8 Bewertungstool: 1 Datei, 441 Zeilen, 29.6 kB; 10 Gemeinden, davon 5 mit Datenstand 12/2023. Quelle: webtools/bewertungstool/index.html Z. 226-237

08.2.9 Cockpit-Code und Tests: 853 Zeilen in 5 Dateien; automatisierte Tests repo-weit: 0; CI: keine. Quelle: wc, find, kein .github/

08.2.10 Durchlaufzeiten laut Katalog: S1 1 Arbeitstag, S2 1-2, S3 2-3, S4 0.5-2 je Stufe, S5 1-2, S6 0.5-1. Quelle: services/KATALOG.md

08.2.11 Personenbezogene Bestände im Git-Repo: 56 Mail-Batch-Dateien, 18 Dateien Kunden-KB, 45 Dateien Anbieter-Stammdaten, 163 Logbuch-Dateien; Pack-Grösse 546 MiB. Quelle: git ls-files, git count-objects

08.2.12 Stationen im Register: 4 Einträge für 3 Geräte (eine Doppelerfassung mit identischem Hostnamen). Quelle: docs/stationen.md

08.2.13 KI-Kontingent: Max-Abo 20x plus Team-Abo mit 2 Seats (Stand 13.08.2026), Kontenmigration nicht umgesetzt. Quelle: docs/konzepte/260813-Kontingent-Takt/260813-Kontingent-Takt.md

08.3 Offene Fragen

08.3.1 Wie viele Mitarbeitende und Stationen sind in welchem Zeitraum geplant? Das Audit vom 10.06.2026 nennt 5 weitere Stationen, das Stationsregister zeigt 3 Geräte.

08.3.2 Soll GM4 tatsächlich Daten fremder Partnerbüros verarbeiten, auf welcher Vertragsgrundlage, und dürfen diese Daten anonymisiert in die eigene Kennwerte-DB zurückfliessen?

08.3.3 Nutzt die künftige AG denselben Hub als zweiter Mandant oder eine getrennte Instanz? Wie wird die Nutzung zwischen den bewusst unabhängigen Rechtsträgern verrechnet?

08.3.4 Ist das eine live gehostete Studio ein Demo- oder ein echtes Kundenobjekt, und wurden bereits Links an Dritte verschickt? Der Inhalt wurde nicht gelesen.

08.3.5 Ist das Bewertungstool bereits öffentlich erreichbar? Im Repo nicht belegt.

08.3.6 Welche Vertragsbedingungen gelten heute für Kundendaten beim KI-Anbieter (Abo-Bedingungen, Trainingsnutzung, Aufbewahrung)? Im Repo nicht dokumentiert.

08.3.7 Welcher Hosting-Weg gilt für Studios: Hostpoint (Katalog) oder Cloudflare Pages (Script)? Ist Schweizer Hosting zwingend oder genügt EU mit Auftragsbearbeitungsvertrag?

08.3.8 Welches Service-Level wird gegenüber Kunden angestrebt (Verfügbarkeit gehosteter Studios, Reaktionszeit, Lieferfristen der Fixpreis-Produkte)?

08.3.9 Die fünf Strategiepapiere (u.a. Investitionsplan AI Hub) liegen auf SharePoint und wurden nicht gelesen; Budgetrahmen und Investitionsbereitschaft sind unbekannt.

08.3.10 Deckt die Berufshaftpflicht KI-gestützte Fixpreis-Produkte wie Bewertung und Ankaufsprüfung? Im Repo nicht belegt.

08.3.11 Sind die offenen Entscheide 08.1 bis 08.6 der Systemanalyse vom 11.09.2026 (Git als einzige Quelle, GitHub als Origin) gefallen? Sie bestimmen, wo Kunden- und Personendaten künftig liegen.

09 Zuverlässigkeit und Vorfallsgeschichte

09.0 Ist-Zustand. Die Betriebs-Chronik (rules/betrieb-chronik.md) führt 95 Einträge auf 3'505 Zeilen (260 kB), entstanden in siebeneinhalb Wochen seit dem 29.07.2026, also rund 13 Betriebsbefunde je Woche. Die Vorfälle bilden elf Ursachenfamilien. Architekturbedingt und mit einer professionellen

Plattform weitgehend verschwindend: (1) SMB-Mount als Fundament des ganzen Hub, (2) Git-Sync mit zwei Schreibern und Eigenbau-Committer, (3) fehlende Job-Orchestrierung (Leerlauf-Treiber, Doppeltakte, sechs Feuermechanismen je Station), (4) Claude-Desktop-App als Scheduler mit persönlichem OAuth, (5) Neustart- und Fernzugangsfestigkeit der Always-On-Station, (6) RAM-Notstände durch hängende Desktop-Sessions, (7) Datenschutz-Sweep mit Auto-Commit nach GitHub. Betrieblich, bleibt auch danach, wird aber durch CI, Monitoring und Runbooks beherrschbar: (8) stille Fehlschläge und falsche Zeiger in Shell-Scripts, (9) Datums- und Uhrfehler, (10) OneDrive/FileProvider-Störungen auf den Arbeitsplätzen, (11) der Inhaber als einziger Freigabe-Engpass für privilegierte Eingriffe. Das System erkennt Störungen zuverlässig und dokumentiert sie vorbildlich, behebt aber fast nur Symptome: jeder Vorfall erzeugt einen weiteren Wächter, eine weitere Regel oder einen Registereintrag. Zwei eigene Analysen (Hub-Audit 12.08., Systemanalyse 11.09.) benennen den Konstruktionsfehler bereits selbst; der Umbau (V0+, lauf.sh) ist begonnen, aber nicht verifiziert und teilweise blockiert.

09.1 Schwachstellen

09.1.1 Gesamte Hub-Fähigkeit hängt an einem SMB-Mountpoint (kritisch, SPOF). Skills, Agenten, Rules (@-Importe), Wissens-Layer und Logbuch liegen auf einer SMB-Freigabe; fällt der Mount, ist die arbeitende Station für den ganzen Hub blind. Belegt sind sieben Ausfalltage vom 25.07. bis 07.09.2026 mit bis 275 Abrissen an einem Tag (07.09., rund 5.5 h ohne Hub), 59 Abrisse am 31.08., vier Stunden Wächter-Endlosschleife unter einem falschen Mountnamen (83 ERROR-Zeilen), zwei stumm gestorbene Treiberläufe (bash liest Script vom NAS nach) und Schreibvorgänge, die nach fehlgeschlagenem cd still im SSD-Klon landeten. Die Wächter heilen, verhindern aber nichts. Beleg: rules/betrieb-chronik.md Abschnitte 260831 (Z. 435-472), 260831b (Z. 401-433), 260901 (Z. 370-399), 260824 (Z. 1266-1292), 260807 (Z. 2294-2329); docs/konzepte/260911-Systemanalyse-Hub-Architektur-NAS-Git/260911-Systemanalyse-Hub-Architektur-NAS-Git.md Abschnitt 02.1 und 03.2

09.1.2 Git-Sync als Eigenbau mit zwei Schreibern auf main (kritisch, Eigenbau). Weil Git über SMB hängt, committet die Synology per Cron alle 15 Min mit git add -A, während die Stationen parallel nach GitHub pushen. Folgen: 18 h ohne Commit und 41 h bis zur Reparatur (178 Skips, 44 uncommittete Dateien), 6 h Divergenz am 24.08. (26 gegen 51 Commits), vier Tage Divergenz 29.08. bis 02.09. mit 488 vergeblichen Abgleichversuchen (Stufe P1), zweimal verwaister index.lock an einem Tag, Konfliktmarker in die kanonische Quelle committet (1'515 statt 822 Zeilen), Fremdarbeit unter falscher Commit-Message eingesammelt. Eine Konfliktmarker-Prüfung fehlt im Committer weiterhin. Beleg: rules/betrieb-chronik.md Abschnitte 260811 (Z. 2140-2171), 260812 (Z. 2058-2103), 260824e (Z. 939-1021), 260823c (Z. 1296-1339), 260813d (Z. 1716-1754), 2026-08-23 (Z. 1208-1231); scripts/nas-selfcommit.sh:81 (git add -A, kein Marker-Check); logbuch/fristen.md Z. 379 (Versuch 488)

09.1.3 Keine Job-Orchestrierung: Leerlauf-Treiber, Doppeltakte, Kontingent-Totalausfall (kritisch, Eigenbau). Je Station feuern sechs unabhängige Mechanismen (App-Scheduled-Tasks, vollgas-supervisor, synctask-runner, wissens-trigger, launchd-Trainings, Nachtschicht) ohne gemeinsame Queue, Idempotenz oder Abbruchlogik. Belegt: über 600 ertraglose Läufe in einer Nacht (344+185+96+52), Doppelläufe am 25.07. mit 53 Mio Tokens und anschliessendem Totalausfall beider Stationen vom 26.07. 00:53 bis 27.07. 12:00 (rund 4'700 Fehlversuche, Morgenbriefing ausgefallen), Energie-Loop drei Nächte doppelt gestartet, verwaistes /tmp-Script verteilte 14 identische erledigte Aufträge, acht Sessions suchten die Quelle. In 30 Tagen waren 1'074 von 2'439 automatischen Läufen Fehlstarts. Beleg: rules/auto-verbesserungen.md Eintrag 260830; rules/betrieb-chronik.md Abschnitte 260727 (Z. 2644-2682), 260728 (Z. 2559-2589), 260803 (Z. 2331-2367), 2026-08-24 03:40 (Z. 1176-1206); logbuch/rollen/260911-rollen-bilanz.md Abschnitt 1

09.1.4 Scheduler ist die Claude-Desktop-App mit persönlichem OAuth-Login (kritisch, SPOF). Die operativen Tasks (Tagesbriefing, Fristen-Radar, Mahnwesen, Zahlungsabgleich) hängen an der GUI-App. Ein gelatchter OAuth-Zustand legte das MacBook Pro rund 66 h still (11.09. 20:08 bis 14.09. 13:59), den Mac Mini rund 39 h mit 11 gescheiterten Starts, ohne Registereintrag; das Tagesbriefing fiel vier Tage in Folge aus. Bereits 09. bis 12.08. vier Tage ohne Briefing, weil die Task-Engine auf der mobilen Station nicht lief. Die Registry ist je Station getrennt, von Sessions nicht steuerbar, Frontmatter-Felder (model, enabled) wirken nicht, ein Nachholmechanismus für verpasste Läufe ist nicht belegt. Beleg: rules/betrieb-chronik.md Abschnitte 260914b (Z. 3376-3384), 260812b (Z. 2034-2056), 260816 (Z. 1442-1465); logbuch/fristen.md Z. 26-38; docs/konzepte/260812-Hub-Audit/260812-Hub-Audit.md R3, R11 (Z. 500-555)

09.1.5 Always-On-Station ist weder neustartfest noch verlässlich von aussen erreichbar (kritisch, SPOF). FileVault ohne Auto-Login und alle 12 Jobs als LaunchAgents im User-Kontext: nach Stromausfall steht der Mac Mini am Passwort-Prompt, kein Dienst startet. Tailscale läuft als GUI-App; der Tunnel war vom 20. bis 24.08. vier Tage aus, viermal korrekt gemessen und nie gemeldet, weil als Hub-Internum eingestuft und der Watchdog über den LAN-Mount frische Stempel sah. Der danach gebaute Wächter wurde 24 Tage lang auf keiner Station geladen und ist auf dem Mini weiterhin offen. Alle Job-Logs lagen in /tmp und gehen beim Reboot verloren. Beleg: docs/konzepte/260812-Hub-Audit/260812-Hub-Audit.md Abschnitt 2.1 (Z. 225-270), R1, R9; rules/betrieb-chronik.md Abschnitte 260824c (Z. 1055-1121), 260826b (Z. 717-743), 260917b (Z. 3442-3467); templates/launchd/ch.jans.tailscale-waechter.plist (Logpfad /tmp)

09.1.6 Monitoring misst Lebenszeichen statt Erfolg; Alarmkanal ist Apple Mail per osascript (hoch, Betriebsaufwand). Der heartbeat hielt einen 41 h stehenden Committer für gesund, weil der Skip selbst eine Logzeile schrieb; git-auto-sync und Sync-Task-Runner schweigen im Leerlauf, ein toter Job sieht aus wie ein ruhiger Tag. Warnmails blieben mehrfach unversendet in den Entwürfen (Kontingent-Warnungen 08./09.08., AG-Monitor drei Tage in Folge), das Versandscript quittiert die Übergabe, nicht die Zustellung; osascript-Timeouts machten den Mail-Monitor drei Tage blind. Der bexio-Zugang war 36 Tage tot (401), die Connector-Diagnose nannte eine falsche Ursache. Beleg: rules/betrieb-chronik.md Abschnitte 260812 (Z. 2093-2103), 260812b (Z. 2043-2048), 260824b (Z. 1235-1264), 260826 (Z. 745-755), 260917c (Z. 34-44); docs/konzepte/260812-Hub-Audit/260812-Hub-Audit.md R5 (Z. 513-518); logbuch/fristen.md Z. 110

09.1.7 Keine CI, keine Testsuite: Schutzmechaniken mit totem Code in Produktion (hoch, Testluecke). Es gibt kein .github/workflows, kein shellcheck, keine Unit-Tests; nur zwei Selbsttest-Scripts. Belegte Folgen: Reboot-Warnung des Tailscale-Wächters war seit dem ersten Tag toter Code (grep -c mit || echo 0), dasselbe Muster in inventar.sh und session-insights.sh; tr -d '-' lieferte auf BSD einen Leerstring und akzeptierte jeden Stempel; kopierter Code mit falschem Variablennamen ohne set -u; das Lauf-Gate prüfte eine falsche Metrik und wies ab 07:34 ausnahmslos jeden automatischen Lauf auf beiden Stationen ab; der Speicher-Wächter erzeugte 24 Fehlalarme täglich je Station; der Freigabe-Guard liess ein relatives rm -rf auf Git-Interna durch. Beleg: rules/betrieb-chronik.md Abschnitte 260826b (Z. 730-739), 260904 (Z. 269-297), 260812f (Z. 1896-1908), 260824f (Z. 915-926), 260728 (Z. 2597-2620), 260811 Befund 2 (Z. 2173-2206), 260730 (Z. 2513-2517); kein Verzeichnis .github vorhanden

09.1.8 Inhaber als einziger Engpass für privilegierte Eingriffe; Befunde warten tagelang (hoch, Governance). Die Schutzschichten (Guard, Klassifikator) halten korrekt zurück, aber niemand hat die Betriebsrolle, die Freigaben zeitnah abzuarbeiten: 41 h Sync-Stillstand wegen ungelesener Freigabe-Queue (Task zudem an die mobile Station adressiert), Energie-Doppeltakt drei Nächte trotz fertigem Dreizeiler, Tailscale-Wächter 24 Tage zwischen gebaut und geladen, /etc/nsmb.conf tagelang offen.

Deployment bedeutet Handarbeit des Inhabers per Terminal; «Aktion Raphael» ist der Standardausgang vieler Einträge. Beleg: rules/betrieb-chronik.md Abschnitte 260812 (Z. 2065-2071, 2105-2123), 260803 (Z. 2342-2367), 260917b (Z. 3444-3463), 260901 (Z. 379-385), 260911b (Z. 71-82)

09.1.9 Datenschutz: Massen-Sweep über Geschäftsdaten plus Auto-Commit nach GitHub; sensible Register in Git (hoch, Datenschutz). Ein 50-Agenten-Inventarlauf über Büroarchive erfasste eine Ablage mit Bewerbungsmappen; die Inventardatei mit Namen einer bewerbenden Person wurde binnen 15 Min vom Auto-Committer gepusht, die Historie bewusst nicht bereinigt. Ausschlusslisten existieren nur als Regeltext, nicht technisch. Zusätzlich liegen in Git-versionierten Betriebsdateien (Chronik, Fristen-Register) Netzadressen, Zertifikats-Fingerprints, Namen und Mailadressen Dritter, Kundenkonflikte und eine Bankverbindung. Für ein Büro mit Ausbauabsicht ist das unter dem Schweizer DSG (Bearbeitungsgrundsätze, Auftragsbearbeitung, Datensicherheit) eine offene Flanke. Beleg: rules/auto-verbesserungen.md Eintrag 260823; rules/betrieb-chronik.md Z. 1049 und Z. 1238-1240 (Kategorie, Inhalt nicht übernommen); logbuch/fristen.md Z. 104-137 (Kategorie, Inhalt nicht übernommen); scripts/nas-selfcommit.sh:81

09.1.10 Lokale OneDrive-Sync-Pfade als Produktionsabhängigkeit (hoch, Cloud-Sync). Hub-Output und Projektablage laufen über lokale FileProvider-Pfade. Vorfall 08. bis 12.08.: rekursiv verschachtelte Sync-Wurzeln, mehrere gescheiterte Neuinstallationen, fünf Reboots, 2.8 GB Quarantäne ohne Abgleich. Am 10.09. synchronisierte das MacBook nur 3 von 17 Bibliotheken bei Status «synchronisiert», der Index war seit dem 11.02. gebrochen (sieben Monate unbemerkt); die Bereinigung dauerte einen Abend, der Quarantäne-Ordner verschwand, am 17.09. entstand erneut ein verdoppelter Stamm. OneDrive syncet zudem unter einer Admin-Identität, Office unter der Benutzeridentität. Beleg: docs/konzepte/260812-OneDrive-Sync-Incident/README.md; rules/betrieb-chronik.md Abschnitte 260910 (Z. 126-246), 260914 (Z. 3358-3374), 260917 (Z. 3421-3440); docs/konzepte/260812-Hub-Audit/260812-Hub-Audit.md R4

09.1.11 Ressourcengrenzen der Arbeitsplatzrechner als Laufzeitumgebung (mittel, Skalierung). Zwei Speicher-Notstände: 28.07. (OneDrive hielt 33 GB bei 32 GB RAM auf dem Mini) und 02.09. (16 hängende Desktop-Sessions, 18.13 GB Footprint auf der 16-GB-Maschine, Mail und Vorschau reagierten nicht). Das Lauf-Gate erreicht die App-Scheduled-Task-Flotte baulich nicht; der Schutz ruht dort allein auf Taktentzerrung. Session-Transcripts wuchsen unbegrenzt (4.3 und 5.2 GB). Mit Mitarbeitenden oder Mandanten skaliert dieses Modell nicht. Beleg: rules/betrieb-chronik.md Abschnitte 260902 (Z. 299-336), 260728 (Z. 2578-2596, 2635-2637); rules/auto-verbesserungen.md Eintrag 260902

09.1.12 Zeit- und Audit-Integrität nicht gesichert (mittel, Governance). Die Stationsuhr lief unbemerkt zehn Tage nach; git log --since schnitt still drei Viertel des Deltas weg (262 statt 1'111 Commits) und führte zur gegenteiligen Betriebsempfehlung. Ein korrekt datierter Lauf wurde irrtümlich um vier Tage umdatiert. Acht Scripts datierten per toISOString in UTC. Commits sind als Urheber- und Datierungsbeleg unbrauchbar (git add -A sammelt Fremdarbeit; 67 % aller Commits sind Auto-Commits), Git-Identität war zeitweise nicht gesetzt. Beleg: rules/betrieb-chronik.md Abschnitte 260813e (Z. 1679-1714), 260807c (Z. 2208 ff.), 260813d (Z. 1716-1754); rules/auto-verbesserungen.md Eintrag 260730b; docs/konzepte/260812-Hub-Audit/260812-Hub-Audit.md R17

09.2 Kennzahlen

09.2.1 Einträge Betriebs-Chronik: 95 Einträge, 3'505 Zeilen, 260 kB seit 29.07.2026 (rund 13 je Woche). Quelle: grep -c '^## ' und wc auf rules/betrieb-chronik.md; Erst-Commit der Datei 29.07.2026 per git log

09.2.2 Mount-Ausfalltage MacBook Pro: 7 Tage zwischen 25.07. und 07.09.2026, Spitze 275 Abrisse an einem Tag, 5.5 h ohne Hub. Quelle: docs/konzepte/260911-Systemanalyse-Hub-Architektur-NAS-Git/...md Abschnitt 00.2 und 02.1

09.2.3 Sync-Stillstände und Spaltungen: 18 h ohne Commit bzw. 41 h bis Reparatur (178 Skips, 44 Dateien); 6 h Divergenz 24.08.; 4 Tage 29.08.–02.09. mit 488 Versuchen (396/266 Commits). Quelle: rules/betrieb-chronik.md 260811, 260812, 260824e; logbuch/fristen.md Z. 379

09.2.4 Ertraglose Läufe in einer Nacht (30.08.2026): 677 (344 + 185 + 96 + 52), je Lauf USD 0.20 bis 1.36. Quelle: rules/auto-verbesserungen.md Eintrag 260830

09.2.5 Kontingent-Totalausfall: 26.07. 00:53 bis 27.07. 12:00 auf beiden Stationen, rund 4'700 Fehlversuche, 53 Mio Tokens Doppelläufe am 25.07.. Quelle: rules/betrieb-chronik.md 260727 (Z. 2655-2660)

09.2.6 Fehlstart-Quote automatischer Läufe (30 Tage bis 11.09.): 1'074 von 2'439 Läufen (44 %); geliefert 1'365 Läufe, 95.8 h, USD 2'095 (SDK-Selbstauskunft); produktive Zeit 6.7 % der Wandzeit. Quelle: logbuch/rollen/260911-rollen-bilanz.md Abschnitte 1 und 3

09.2.7 Anteil Aufsichts-Bausteine: 35 von 110 Bausteinen und 17 von 37 Loops tragen die Rolle maintainer. Quelle: logbuch/rollen/260911-rollen-bilanz.md Abschnitt 2

09.2.8 Commit-Rauschen (30 Tage): 4'065 Commits, davon 2'736 nas-selfcommit (67 %) und 356 auto-sync; zusammen 76 % maschinell. Quelle: git log --since='30 days ago' im Worktree, 19.09.2026

09.2.9 Scripts für Selbstreparatur: mindestens 15 von 100 Scripts sind Wächter, Guards, Gates oder Reparatur Helfer; 8 Scripts existieren nur wegen der SMB-Bauweise; 236 Dateien mit fest verdrahtetem NAS-Pfad. Quelle: ls scripts (Namensmuster), grep -rl '/Volumes/daten'; Systemanalyse 260911 Abschnitt 01.2

09.2.10 Fernzugangs-Ausfall Tailscale: 4 Tage (20.–24.08.2026), viermal gemessen, nie gemeldet; Wächter 24 Tage zwischen gebaut und geladen. Quelle: rules/betrieb-chronik.md 260824c, 260917b; rules/auto-verbesserungen.md 260824

09.2.11 Ausfall App-Scheduler (OAuth-Latch): MacBook Pro rund 66 h, Mac Mini rund 39 h mit 11 gescheiterten Starts; Tagesbriefing 4 Tage in Folge ausgefallen. Quelle: rules/betrieb-chronik.md 260914b; logbuch/fristen.md Z. 26-38

09.2.12 Speicher-Notstände: 2 (28.07.: 20.24 GB bzw. OneDrive 33 GB; 02.09.: 16 Sessions, 18.13 GB auf 16 GB RAM). Quelle: rules/betrieb-chronik.md 260902, 260728

09.2.13 OneDrive-Vorfälle: 3 (08.–12.08. mit 5 Reboots und 2.8 GB Quarantäne; 10.09. nur 3 von 17 Bibliotheken, Index seit 11.02. gebrochen; 17.09. erneut verdoppelter Stamm). Quelle: docs/konzepte/260812-OneDrive-Sync-Incident/README.md; rules/betrieb-chronik.md 260910, 260917

09.2.14 Toter Connector-Zugang: bexio 36 Tage 401 (12.08.–17.09.2026), Token-Ablauf 60 Tage nicht überwacht. Quelle: rules/betrieb-chronik.md 260917c

09.2.15 Testabdeckung: 0 CI-Workflows, 0 Unit-Tests, 2 Selbsttest-Scripts; 67 von 84 Shell-Scripts mit set -u. Quelle: Verzeichnisprüfung .github, find '*test*', grep auf scripts/*.sh

09.2.16 Freigabe-Queue Stand 19.09.2026: 1 wartend, 184 erledigt; Queue nicht versioniert. Quelle: /Volumes/daten/jans-ai-hub/sync-tasks (nur Dateizählung); rules/betrieb-chronik.md Z. 2130-2134

09.3 Offene Fragen

09.3.1 Wie gross ist der wahre Anteil von Aufsicht und Selbstreparatur am Gesamtverbrauch? Die Rollen-Bilanz erfasst nur die Runner-Läufe; die App-Scheduled-Task-Flotte (wo fast die ganze Aufsicht liegt) und interaktive Reparatur-Sessions sind nicht gemessen (logbuch/rollen/260911-rollen-bilanz.md Abschnitt 6). Die ausgewiesenen 1 % maintainer-Zeit sind darum kein belastbarer Wert.

09.3.2 Sind die USD-Beträge des Lauf-Journals reale Grenzkosten oder rechnerische Werte unter dem Max-Abo? Für eine Kosten-Nutzen-Rechnung der neuen Plattform muss das geklärt werden.

09.3.3 Stand des Umbaus V0+ vom 11.09.2026: Index-Bereinigung und launchd-Installation auf dem Mini waren vom Klassifikator geblockt; laufen die Aufsichts-Tasks derzeit doppelt auf MacBook Pro und Mac Mini?

09.3.4 lauf.sh ist gebaut, aber von keinem Treiber aufgerufen; Verifikationskriterien V1 und V3 sind nicht gemessen, die Hypothese zum Fehler «OAuth session expired» bei abgelösten Läufen ist unbestätigt.

09.3.5 Liegt der Commit mit dem Bewerberdatensatz vom 23.08.2026 weiterhin in der GitHub-Historie, und bestehen Auftragsbearbeitungsverträge bzw. eine Datenstandortklärung mit Anthropic, GitHub und Microsoft? In den gelesenen Quellen nicht belegt.

09.3.6 Ursachen unbelegt: Stopp von Tailscale am 20.08., zehn Tage nachgehende Stationsuhr am 12.08., verschwundener Quarantäne-Ordner am 10.09., wiederkehrender OAuth-Latch der Desktop-App.

09.3.7 Wurde die OneDrive-Quarantäne vom 08.08. (2.8 GB, 1'133 Dateien) je gegen SharePoint abgeglichen? Das Hub-Audit führt sie als R4 ohne Registereintrag.

09.3.8 Holt die Claude-Desktop-App verpasste Scheduled Tasks nach? Im Fristen-Register ausdrücklich als nicht belegt vermerkt.

09.3.9 Backup-Konzept des NAS (Off-Site, Wiederherstellungstest) und der nicht versionierten Bereiche (sync-tasks, logbuch/laeufe, Archimed 460 MB) ist in den gelesenen Quellen nicht dokumentiert; NAS-Volume zu 93 % belegt.

09.3.10 Ist /etc/nsmb.conf inzwischen wirksam? Die Frühwarnung vom 01.09. deutet laut Systemanalyse auf eine Fehldiagnose hin (User-Fassung galt bereits); der tatsächliche Zustand je Station ist offen.

10 Backup, Wiederherstellung, Monitoring, Kosten

10.0 Ist-Zustand. Backup, Wiederherstellung und Überwachung des JANS AI Hub sind vollständig Eigenbau aus Shell-Scripts, launchd-Jobs und Claude-Scheduled-Tasks. Für den Hub selbst besteht eine funktionierende Sicherungskette: Das NAS committet sich alle 15 Minuten nativ nach GitHub (einziges Off-Site-Ziel), die Stationen halten SSD-Klone. Für die eigentlichen Geschäftsdaten auf dem NAS (Archiv, Büro, Bauprodukte; 6.0 TiB, 93 % belegt) ist im Repo dagegen nur eine Inhaber-Aussage vom 10.06.2026 belegt («Hyper Backup/Snapshots vorhanden»); Ziel, Takt, Aufbewahrung und ein Restore-Test sind nirgends dokumentiert, RPO/RTO und ein Notfallplan fehlen. Das NAS ist zugleich Dateiserver, kanonische Hub-Quelle und Ablage aller Monitoring-Logs. M365 (SharePoint-Pool 79 %) wird durch einen täglichen Hygiene-Lauf gut vermessen, aber nicht gesichert; bexio ebenso wenig. Die

Überwachung besteht aus Heartbeat (Script 9 Checks, Skill 16 Checks), Stations-Watchdog, Speicher-, Kontingent-, Tailscale- und Session-Wächter sowie einem statischen HTML-Cockpit. Einziger Alarmkanal ist Apple Mail per osascript, der mehrfach still in den Entwürfen endete. Mehrere mehrtägige Ausfälle (Tailscale 4 Tage, Aufsichts-Tasks 65 h, Sync-Spaltung 4 Tage) wurden gemessen, aber spät oder gar nicht gemeldet. Kosten werden nur für das Claude-Kontingent erhoben, diese Messung ist seit 23.08.2026 blind. Die Betriebschronik dokumentiert Vorfälle vorbildlich; die Architektur erkennt ihre eigenen Ausfälle jedoch nicht zuverlässig. Pfadangaben in den Belegen sind relativ zur Hub-Wurzel /Volumes/daten/jans-ai-hub (Git-Spiegel im Worktree).

10.1 Schwachstellen

10.1.1 Keine belegte, getestete 3-2-1-Backupstrategie für die NAS-Geschäftsdaten (kritisch, Backup/DR). Für Archiv, Büro- und Projektdaten auf dem NAS existiert im gesamten Hub nur eine Inhaber-Aussage, das Backup «laufe bereits». Ziel, Takt, Aufbewahrung, Verschlüsselung, Off-Site-Standort und letzter erfolgreicher Lauf sind nirgends dokumentiert oder überwacht. Ein Wiederherstellungstest ist nicht belegt. Der Ordner logbuch/backups/ ist leer und wird von keinem Script referenziert. Beleg: docs/systemaudit-260610.md:52 («NAS-Backup: laeuft bereits (Hyper Backup/Snapshots vorhanden) → kein Handlungsbedarf»); Volltextsuche nach Hyper Backup, Snapshot, RAID, Restore, Wiederherstellungstest in rules/betrieb-chronik.md, logbuch/fristen.md, connectors/hub-setup-daten.json, docs/, scripts/ ohne Treffer zur Konfiguration; /Volumes/daten/jans-ai-hub/logbuch/backups/ leer (Stand 19.09.2026)

10.1.2 NAS ist SPOF für Büro UND Hub, 93 % voll, ohne Hardware-Health-Monitoring (kritisch, SPOF). Ein einzelnes DS918+ ist Dateiserver, kanonische Hub-Quelle, Sync-Task-Queue und Ablage aller Monitoring-Logs. Kein Check überwacht Volume-Füllstand, RAID-Zustand, SMART, Temperatur oder den Backup-Job. Ein mehrstündiger NAS-Ausfall legte den ganzen Hub still. Beleg: df /Volumes/daten am 19.09.2026: 6.0 TiB, 93 %, 467 GiB frei; docs/konzepte/260911-Systemanalyse-Hub-Architektur-NAS-Git/...md Abschnitt 00.1 und 02.1 (07.09.: NAS rund 5.5 h nicht erreichbar, bis 275 Mount-Abrisse pro Tag); scripts/heartbeat.sh:112-133 prüft nur die lokale Disk (\$HOME); grep nach smartctl|mdstat|synobackup|RAID in scripts/ und skills/heartbeat ohne Health-Check-Treffer

10.1.3 Alarmkanal Apple Mail per osascript fällt still aus, kein zweiter Kanal (kritisch, Alarmierung). Alle Wächter und Aufsichts-Loops melden ausschliesslich per Apple Mail über osascript. Der Weg endete mehrfach unbemerkt im Entwurfsordner bzw. im AppleEvent-Timeout; send quittiert nur die Übergabe, nicht die Zustellung. Es gibt weder Push/SMS noch einen externen Dead-Man-Switch. Beleg: logbuch/abo-check/UNZUSTELLBAR.md (06.09.2026: «Ein Alarmweg, der still in den Entwürfen endet, ist kein Alarmweg»); rules/betrieb-chronik.md Abschnitte 260826 (osascript-Timeout, dritter Tag, Monitor blind) und 260824b; logbuch/kontingent/waechter.log 24.08.2026 zweimal «MAIL FEHLGESCHLAGEN»; grep nach ntfy|pushover|healthchecks|uptime|grafana|prometheus in scripts/, skills/, connectors/, docs/ ohne Treffer

10.1.4 Die Überwachung hängt am Überwachten (hoch, Monitoring). Wächter schreiben und lesen ihren Zustand auf dem NAS und treten bei fehlendem Mount still zurück. Die Aufsichts-Tasks (Heartbeat, Tagesbriefing, Radar) laufen als Claude-Desktop-Scheduled-Tasks und fielen wegen eines App-Login-Latch rund 65 Stunden auf beiden Stationen aus; der Ausfall des Mac Mini stand in keinem Register. Es gibt keine Instanz ausserhalb von NAS, Claude-App und den zwei Macs. Beleg: scripts/kontingent-waechter.sh:45 («NAS nicht gemountet: still zuruecktreten»); scripts/stationen-watchdog.sh:35 und :64-69 (Exit 2 «nicht pruefbar»); rules/betrieb-chronik.md Abschnitt 260914b; logbuch/LOGBUCH.md:13288-13299 (hub-chef, heartbeat-daily u.a. viermal in Folge gescheitert, rund 65 h); docs/konzepte/260812-Hub-Audit/260812-Hub-Audit.md Kernaussage («Das System erkennt seine eigenen Ausfälle nicht»)

10.1.5 Kein Backup der SaaS-Daten (M365/SharePoint, bexio); Versionsverlauf als einziges Netz wurde gekürzt (hoch, Backup/DR). Projektdaten liegen kanonisch in SharePoint, die Buchhaltung kanonisch in bexio. Ein Drittanbieter-Backup oder periodischer Export ist nicht belegt. Aus Platzgründen wurde der Versionsverlauf auf 5 Hauptversionen mit 30 Tagen Verfall getrimmt (endgültige Löschung am Papierkorb vorbei). Seit 17.09.2026 bucht Claude in bexio selbständig ohne Betragsgrenze, was den Bedarf an einem unabhängigen Datenstand erhöht. Beleg: rules/betrieb-chronik.md Abschnitt 260915 (Trim MajorVersionLimit 5, «permanently deleted»); tenant-hygiene/reports/260918-hygiene.md Big Points (828.71 GB = 79.38 %, rund 1 GB/Tag, ExpireVersionsAfterDays 30); rules/auto-verbesserungen.md Eintrag 260702 (bexio führend, selbständiges Buchen); Suche nach M365-Backup/Retention/bexio-Export in docs/, rules/, skills/, scripts/ ohne Treffer

10.1.6 Kein Notfallplan, keine RPO/RT0; Always-On-Station ist nicht neustartfest (hoch, Governance/DR). Es gibt kein Wiederanlauf-Dokument, keine definierten Wiederherstellungsziele und keine Vertretungsregel. Der Mac Mini bleibt nach Stromausfall am FileVault-Prompt stehen, alle Jobs sind User-LaunchAgents; der Entscheid dazu ist seit 12.08.2026 offen. Eine USV ist nirgends dokumentiert. Beleg: docs/konzepte/260812-Hub-Audit/260812-Hub-Audit.md:244-252; docs/konzepte/260911-Systemanalyse-...md Abschnitt 08.2 (Entscheid offen); Wortsuche RTO|RPO|Notfallplan|Disaster in docs/, rules/, scripts/ ohne Treffer; USV|Stromausfall liefert nur die FileVault-Befunde

10.1.7 GitHub als einziges Off-Site: deckt Geschäftsdaten nicht ab und enthält Personendaten Dritter (hoch, Datenschutz). Off-Site liegt nur, was im Git ist. Ausgeschlossen sind sync-tasks, Lauf-Journal, secrets, Archimed (460 MB), Behörden-Spiegel. Umgekehrt liegen Register mit Kontakt-, Projekt- und Korrespondenzdaten (fristen.md 1.1 MB, Konversations-Destillate) bei einem US-Anbieter; einmal gepushte Bewerberdaten blieben bewusst in der Historie. Eine DSGVO-Beurteilung (Auslandbekanntgabe, Lösbarkeit) fehlt. Beleg: docs/konzepte/260812-Hub-Audit/260812-Hub-Audit.md:150 («einziges Off-Site»); .gitignore (output/, sync-tasks/, dispatch/, logbuch/laeufer/, secrets/, behoerden-dokumente/); 260911-Systemanalyse Abschnitt 01.2 («Im NAS-Repo, aber nicht in Git»); rules/auto-verbesserungen.md Eintrag 260823 (Commit gepusht, Historie nicht umgeschrieben)

10.1.8 Heartbeat ist Eigenbau ohne Tests, mit Drift zwischen Skill und Script und ohne Zeitreihe (mittel, Eigenbau/Testlücke). Das Script kennt 9 Checks, der Skill 16 (Rest als Prosa, vom LLM ausgeführt). Tageslock erlaubt nur einen Lauf pro Tag, das Ergebnis wird nicht persistiert (kein Verlauf, keine Trends). IP-Adresse sowie Tenant- und App-ID sind im Script hartkodiert. Der Skill ruft git gegen den SMB-Mount auf, was die eigene Rule verbietet. Für 84 Shell-Scripts existieren 2 Test-Scripts, keine CI. Beleg: scripts/heartbeat.sh:25-32 (Tageslock), :56 und :101 (hartkodierte Werte), :338-347 (8 JSON-Checks); skills/heartbeat/SKILL.md:11 («kein Datei-Output»), :110-113 (git -C /Volumes/daten/...), Überschriften 1-16; scripts/station-status.sh:98 (git über SMB); ls scripts/*.sh = 84, Tests nur schutzmechanik-selbsttest.sh und vermaschungs-test.sh

10.1.9 Keine Metriken, kein zentrales Logging, keine Log-Rotation (mittel, Observability). Betriebsdaten liegen als frei formatierte Textlogs verstreut auf NAS, /tmp und in Home-Verzeichnissen; Aufbewahrung und Rotation sind nur für Transkripte geregelt. Das Cockpit ist eine statische HTML-Seite aus Markdown-Registern ohne Zeitreihen; der frühere Live-Monitor ist ausgebaut. Trends (Kapazität, Job-Erfolg, Kosten) sind nicht sichtbar. Beleg: /Volumes/daten/jans-ai-hub/logbuch/speicher/ (gate-Macmini.log 157 kB, Macbookpro.log 129 kB, ohne Rotation); webtools/cockpit/README.md (Panels, Logs unter /tmp/com.jans.cockpit*); scripts/stationen-

watchdog.sh (LOGFILE in ~/jans-watchdog); scripts/vollgas-monitor-build.sh:1-12 («AUSGEBAUT ... NICHT MEHR LADEN»)

10.1.10 Cockpit-Webserver ohne Authentisierung und ohne TLS an allen Interfaces (mittel, Sicherheit). Der Mac Mini liefert das Cockpit (Fristen, Projekt- und Kundenbezüge, Stationszustand) per HTTP an 0.0.0.0 aus; Schutz besteht allein durch Netzgrenze (LAN/Tailnet). Mit Mitarbeitenden, Gästen im LAN oder geteilten Tailnet-Knoten ist das nicht tragfähig. Beleg: webtools/cockpit/serve-cockpit.mjs:34 (listen(PORT, '0.0.0.0')); webtools/cockpit/README.md Abschnitt «Zugriff von MacBook / Handy» (LAN + Tailnet, keine Anmeldung)

10.1.11 Kontingent- und Kostenmessung fragil; keine Gesamtkostensicht (mittel, Kosten). Der Usage-Connector nutzt einen nicht dokumentierten OAuth-Endpunkt mit der Token-Kopie der Claude-CLI und schreibt rotierte Tokens in deren Keychain-Eintrag zurück. Seit 23.08.2026 liegt keine belastbare Abo-Messung vor (Ampel SCHWARZ). Team-Seats sind nicht messbar. Eine Übersicht aller IT-Kosten (Claude-Abos, M365, bexio, Hosting, GitHub, Tailscale, Hardware-Abschreibung) existiert nicht. Beleg: connectors/claude-usage.mjs:48-68 (persistiereRotation) und :141 (Usage-Endpunkt); logbuch/abo-check/status.json (ausfaelleInFolge 2, ampel SCHWARZ, letzter Lauf 06.09.2026, «Team-Seat 1 und 2 ... nicht separat messbar»); Suche nach Kostenübersicht/Betriebskosten/TCO in docs/ und services/ ohne Treffer

10.1.12 Transkript-Archiv und Betriebslogs mit Kundendaten unverschlüsselt; Endgeräte-Backup uneinheitlich (mittel, Datenschutz/Backup). Session-Transkripte enthalten laut eigener Doku Mailinhalte, Kundennamen und Projektinterna und liegen 180 Tage als tar.gz auf dem NAS-Share; ob sie im NAS-Backup verschlüsselt mitgesichert werden, ist offen. Time Machine ist nur für den Mac Mini belegt (eine externe Platte), für das MacBook Pro wurden 0 lokale Snapshots gemessen; lokale Secrets (.env nur auf einer Station, Zertifikat) haben kein belegtes Escrow. Beleg: scripts/transcript-rotation.sh:17-22 und :33-36; rules/betrieb-chronik.md Abschnitt 260814e (Mini: Ziel «Elements», 24 Snapshots; MacBook: 0) und 260814d; docs/konzepte/260812-Hub-Audit/260812-Hub-Audit.md SPOF 4 (.bexio.env nur auf dem MacBook Pro)

10.2 Kennzahlen

10.2.1 NAS-Volume Belegung: 6.0 TiB gesamt, 5.6 TiB belegt (93 %), 467 GiB frei. Quelle: df -h /Volumes/daten, gemessen 19.09.2026

10.2.2 SharePoint-Pool: 828.71 GB von 1044.00 GB (79.38 %), Zuwachs rund 1 GB/Tag, Restlaufzeit 91 bis 201 Tage. Quelle: tenant-hygiene/reports/260918-hygiene.md

10.2.3 Dokumentierte Restore-Tests / definierte RPO-RTO: 0 / keine. Quelle: Volltextsuche docs/, rules/, scripts/, logbuch/ (19.09.2026)

10.2.4 Heartbeat-Checks Script vs. Skill: 9 vs. 16. Quelle: scripts/heartbeat.sh, skills/heartbeat/SKILL.md

10.2.5 NAS-Mount-Abrisse (MacBook Pro): 7 Ausfalltage 25.07. bis 07.09.2026, bis 275 Abrisse pro Tag; am 07.09. rund 5.5 h ohne NAS. Quelle: docs/konzepte/260911-Systemanalyse-Hub-Architektur-NAS-Git, Abschnitt 02.1

10.2.6 Spaltungen der Sicherungskette NAS/GitHub: 18 h ohne Commit (11./12.08.), 6 h (24.08.), 4 Tage mit 488 vergeblichen Abgleichen (29.08. bis 02.09.). Quelle: 260911-Systemanalyse, Abschnitt 02.2

10.2.7 Unbemerkte bzw. ungemeldete Ausfälle: Tailscale Mac Mini 4 Tage (20. bis 24.08.); Aufsichts-Tasks rund 65 h (11. bis 14.09.). Quelle: rules/auto-verbesserungen.md 260824; rules/betrieb-chronik.md 260914b; logbuch/LOGBUCH.md:13288ff

10.2.8 Letzte belastbare Abo-Auslastungsmessung: 23.08.2026 (42 %); seither Ampel SCHWARZ, Stand Statusdatei 06.09.2026. Quelle: logbuch/abo-check/status.json, ALARM.md

10.2.9 Claude-Abo: Max 20x, USD 216.20 pro Monat, zusätzlich Team-Abo mit 2 Seats (nicht separat messbar). Quelle: Memory reference_claude_abo; rules/betrieb-chronik.md 260813; logbuch/abo-check/status.json

10.2.10 Transkript-Archiv auf NAS: 5.2 GB, Zuwachs rund 600 MB pro Woche, Verfall 180 Tage. Quelle: du /Volumes/daten/06_Claude_Archiv/transcripts; rules/betrieb-chronik.md 260814d

10.2.11 Hub-Repo: rund 136 Commits pro Tag, Pack 271 MB; nur auf NAS (nicht in Git): Archimed 460 MB, behoerden-dokumente 27 MB, sync-tasks 3.8 MB, laeufe 1.6 MB. Quelle: 260911-Systemanalyse, Abschnitt 01.2

10.2.12 Shell-Scripts vs. Tests: 84 Scripts, 2 Test-Scripts, keine CI. Quelle: ls scripts/*.sh, 19.09.2026

10.2.13 Tenant-Hygiene-Reports: 38 Reports, täglicher Takt. Quelle: tenant-hygiene/reports/

10.2.14 Alarmkanäle: 1 (Apple Mail via osascript); mindestens 3 belegte stille Zustellausfälle (24.08., 25./26.08., 06.09.2026). Quelle: logbuch/kontingent/waechter.log; rules/betrieb-chronik.md 260826; logbuch/abo-check/UNZUSTELLBAR.md

10.3 Offene Fragen

10.3.1 Existiert das Hyper Backup tatsächlich? Wenn ja: Ziel (lokal/USB/Cloud/zweites NAS), Standort, Takt, Versionierung, Verschlüsselung, letzter erfolgreicher Lauf, je ein Restore durchgeführt? DSM war in diesem Lauf nicht einsehbar.

10.3.2 RAID-Typ bzw. SHR, Alter und SMART-Zustand der Platten, Dateisystem (Btrfs mit Snapshots?), DSM-Benachrichtigungen aktiv? USV für NAS, Mac Mini und OPNsense vorhanden?

10.3.3 Time Machine auf MacBook Pro und MacBook Revendo konfiguriert? Steht die externe Platte «Elements» des Mac Mini am selben Ort wie das NAS (kein Standortschutz)?

10.3.4 Gibt es für M365 Aufbewahrungsrichtlinien oder ein Drittanbieter-Backup ausserhalb des Hub-Repos? Welche Lizenzstufe liegt vor? Der M365-MCP-Connector war in dieser Session nicht verbunden (CONNECTION_CLOSED), der Tenant-Stand stammt aus dem Report vom 18.09.2026.

10.3.5 Wie werden bexio-Daten und die Website bzw. das Studio-Hosting beim Hoster gesichert?

10.3.6 Wo sind Secrets gesichert (Zertifikat gültig bis 23.03.2028, .env-Dateien, Recovery-Keys FileVault, M365-Admin-Notfallzugang)? Der Ordner secrets/ auf dem NAS existiert (1 Eintrag), Inhalt bewusst nicht gelesen.

10.3.7 Deckt das Lauf-Journal (logbuch/laeufe/) auch die Claude-Desktop-Scheduled-Tasks ab? Am 18.09.2026 enthält es nur 4 Zeilen, was auf eine Erfassungslücke hindeutet (nicht verifiziert).

10.3.8 Stand des internen Umbaus V0+/V2 (Git als einzige Quelle, Mac Mini als Zentrale): mehrere Schritte lagen per 11.09.2026 beim Inhaber (logbuch/fristen.md:40). Das Zielbild bestimmt das Backup-Design des Hub mit.

10.3.9 Ist der Stations-Watchdog und der Tailscale-Wächter inzwischen auf dem Mac Mini installiert?
Per 17.09.2026 (rules/auto-verbesserungen.md 260917b, Chronik 260917b) war der Mini noch offen.

10.3.10 Welche RPO/RTO sind aus Sicht des Inhabers tragbar (z.B. laufendes Bauprojekt mit Fristen), und wer vertritt ihn im Notfall? Davon hängen Umfang und Kosten der Offerte ab.

10.3.11 Gesamtkosten IT heute (M365-Lizenzen, bexio, Hosting, GitHub, Tailscale, Claude privat plus Team-Seats): im Repo nicht erhoben, für die Offerte beim Inhaber zu erfragen.